EQUIPO 4	
García Cerda Randal Guillermo (2112493 ITS) Sarmiento Hernández Cesar Martín (2119427 ITS) Torres Pasillas Angela Susana (2101844 ITS).	
Nombre del curso: Componentes de Lógica de Negocios	**Nombre del profesor**: Miguel Ángel Gómez Marroquín
Modulo Netacad: 5	**Actividad**: Unidad 5.
Fecha: 05/11/2026	
Bibliografia (Formato APA): Notas de clase	

Documentar cómo aplicamos la integración, las pruebas y el control de calidad en el proyecto de la unidad. En este reporte contamos el procedimiento que seguimos y los archivos que utilizamos, presentamos las evidencias de cada prueba con sus capturas, reunimos los resultados generales y cerramos con una matriz de prueba que relaciona cada regla con sus casos, su prueba automática y su evidencia.

Índice de figuras

A continuación se listan las capturas incluidas en el reporte, en el orden en que aparecen.

Fig.	Descripción
1	Instalación de dependencias y primera ejecución de la suite base (13 passed in 8.64s).
2	Lectura de src/domain.py y tests/test_order.py desde PowerShell.
3	Primer intento: el Bloc de notas no guardó los cambios y la suite sigue en 5 pruebas.
4	Segundo intento: alta de las pruebas nuevas con Add-Content.
5	Siete pruebas del dominio PASSED tras agregar los dos casos del ejercicio.
6	Fase RED: la prueba nueva falla porque el código aún permite confirmar dos veces.
7	Implementación mínima: guarda «already confirmed» al inicio de confirm().
8	Fase GREEN: la prueba nueva pasa y toda la suite sigue en verde (16 passed).
9	Dos pruebas originales PASSED y alta de test_repository_commit_order_250.
10	Tres pruebas de integración PASSED, incluida la del pedido de 250.
11	Tres pruebas del servicio PASSED, con la llamada esperada al mock.
12	Demostración del riesgo de consistencia: tras el rechazo el pedido queda CONFIRMED.
13	Seis pruebas de autorización PASSED, con fecha fija FIXED_NOW.
14	Primera ejecución de Ruff: errores I001 (imports sin ordenar) y UP017 (alias datetime.UTC).
15	Corrección con --fix y revisión limpia: «All checks passed!».
16	Resumen de cobertura: 20 pruebas PASSED y 100 % en las cinco unidades de src/.
17	Ejecución completa con apertura del reporte HTML (Invoke-Item htmlcov\index.html).
18	Reporte HTML de cobertura: 100 %, 53 declaraciones y 0 desaparecidas.
19	Ejecución #1 «Unidad 5: pruebas, TDD y CI»: roja.
20	Detalle del job de la ejecución #1: falla el paso de pytest.
21	Ejecución #2 «Corrige CI: pythonpath para que pytest encuentre src»: verde.
22	Ejecución #3 «Rompe a propósito una prueba para ver el CI en rojo»: roja.
23	Ejecución #4 «Revertir …»: verde.
24	Etapa 1 · /health devuelve 200 con {"status": "ok"}.
25	Etapa 1 · ejecución de docker compose --profile tests run --rm tests.
26	Etapa 1 · 4 pruebas y cobertura 100 %.
27	Etapa 1 · arranque de PostgreSQL 16.15 y docker compose down.
28	Etapa 2 · importador y oráculo: cobertura 95.65 %.
29	Etapa 3 · construcción de la imagen de pruebas.
30	Etapa 3 · dominio: cobertura 98.19 %.
31	Etapa 4 · construcción de la imagen de pruebas.
32	Etapa 4 · Specifications: cobertura 98.67 %.
33	Etapa 5 · construcción de la imagen de pruebas.
34	Etapa 5 · fin de la suite.
35	Etapa 5 · máquina de estados y capacidad: cobertura 98.91 %.
36	Etapa 6 · construcción de la imagen de pruebas.
37	Etapa 6 · 215 pruebas recolectadas.
38	Etapa 6 · Domain Events: 215 passed, cobertura 99.05 %.
39	Etapa 7 · construcción de la imagen de pruebas.
40	Etapa 7 · 300 pruebas recolectadas.
41	Etapa 7 · 300 passed, cobertura 99.12 %.
42	Etapa 7 · sembrado de datos.
43	Etapa 7 · sembrado repetido.
44	Etapa 8 · construcción de la imagen de la aplicación.
45	Etapa 8 · construcción y arranque de la suite.
46	Etapa 8 · 394 pruebas recolectadas.
47	Etapa 8 · 394 passed, cobertura 98.54 %.
48	Etapa 8 · evidencia de la API: 120/120, 20/20 y exportación OpenAPI.
49	Etapa 9 · reconstrucción, down -v y sembrado.
50	Etapa 9 · vacunación: 491 passed, cobertura 98.61 %.
51	Respaldo de la etapa 9, descompresión de la etapa 10 y listado de archivos nuevos.
52	Suite de la Etapa 10: 574 passed y cobertura 98.81 %.
53	Prueba dirigida con -k: 4 pruebas de emergencias PASSED en SQLite y PostgreSQL.
54	Reporte HTML de cobertura (1.ª parte).
55	Reporte HTML de cobertura (2.ª parte) con el total.
56	Sembrado idempotente: «La base ya tenía datos; no se cargó nada».
57	Swagger UI: secciones Sistema, Catálogo, Registro y Citas.
58	Swagger UI: Evaluaciones, Eventos, Vacunación y Emergencias.
59	Swagger UI: endpoints de emergencias y comienzo de los esquemas.

60	Swagger UI: lista de esquemas.
61	CU-05 · evaluación de atención antes de la emergencia: cumple = false.
62	CU-05 · POST /emergencias → 201 IDENTIFICADA.
63	CU-05 · evaluación de atención tras el alta: cumple = true.
64	CU-05 · GET /avisos-veterinario con el aviso registrado.
65	CU-05 · POST /emergencias sin mascota registrada → 201.
66	CU-05 · POST /emergencias como veterinario → 403.
67	CU-05 · iniciar-atencion como personal clínico → 200 EN_ATENCION.
68	CU-05 · iniciar-atencion repetido → 409 TRANSICION_INVALIDA.
69	CU-05 · GET /eventos con los eventos de dominio.
70	Errores de entrada · rol mal escrito → 400 ROL_DESCONOCIDO.
71	Errores de entrada · motivo en blanco → 422 MOTIVO_VACIO.
72	Consultas psql: tres emergencias, tres eventos y el esquema con restricciones.
73	Evidencia por HTTP: 20/20 escenarios, 120/120 casos y 20/20 transiciones; OpenAPI con 23 rutas.
74	api_emergencias.csv abierto en Excel: 20 escenarios, todos «coincide = True».
75	Ejecución #1 de ci.yml: lint, test y docker en verde (2 min 24 s).
76	Job test: se sube el artefacto cobertura.zip (5938 bytes).
77	Job test: se sube el artefacto evidencia.zip (14 350 bytes).
78	Registro del job test: pytest, generación de la evidencia y artefactos.
79	Resumen de la re-ejecución sobre el commit bc6cdd3: verde en 1 min 53 s.
80	Primer error: pytest no encuentra tests/test_u5_matriz.py dentro de Docker.
81	Segundo error: FileNotFoundError de docs/u5/matriz_pia_u5.csv en la imagen.
82	Suite completa con la matriz incorporada: 584 passed y cobertura 98.81 %.
83	Repositorio VetCare24_Unidad5_EQ4 creado en GitHub.
84	git push -u origin main solicita usuario y contraseña.
85	«Password authentication is not supported for Git operations».
86	Push rechazado por falta del permiso workflow.
87	Push exitoso: «[new branch] main -> main».
88	Ejecución final del laboratorio: 32 pruebas, cobertura del 100 % y revisión de Ruff en verde.

1. Procedimiento y archivos utilizados

1.1 Cómo abordamos la actividad

Durante esta unidad se trabajó con dos proyectos de manera complementaria. En primer lugar se utilizó el proyecto base de pedidos incluido en el laboratorio, cuyo tamaño reducido permitió aplicar cada técnica con resultados fáciles de anticipar: pruebas unitarias, desarrollo guiado por pruebas, integración con base de datos, simulación de servicios externos, autorización, cobertura e integración continua. Posteriormente, lo aprendido se aplicó al proyecto de la clínica veterinaria, VetCare 24, que el equipo había desarrollado por etapas y que incorpora reglas de negocio reales, como la capacidad de los servicios, el monto mínimo para acceder a un beneficio y el registro de emergencias sin cita.

El trabajo se realizó el 05/11/2026 en un equipo con Windows 11 y PowerShell. Las pruebas de VetCare 24 se ejecutaron dentro de contenedores mediante Docker Compose, dado que el proyecto utiliza una base de datos PostgreSQL y se buscaba que las pruebas tuvieran el mismo comportamiento en cualquier computadora del equipo. El proyecto base, por su parte, se ejecutó en un entorno virtual de Python.

Este documento describe el desarrollo de la actividad. En primer lugar se explican el procedimiento y los archivos utilizados; a continuación se presentan las evidencias con sus capturas; después se concentran los resultados generales; y, por último, se incluye la matriz de prueba, con las reglas verificadas, los casos, la distribución y las evidencias.

1.2 Entorno de trabajo

El proyecto base se ejecutó con Python 3.13, pytest y pytest-cov para medir la cobertura, Ruff para revisar el estilo del código y SQLAlchemy para la base de datos en memoria. Aunque el manual sugiere Python 3.11, se utilizó la versión instalada en el equipo sin que se presentaran inconvenientes; además, GitHub Actions emplea Python 3.11, por lo que el código quedó comprobado en ambas versiones. VetCare 24 se ejecuta con Python 3.12 dentro de Docker, con FastAPI, PostgreSQL 16 y SQLite para las pruebas de menor costo.

Ambos proyectos se alojan en GitHub: el laboratorio en el repositorio CLN_U5_Laboratorio_EQ4 y VetCare 24 en VetCare24_Unidad5_EQ4. En los dos se configuró un flujo de integración continua para que las verificaciones se repitan con cada envío de cambios.

1.3 Cómo está organizado lo que probamos

La separación en capas, presente tanto en el proyecto base como en VetCare 24, facilitó probar cada una de ellas sin depender de las demás. En el proyecto base, las reglas del pedido residen en un módulo de dominio que desconoce la base de datos y el cobro; el servicio de cobro recibe la pasarela de pago como parámetro, de modo que en las pruebas puede sustituirse por un objeto simulado; y únicamente el repositorio utiliza SQLAlchemy. VetCare 24 sigue el mismo principio con una estructura más amplia: dominio, aplicación, infraestructura y API.

Tabla 1. *Capas del software y forma de probarlas.*

Capa	En el proyecto base	En VetCare 24	Cómo la probamos
Dominio	Order y can_cancel	Citas, emergencias, reglas y especificaciones	Pruebas unitarias sin base de datos
Aplicación	OrderService	Servicios de citas, vacunación y emergencias	Servicio simulado y bases de datos reales
Infraestructura	Repositorio SQLAlchemy	Repositorios SQL, esquema y sembrado	Integración con SQLite y PostgreSQL
API	—	Rutas FastAPI y errores HTTP	Pruebas HTTP y recorrido de 20 escenarios

1.4 Archivos utilizados

En el proyecto base se trabajó principalmente con cuatro archivos de código y cuatro de pruebas. En cada paso del laboratorio se revisó el archivo correspondiente para comprender su función y, posteriormente, se agregaron o ejecutaron las pruebas. La siguiente tabla resume el contenido de cada archivo y el paso en que se utilizó.

Tabla 2. *Archivos del proyecto base.*

Archivo	Qué contiene	Paso
src/domain.py	La clase Order: valida el total, la deuda y la doble confirmación, y calcula el descuento VIP del 15 %	E2, E3
src/application.py	El servicio de checkout, que cobra con la pasarela que recibe	E5
src/repository.py	La tabla de pedidos, el repositorio y la base SQLite en memoria	E4

src/security.py	La política para cancelar pedidos	E6
tests/test_order.py	8 pruebas del dominio (5 originales y 3 agregadas por el equipo)	E1–E3
tests/test_repository.py	3 pruebas de integración con la base	E4
tests/test_service.py	3 pruebas con la pasarela simulada	E5
tests/test_security.py	6 pruebas de autorización	E6
pyproject.toml	Configuración de pytest, cobertura y Ruff	E1, E7, E8
requirements.txt	Dependencias del proyecto	E1
.github/workflows/ci.yml	Flujo de integración continua «CI Business Logic»	E8
.gitignore	Evita subir el entorno virtual y archivos generados	E8
htmlcov/index.html	Reporte de cobertura que genera pytest-cov	E7

En VetCare 24, el trabajo de esta unidad se concentró en la etapa de emergencias y en la matriz de pruebas. Además de los archivos de la lógica de negocio, se emplearon los de pruebas, los de configuración de Docker y los archivos de evidencia que genera el propio proyecto. Considerando todas sus etapas, el proyecto reúne 584 pruebas distribuidas en 31 archivos.

Tabla 3. *Archivos de VetCare 24 relacionados con esta unidad.*

Archivo	Para qué lo usamos
domain/emergencia.py y reglas_emergencia.py	Definen cuándo una emergencia es válida y cuándo una mascota puede recibir atención (cita confirmada o emergencia activa)
domain/reglas_cita.py	Contiene las reglas de capacidad suficiente y de total mínimo de $1500
application/servicios_emergencias.py	Registra la emergencia, avisa al veterinario y permite iniciar la atención
infrastructure/notificador.py	Envía el aviso al veterinario; en las pruebas lo cambiamos por uno que falla
api/rutas.py y errores_http.py	Exponen los endpoints y traducen los errores a códigos HTTP
tests/test_emergencia.py	28 pruebas del agregado de emergencias
tests/test_servicios_emergencias.py	30 pruebas de los servicios, la mitad con SQLite y la mitad con PostgreSQL
tests/test_api_emergencias.py	20 pruebas de la API de emergencias
tests/test_u5_matriz.py	10 pruebas que leen la matriz CSV y ejecutan los casos CP-01 a CP-09
docs/u5/matriz_pia_u5.csv	La matriz de 11 casos que diseñamos para esta unidad
Dockerfile y docker-compose.yml	Permiten construir la imagen de pruebas y levantar la base de datos
.github/workflows/ci.yml	Flujo de integración continua con tres trabajos: revisión de estilo, pruebas y construcción de imágenes

Los archivos de evidencia se generan mediante comandos propios del proyecto: el oráculo produce los resultados esperados de 120 casos y 20 transiciones; la evidencia de la API contrasta esos resultados con las respuestas del servicio por HTTP; y la evidencia de emergencias ejecuta 20 escenarios y los registra en

api_emergencias.csv, junto con el contrato OpenAPI y el reporte de cobertura. Todos se almacenan en la carpeta docs/evidencia.

1.5 Procedimiento que seguimos

Se siguieron en orden los nueve pasos del laboratorio. A continuación se describe lo realizado en cada uno y el resultado que se esperaba obtener.

Paso 1 · Instalar y comprobar el proyecto

Se creó un entorno virtual, se activó, se instalaron las dependencias y se ejecutaron todas las pruebas antes de realizar modificaciones. Esto estableció una línea base: cualquier falla posterior podría atribuirse a un cambio del equipo y no al entorno.

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pytest -v
```

Resultado esperado: Se esperaban 13 pruebas exitosas, resultado que se obtuvo.

Paso 2 · Prueba unitaria

Se analizó el método Order.confirm() y se agregaron dos pruebas: un pedido VIP de 200 con cobro esperado de 170 y un pedido regular de 200 con cobro esperado de 200. Para incorporarlas se utilizó el comando Add-Content de PowerShell, debido a que el primer intento mediante el Bloc de notas no guardó los cambios.

```
Get-Content src\domain.py
Add-Content -Path tests\test_order.py -Encoding utf8 -Value @'
...
'@
python -m pytest tests/test_order.py -v
```

Resultado esperado: Las pruebas del archivo debían pasar de 5 a 7.

Paso 3 · Desarrollo guiado por pruebas

Se definió una regla nueva, no confirmar un pedido que ya fue confirmado, y se redactó primero la prueba. Esta se ejecutó para comprobar su falla; después se incorporó la validación mínima en el código y se volvió a ejecutar.

```
python -m pytest tests/test_order.py -v   # RED
# agregamos la guarda en src/domain.py
python -m pytest tests/test_order.py -v   # GREEN
```

```
python -m pytest -v
```

Resultado esperado: Primero una falla y después todas las pruebas en verde.

Paso 4 · Base de datos y transacciones

Se revisó cómo el repositorio almacena pedidos en una base SQLite en memoria y se comprobó la diferencia entre confirmar y deshacer una transacción. Se agregó una prueba que guarda un pedido de 250 con id 2.

```
python -m pytest tests/test_repository.py -v
```

Resultado esperado: Tres pruebas pasando.

Paso 5 · Servicios externos

Se sustituyó la pasarela de pago por un objeto simulado y se probaron el cobro aprobado y el rechazado. Se añadió el caso de un pedido VIP de 200 rechazado, en el que el cobro intentado debía ser de 170.

```
python -m pytest tests/test_service.py -v
```

Resultado esperado: Tres pruebas pasando y el riesgo de consistencia documentado.

Paso 6 · Seguridad y autorización

Se verificó quién puede cancelar un pedido. Se añadieron el caso del administrador que cancela un pedido nuevo de otro usuario y el del propietario que intenta cancelar exactamente a las 24 horas, empleando una fecha fija para que el resultado no dependa del reloj.

```
python -m pytest tests/test_security.py -v
```

Resultado esperado: Seis pruebas pasando.

Paso 7 · Cobertura y calidad

Primero se corrigieron los avisos de estilo reportados por Ruff y después se midió la cobertura con un mínimo del 80 %, generando además el reporte HTML.

```
python -m ruff check src tests --fix
python -m pytest -v --cov=src --cov-report=term-missing --cov-report=html --cov-
fail-under=80
Invoke-Item htmlcov\index.html
```

Resultado esperado: Ruff sin errores y cobertura por encima del 80 %.

Paso 8 · GitHub Actions

Se subió el proyecto a GitHub y se observó el flujo automático. La primera ejecución resultó fallida y fue corregida; posteriormente se introdujo una falla intencional para comprobar que el flujo la detectara y, finalmente, se revirtió el cambio.

```
git init
git add .
git commit -m "Unidad 5: pruebas y CI"
git branch -M main
git remote add origin <URL>
git push -u origin main
git revert <commit>
```

Resultado esperado: Ejecuciones verde, roja y de nuevo verde.

Paso 9 · Aplicación a VetCare 24

Se seleccionaron tres reglas reales del proyecto y se documentó para cada una un caso de éxito, uno de error y uno de límite en un archivo CSV. Se escribió una prueba que lee dicho archivo y ejecuta cada caso, y se corrió dentro de Docker junto con el resto de la suite.

```
docker compose --profile tests build tests
docker compose --profile tests run --rm tests
```

Resultado esperado: Toda la suite pasando con cobertura superior al 80 %.

1.6 Cómo diseñamos las pruebas

Aunque se trata de proyectos distintos, en ambos se aplicaron los mismos criterios. Cada prueba sigue el patrón de preparar, actuar y verificar, y su nombre indica la regla que comprueba y el escenario, por ejemplo test_owner_exactly_24h_denied. Los valores esperados se calcularon manualmente y no se tomaron del resultado del propio código, con el fin de evitar que un error se valide a sí mismo: el 15 % de 200 es 30, por lo que el pedido VIP debe cobrar 170.

Asimismo, se procuró probar siempre el valor exacto en el límite de cada regla, pues es allí donde suelen presentarse los errores por una unidad de diferencia: las 24 horas exactas de la cancelación, la cantidad igual a la capacidad (10 de 10) o el total que justamente alcanza $1500. En los casos de error se comparó el código de la violación y no el mensaje de texto, para que un cambio de redacción

no afecte las pruebas. Además, cada prueba parte de una base vacía en memoria, por lo que en ningún momento se manipularon datos reales.

Para clasificar los casos se emplearon cuatro tipos: **éxito** cuando se comprueba el camino normal, **error** cuando una entrada inválida debe rechazarse, **límite** cuando se prueba el valor exacto donde cambia el comportamiento e **integración** cuando varias piezas trabajan juntas, como el repositorio con la base de datos.

1.7 Integración continua

En el laboratorio se configuró el flujo «CI Business Logic», que con cada envío de cambios descarga el código, instala Python 3.11 y las dependencias, ejecuta Ruff, corre las pruebas con un mínimo de cobertura del 80 % y compila el código. VetCare 24 cuenta con su propio flujo de tres trabajos encadenados: primero revisa el estilo; luego ejecuta las pruebas, genera la evidencia (los 120 casos, las 20 transiciones y los escenarios de emergencias) y publica dos archivos descargables con la cobertura y la evidencia; y finalmente construye las imágenes de Docker.

Cabe señalar que contar con el archivo del flujo en el repositorio provoca que las verificaciones se ejecuten, pero no impide por sí solo que se fusione código cuando el flujo está en rojo. Para bloquear la fusión es necesario configurar reglas de protección de rama en GitHub, las cuales no se activaron en este laboratorio.

1.8 Cómo manejamos las evidencias

Todas las capturas corresponden a ejecuciones reales del equipo y únicamente se recortaron para eliminar la barra de tareas y el espacio vacío de la terminal. Cada una lleva su número de figura y un texto que explica lo que se observa y lo que demuestra. No se incluyeron entornos virtuales, claves ni datos personales, y el token que GitHub exige en lugar de la contraseña se escribió una sola vez en la terminal, sin aparecer en ninguna captura. Los errores encontrados durante el proceso también se conservaron como evidencia, ya que muestran cómo se diagnosticaron y se resolvieron.

2. Evidencias

A continuación se presentan las evidencias de las actividades realizadas durante la unidad. Para cada paso se explica primero el propósito, se muestra la captura correspondiente y se comenta lo que en ella se observa. Las figuras llevan numeración corrida y al inicio del documento se incluye un índice que facilita su localización.

2.1 Preparación del entorno (E1)

La primera actividad consistió en comprobar que el proyecto funcionaba antes de modificarlo. Se creó el entorno virtual, se instalaron las dependencias y se ejecutaron todas las pruebas en su estado original. Esta ejecución sirvió de referencia: cualquier falla posterior podría atribuirse a un cambio realizado por el equipo.

En la captura se ve la instalación de pytest 9.2.0, pytest-cov 7.1.0, SQLAlchemy 2.1.8 y Ruff 0.17.3, la ruta del intérprete (.venv\Scripts\python.exe) y la recolección de 13 pruebas, que avanzan hasta el 100 %. Con ello se comprueba que el entorno virtual es funcional y que el proyecto entregado pasa antes de cualquier cambio.

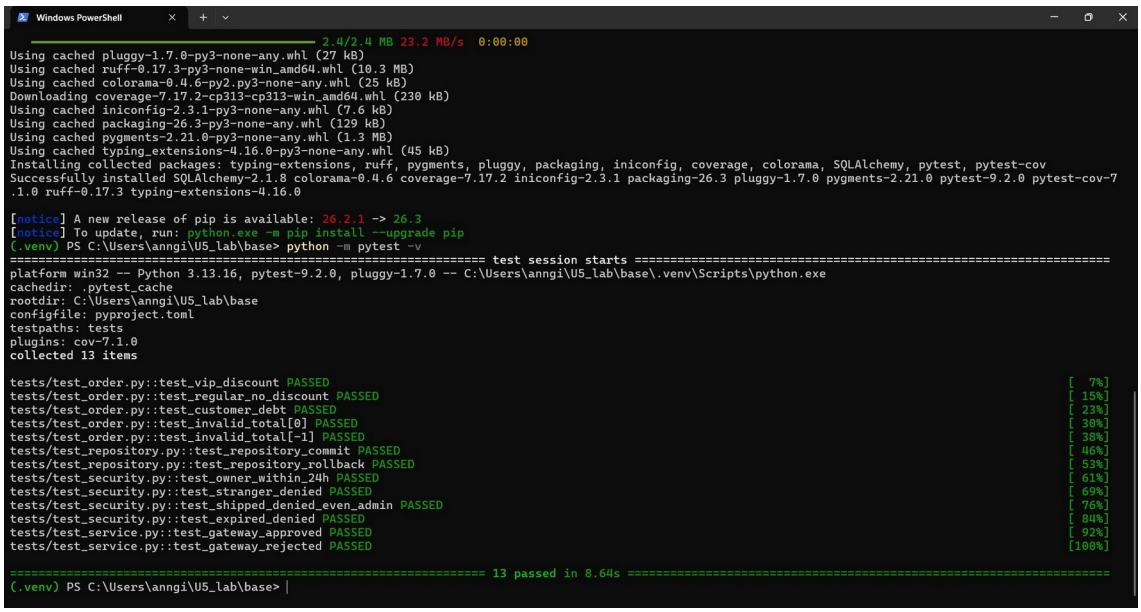


Figura 1. *Instalación de dependencias y primera ejecución de la suite base (13 passed in 8.64s).*

2.2 Pruebas unitarias del dominio (E2)

Antes de escribir pruebas se analizó el código a verificar. El método Order.confirm() revisa primero el total y la deuda del cliente, luego aplica el descuento VIP del 15 % y, por último, cambia el estado a confirmado. Con ello definido, se agregaron las dos pruebas solicitadas en el ejercicio siguiendo el patrón de preparar, actuar y verificar.

Muestra la clase Order, la excepción CustomerHasDebt y las cuatro pruebas originales del archivo. En la parte superior se conserva la línea de la ejecución anterior («13 passed in 8.64s»). Sirve para dejar documentado el código que se probó.

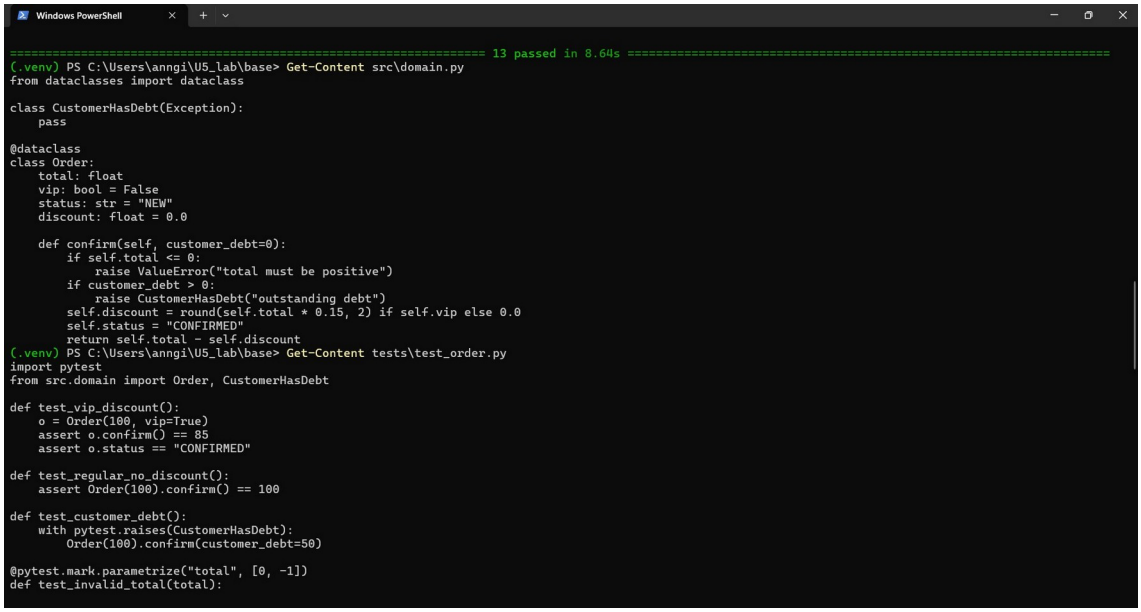


Figura 2. *Lectura de src/domain.py y tests/test_order.py desde PowerShell.*

La ejecución indica «collected 5 items … 5 passed in 0.10s» cuando se esperaban 7. Es un error de procedimiento detectado gracias a contar las pruebas recolectadas; demuestra la importancia de verificar que el número de pruebas coincide con lo esperado.

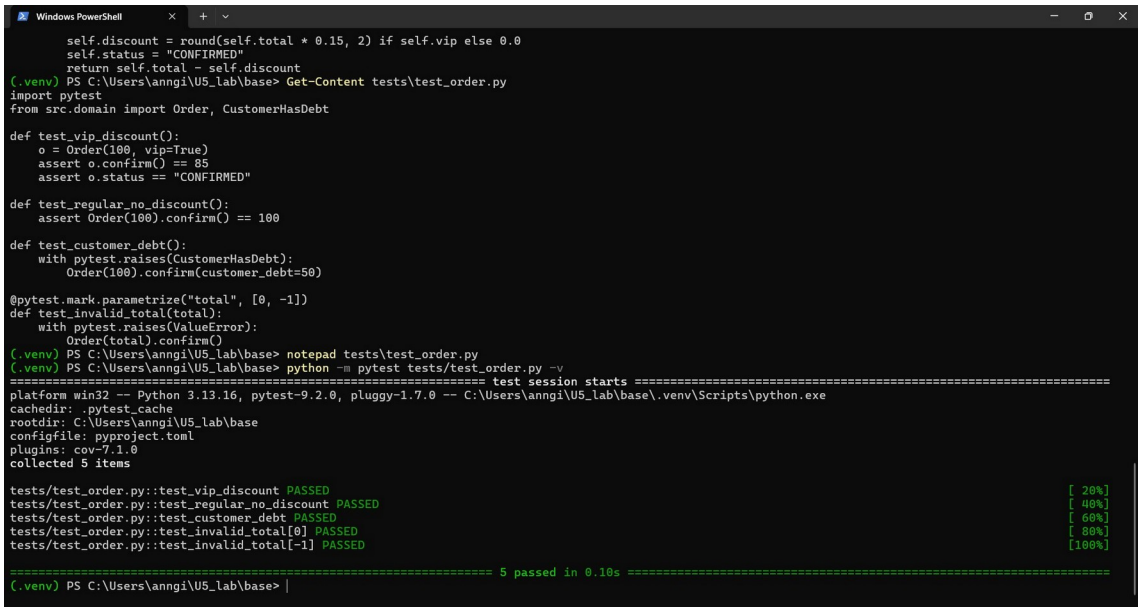


Figura 3. *Primer intento: el Bloc de notas no guardó los cambios y la suite sigue en 5 pruebas.*

En la captura se observa la salida previa (5 passed) y el comando Add-Content que agrega test_vip_discount_200 y el inicio de test_regular_customer_200_no_discount. Este método escribe el texto directamente al final del archivo, sin depender de un editor.

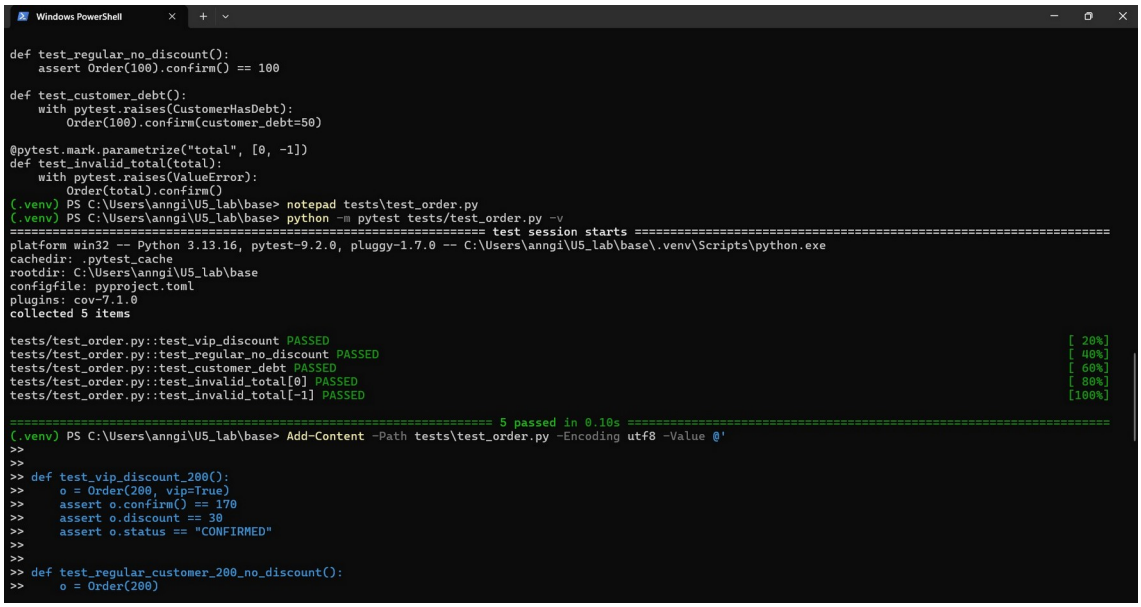


Figura 4. *Segundo intento: alta de las pruebas nuevas con Add-Content.*

En la captura se lee el código de las dos pruebas nuevas y debajo la ejecución: «collected 7 items … 7 passed in 0.13s». Con ello se comprueba que el VIP de 200 paga 170 con 30 de descuento y que el cliente regular de 200 paga 200 sin descuento.

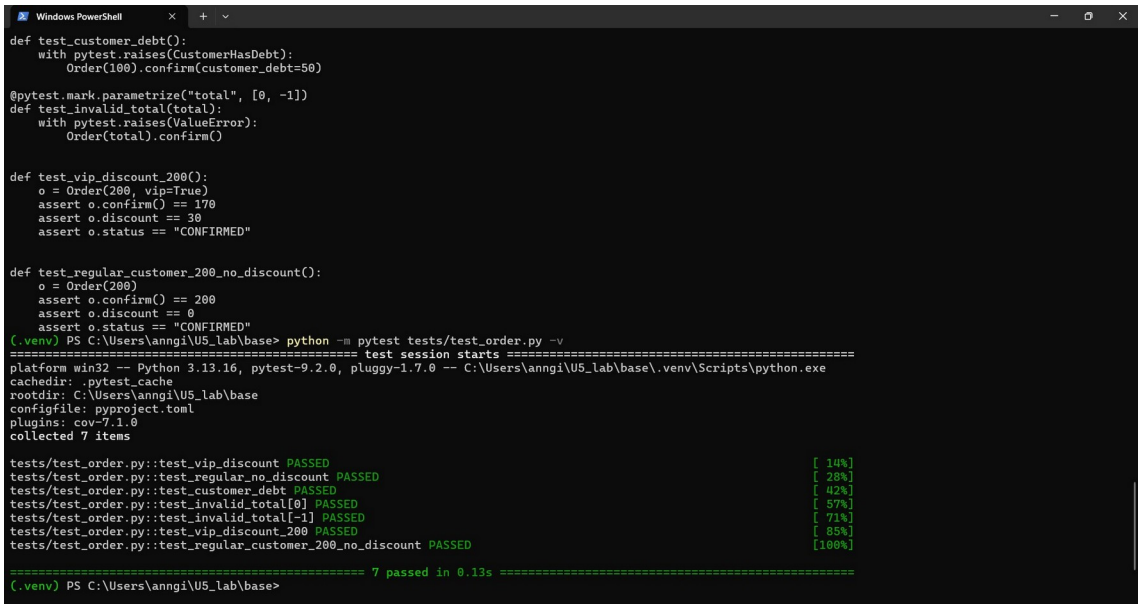


Figura 5. *Siete pruebas del dominio PASSED tras agregar los dos casos del ejercicio.*

2.3 TDD: Red, Green y Refactor (E3)

Para aplicar el desarrollo guiado por pruebas se eligió la regla de no confirmar un pedido que ya está confirmado. Dado que el proyecto ya validaba que el total fuera positivo, repetir esa regla no habría producido una fase roja genuina. Se redactó entonces la prueba, se ejecutó para comprobar su falla, se añadió el código mínimo y se volvió a correr la suite completa.

La ejecución reporta «collected 8 items … 1 failed, 7 passed» y el mensaje «Failed: DID NOT RAISE ValueError» en test_cannot_confirm_twice. Con ello se comprueba que la prueba es capaz de detectar el defecto antes de la implementación, que es la razón de ser del ciclo.

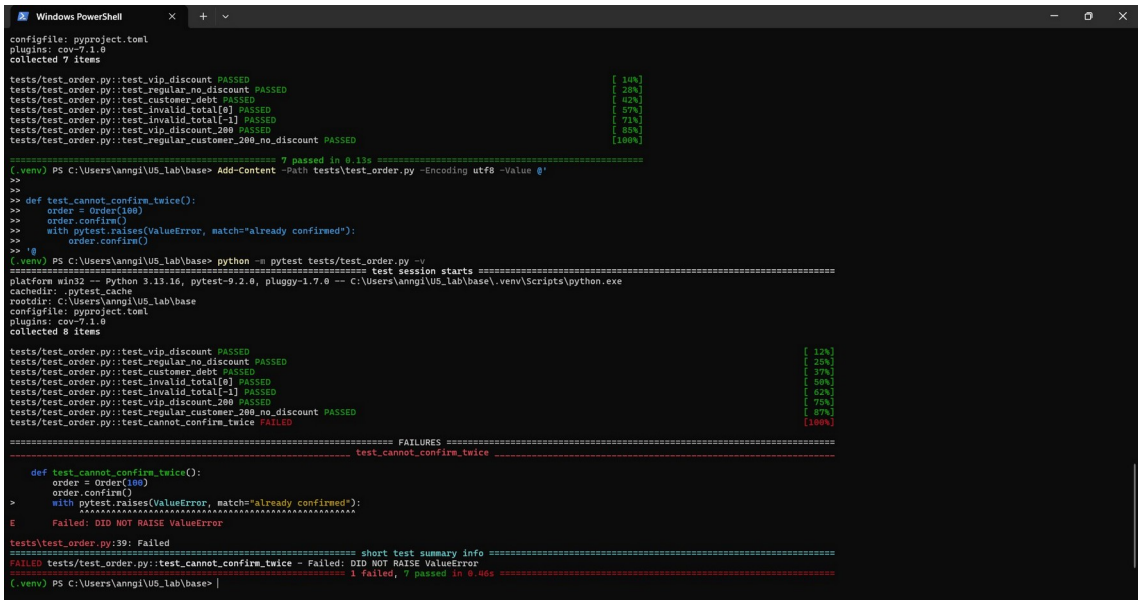


Figura 6. *Fase RED: la prueba nueva falla porque el código aún permite confirmar dos veces.*

Get-Content src\domain.py muestra la guarda como primera instrucción del método. El error rojo del final es un ParserError causado por pegar en la consola, por error, líneas de referencia del manual; no afectó al archivo.

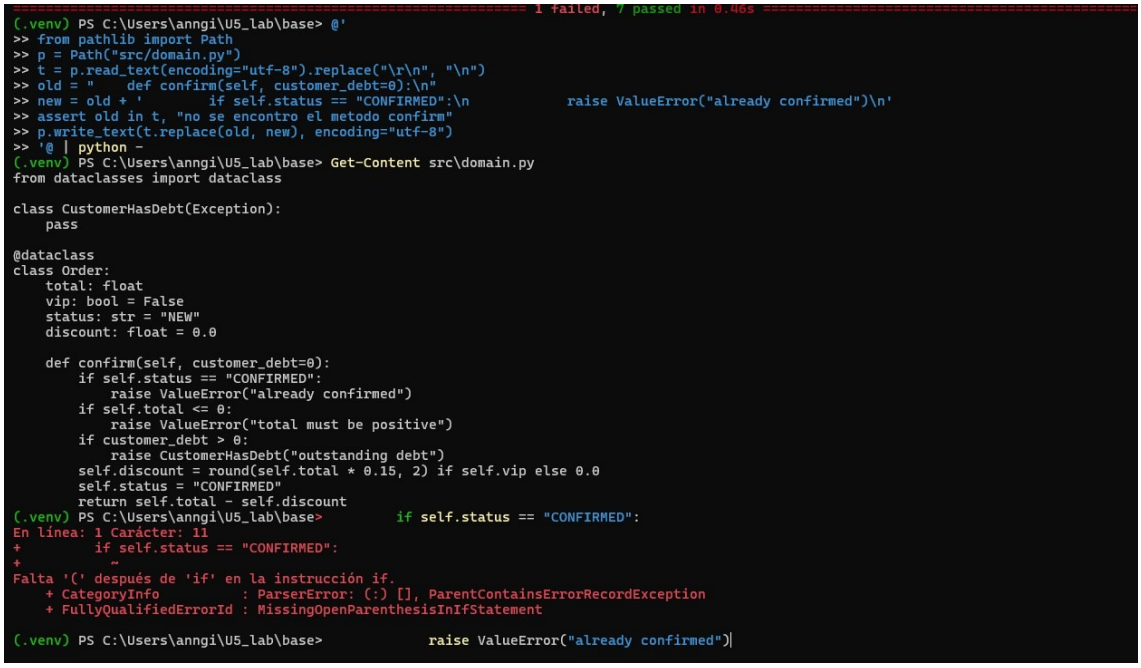


Figura 7. *Implementación mínima: guarda «already confirmed» al inicio de confirm().*

Primero se ejecuta test_order.py («8 passed in 0.15s») y luego toda la suite («collected 16 items … 16 passed in 1.16s»). Con ello se comprueba que el cambio mínimo satisface la prueba sin romper ninguna regla anterior. En la fase REFACTOR no se modificó el código porque confirm() tiene cuatro guardas de una línea y es legible.

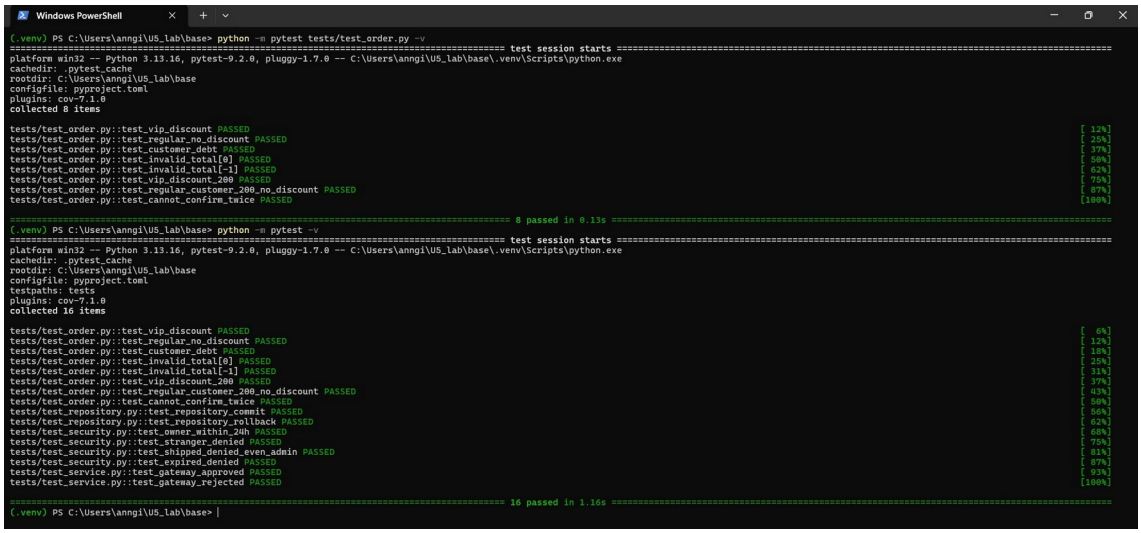


Figura 8. *Fase GREEN: la prueba nueva pasa y toda la suite sigue en verde (16 passed).*

2.4 Integración con base de datos y transacciones (E4)

En esta etapa intervino la base de datos. El repositorio crea una base SQLite en memoria, por lo que cada prueba parte de un estado limpio y no se manipulan datos reales. Las dos pruebas originales comprueban que un commit conserva el pedido y que un rollback lo descarta; el equipo agregó una tercera con un pedido de 250 y el id 2.

La captura muestra «2 passed in 1.07s», el Add-Content con el código de la prueba nueva y el listado de test_repository.py con las pruebas de commit y rollback. Esto demuestra el punto de partida y el procedimiento de ampliación.

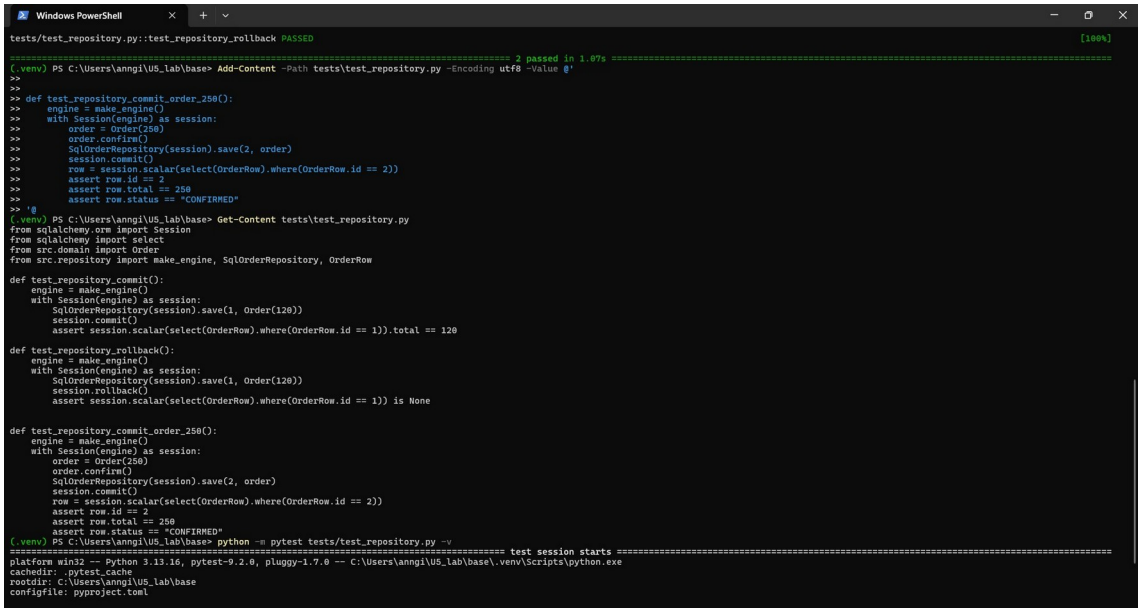


Figura 9. *Dos pruebas originales PASSED y alta de test_repository_commit_order_250.*

El archivo muestra la prueba nueva al final y la ejecución «collected 3 items … 3 passed in 1.07s». La prueba verifica id == 2, total == 250 y status == "CONFIRMED" tras el commit, es decir, que el repositorio persiste el estado del pedido.

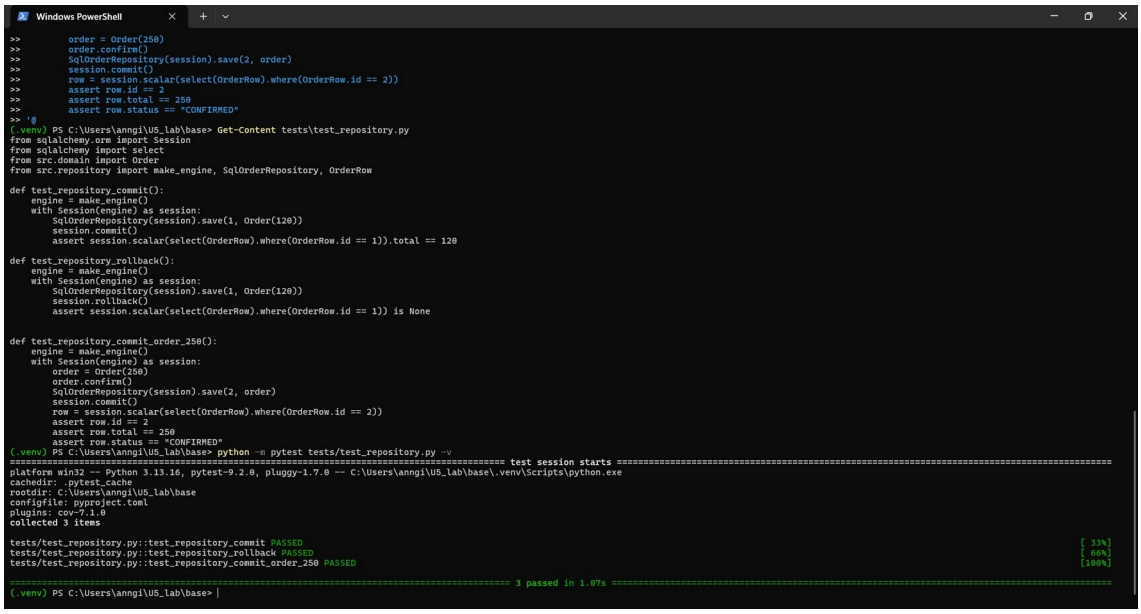


Figura 10. *Tres pruebas de integración PASSED, incluida la del pedido de 250.*

2.5 Servicios externos con mock (E5)

El servicio de checkout cobra a través de una pasarela de pago. Para evitar cobros reales, en las pruebas se utiliza un objeto simulado que responde de forma afirmativa o negativa y que permite verificar con qué importe fue invocado. La aportación del equipo fue la prueba de rechazo con un pedido VIP de 200, en la que el cobro esperado era de 170.

En la captura se ve el archivo completo con test_gateway_approved, test_gateway_rejected y test_gateway_rejected_vip_200 y la ejecución «3 passed in 0.09s». La aserción assert_called_once_with(170) demuestra que se intentó cobrar exactamente el importe con descuento y una sola vez.

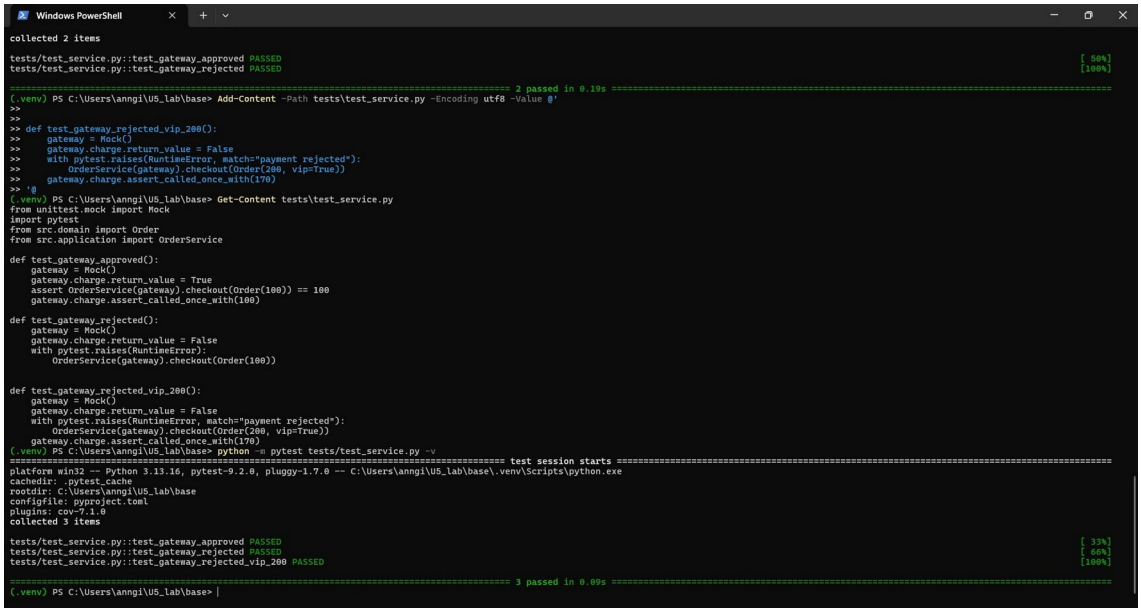


Figura 11. *Tres pruebas del servicio PASSED, con la llamada esperada al mock.*

La comprobación con python -c imprime «payment rejected» y «estado del pedido: CONFIRMED». Esto evidencia que el código confirma el pedido antes de saber si el pago fue aceptado; se documenta como propuesta de mejora (calcular, cobrar y solo entonces confirmar).

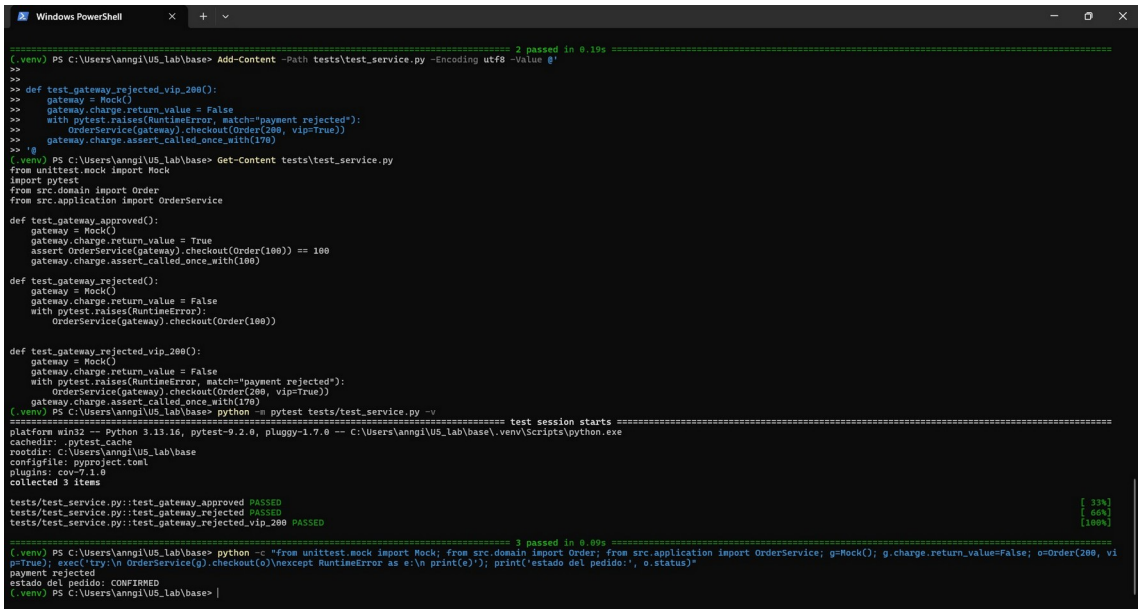


Figura 12. *Demostración del riesgo de consistencia: tras el rechazo el pedido queda CONFIRMED.*

2.6 Seguridad y autorización (E6)

En este paso se verificó quién puede cancelar un pedido. La función evalúa tres condiciones en orden: un pedido enviado nunca se cancela, el administrador puede cancelar en los demás casos y el propietario únicamente si no han transcurrido 24 horas. Para que las pruebas no dependieran del reloj del equipo se utilizó una fecha fija.

En la captura se observan las cuatro pruebas originales («4 passed in 0.04s»), el alta de las dos nuevas (administrador sobre pedido NEW de otro usuario; propietario a las 24 horas exactas) y la ejecución final «6 passed in 0.05s». El caso de las 24 horas exactas demuestra que el límite es estricto (menor que 24 h).

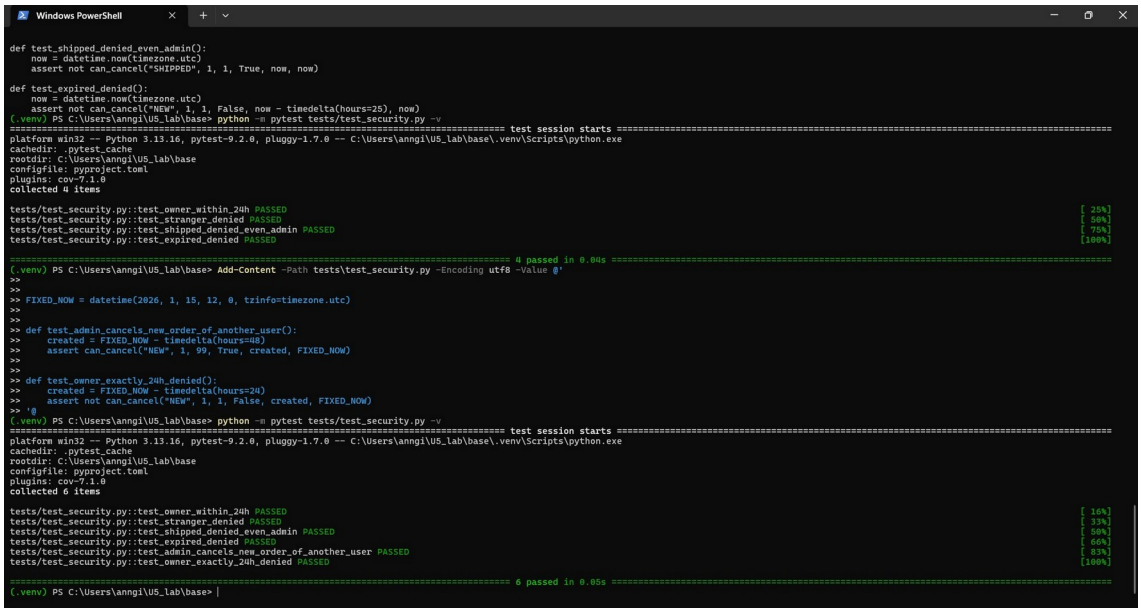


Figura 13. *Seis pruebas de autorización PASSED, con fecha fija FIXED_NOW.*

2.7 Cobertura y calidad (E7)

Antes de medir la cobertura se ejecutó Ruff, que señaló 13 avisos de estilo corregibles de forma automática. Una vez depurado el código, se midió la cobertura con pytest-cov exigiendo un mínimo del 80 %.

En la captura se listan los avisos por archivo (domain.py, repository.py, security.py). No son fallos de lógica, pero el pipeline ejecuta el mismo comando y habría quedado en rojo; por eso se corrigieron antes del primer push.

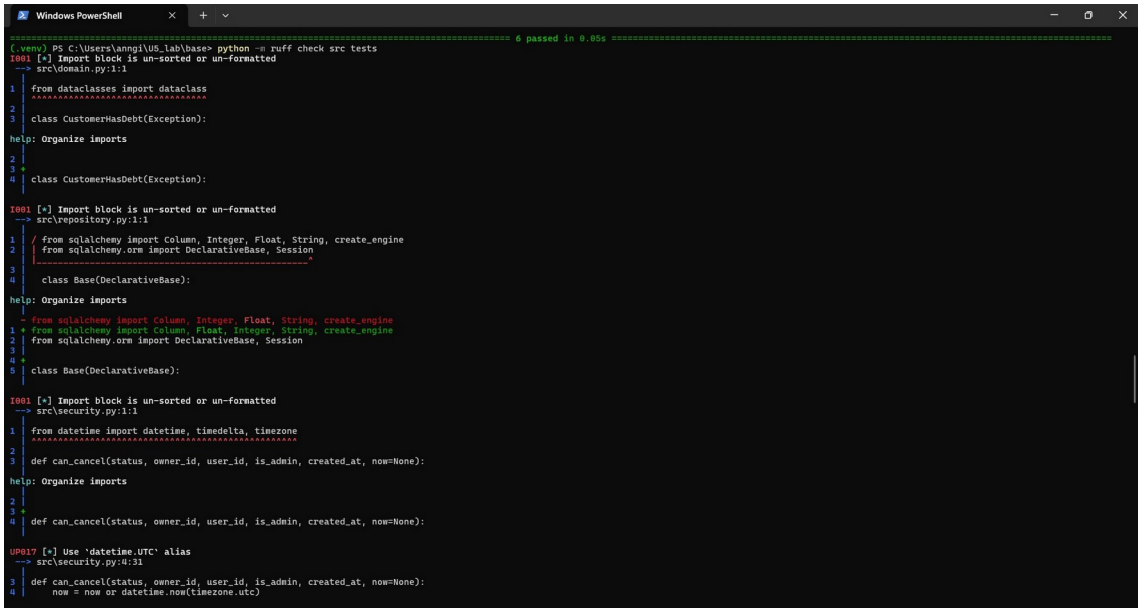


Figura 14. *Primera ejecución de Ruff: errores I001 (imports sin ordenar) y UP017 (alias datetime.UTC).*

Al final del listado aparece «Found 13 errors», el comando con --fix («Found 15 errors (15 fixed, 0 remaining)») y la segunda revisión sin avisos. Con ello se comprueba que el análisis estático quedó en verde.

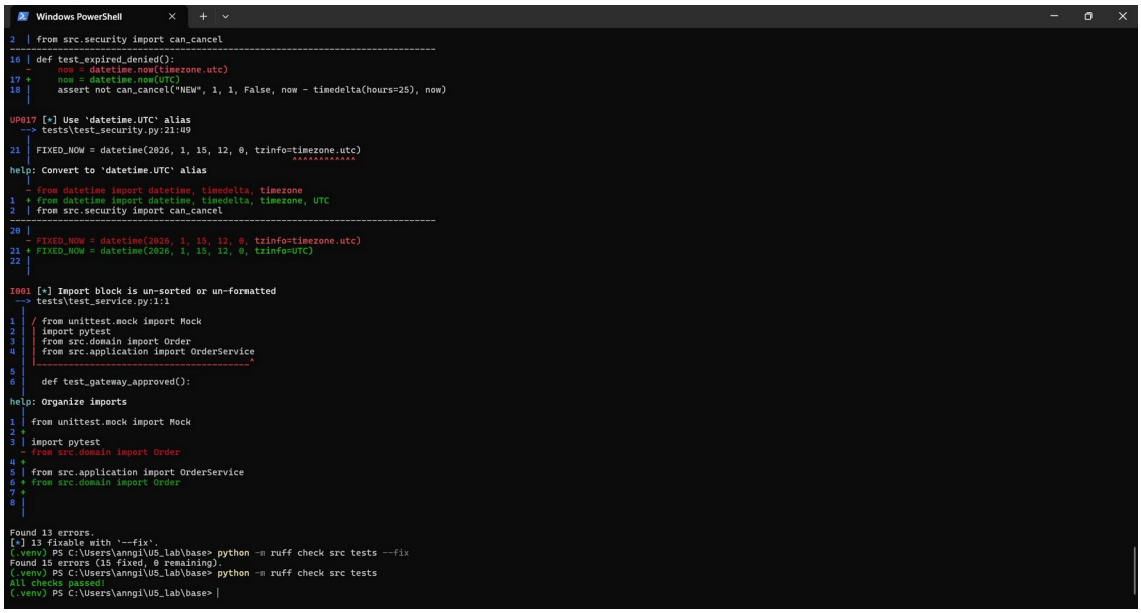


Figura 15. *Corrección con --fix y revisión limpia: «All checks passed!».*

La tabla por archivo reporta TOTAL de 53 sentencias, 0 sin cubrir, y «Required test coverage of 80% reached. Total coverage: 100.00%». Con ello se comprueba que la suite completa supera el umbral con holgura sin excluir módulos.

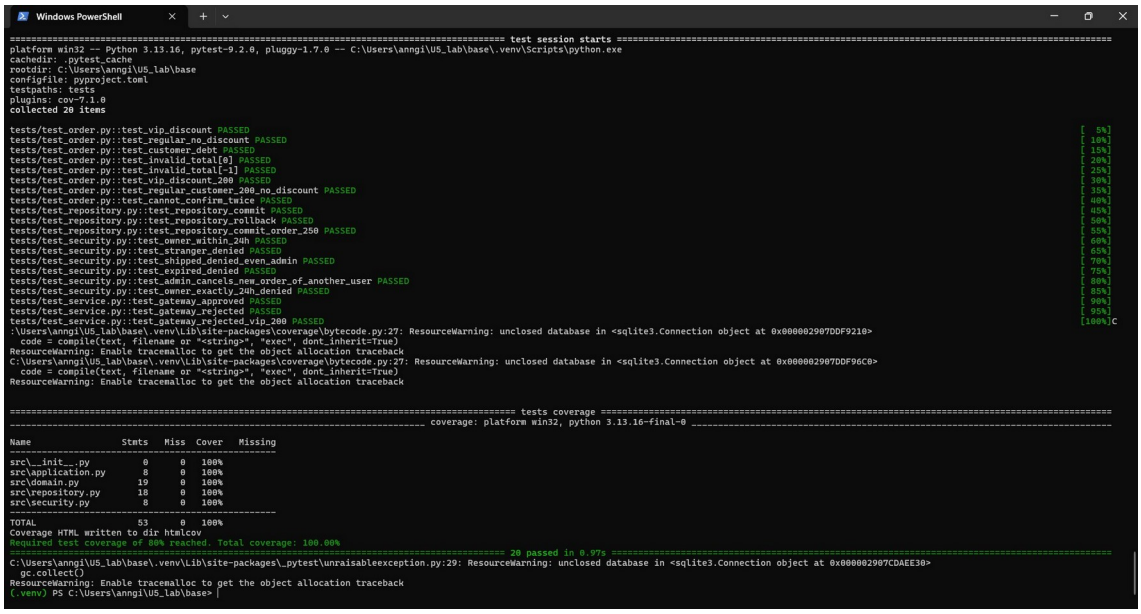


Figura 16. *Resumen de cobertura: 20 pruebas PASSED y 100 % en las cinco unidades de src/.*

Es la misma ejecución con la línea final que abre el reporte en el navegador. Queda documentado el procedimiento para visualizar la cobertura archivo por archivo.

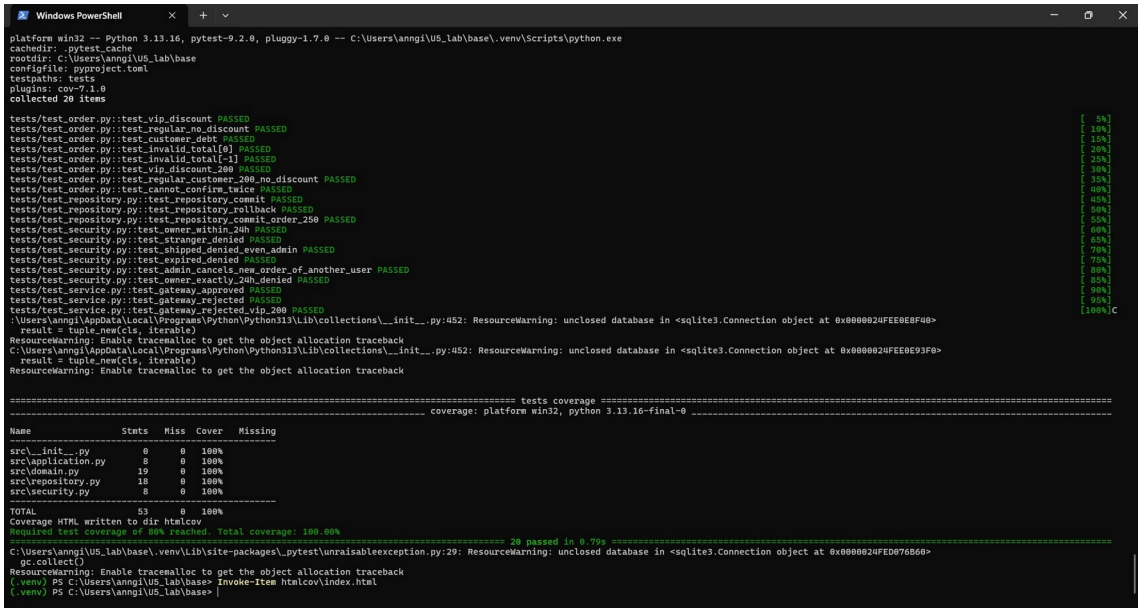


Figura 17. *Ejecución completa con apertura del reporte HTML (Invoke-Item htmlcov\ index.html).*

El reporte generado por coverage.py 7.17.2 lista src/__init__.py, application.py, domain.py, repository.py y security.py al 100 %. Complementa la captura de terminal con la vista del navegador.

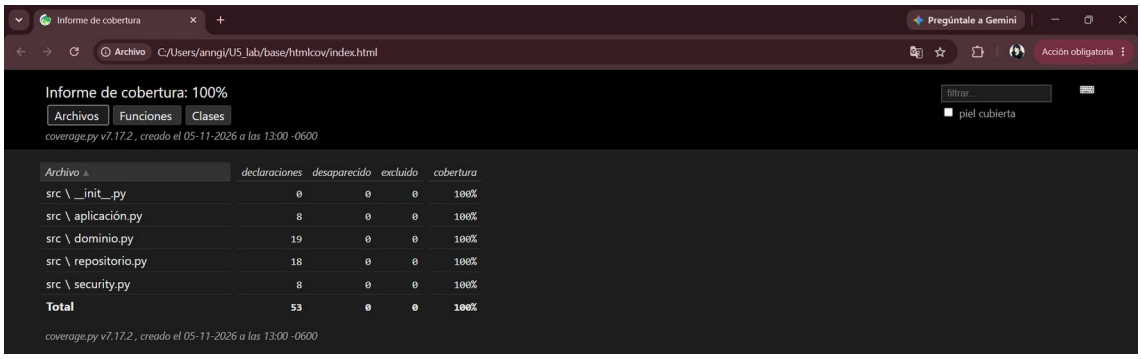


Figura 18. *Reporte HTML de cobertura: 100 %, 53 declaraciones y 0 desaparecidas.*

Aunque el resultado fue del 100 %, debe considerarse que esa cifra solo indica qué líneas se ejecutaron y no qué escenarios se comprobaron. Por ejemplo, no se probó el comportamiento ante una excepción de la pasarela durante el cobro, ante un id repetido ni ante la combinación de un total no positivo con una deuda. Dichos casos quedaron anotados como trabajo futuro.

2.8 Integración continua en GitHub Actions (E8)

El proyecto se subió al repositorio CLN_U5_Laboratorio_EQ4 y el flujo «CI Business Logic» se ejecutó con cada envío. El historial resultante fue el siguiente: la ejecución 1 en rojo por un problema real, la 2 en verde tras su corrección, la 3 en rojo debido a una prueba alterada intencionalmente y la 4 en verde después de revertir ese cambio.

El trabajo «test» falla en 13 segundos y GitHub muestra «1 error, 1 advertencia y 1 aviso». Es el primer resultado del pipeline, que reveló un problema que en la máquina local no se veía.

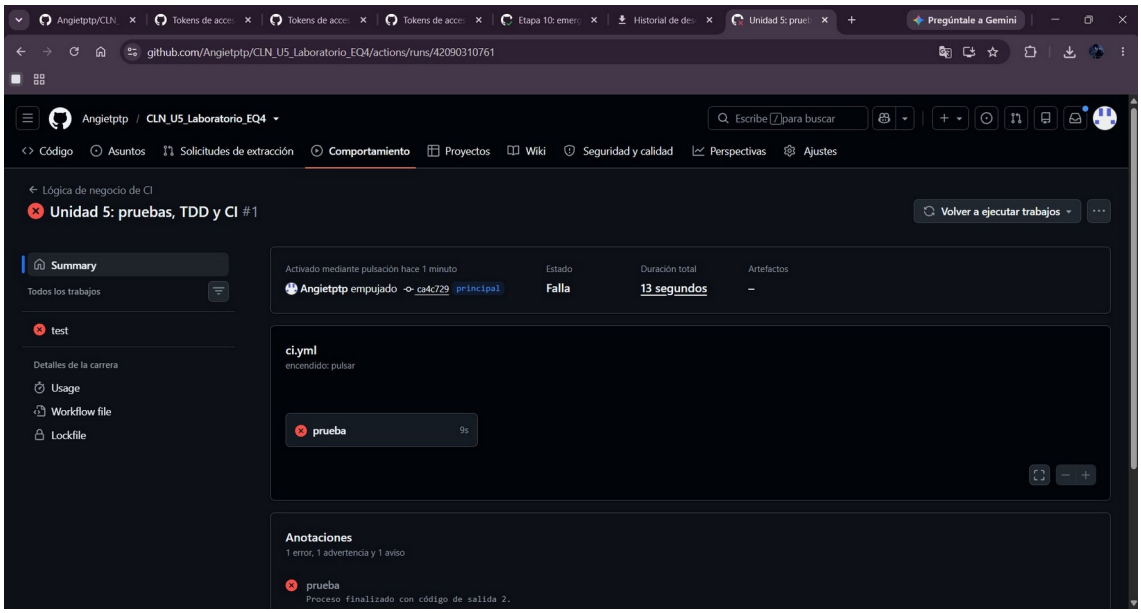


Figura 19. *Ejecución #1 «Unidad 5: pruebas, TDD y CI»: roja.*

Los pasos de checkout, setup-python, pip install y ruff terminan en verde; falla «Run pytest -v --cov=src …». La causa fue que el workflow ejecuta pytest sin añadir la carpeta del proyecto al path y no encontraba el paquete src; se corrigió con pythonpath = ["."] en pyproject.toml.

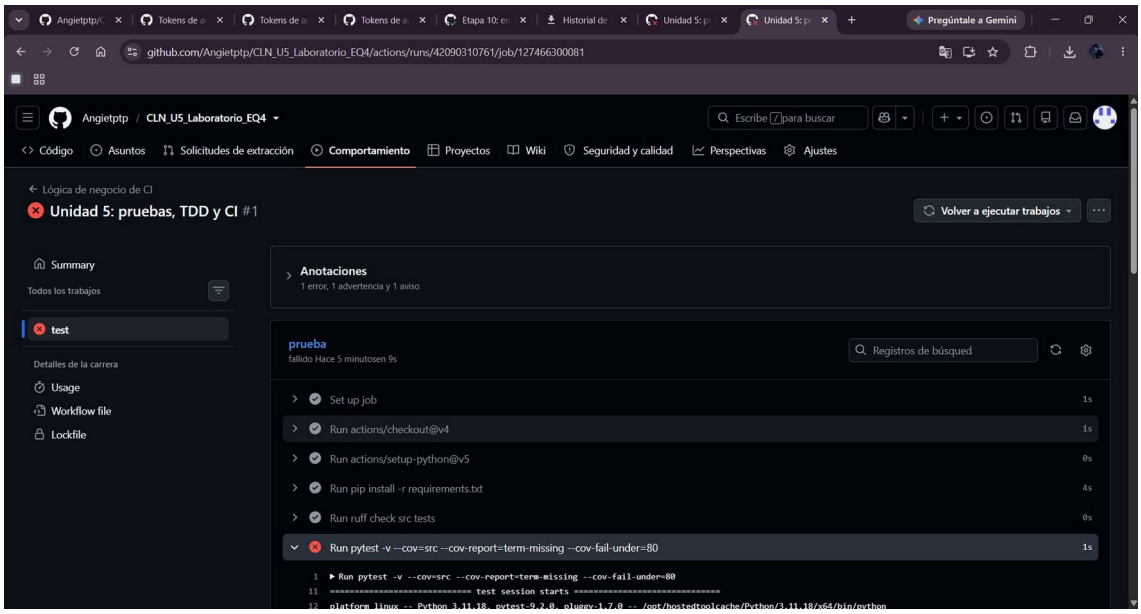


Figura 20. *Detalle del job de la ejecución #1: falla el paso de pytest.*

El job «prueba» termina con éxito en 11 segundos con todos los pasos en verde (checkout, setup-python, pip install, ruff, pytest con cobertura y compileall). Con ello se comprueba que la corrección resolvió el hallazgo.

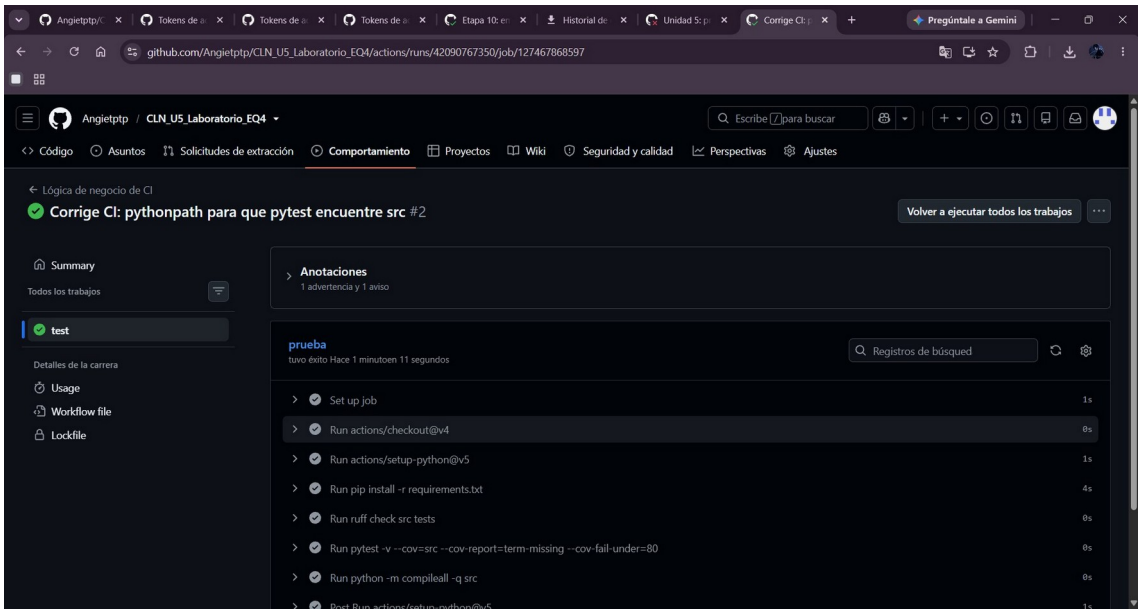


Figura 21. *Ejecución #2 «Corrige CI: pythonpath para que pytest encuentre src»: verde.*

Con una aserción falsa intencional, ruff pasa y falla únicamente el paso de pytest (2 s). Con ello se comprueba que el pipeline detecta una regresión de lógica sin intervención humana.

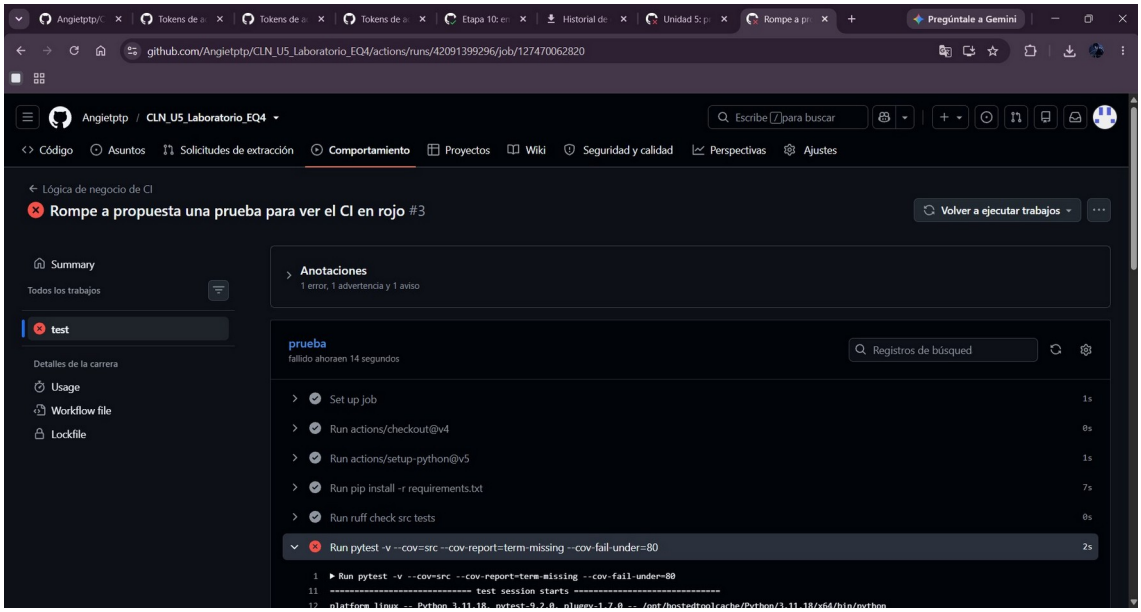


Figura 22. *Ejecución #3 «Rompe a propósito una prueba para ver el CI en rojo»: roja.*

El commit de reversión se creó con git revert, que conserva el historial. El pipeline vuelve a verde, con lo que se completa la secuencia verde–roja–verde que pide el laboratorio.

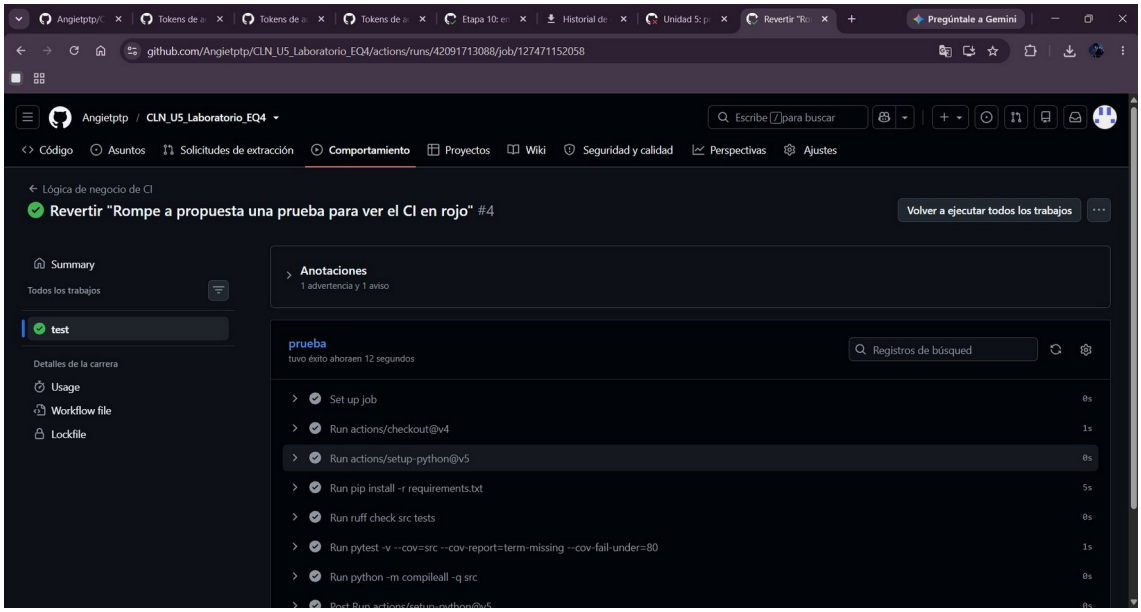


Figura 23. *Ejecución #4 «Revertir …»: verde.*

2.9 VetCare 24: Etapas 1 a 9

Concluido el laboratorio, se documentó el desarrollo de VetCare 24. El proyecto avanzó por etapas y en cada una se ejecutaron las pruebas con Docker Compose. Las capturas de esta sección muestran el crecimiento del número de pruebas y de la cobertura, y constituyen el antecedente de la matriz de pruebas, pues sin un dominio probado y una API estable no habría reglas reales que verificar.

Etapa 1 · entorno base. Swagger UI en localhost:8000/docs confirma que la API arranca. Con ello se comprueba que el contenedor de la aplicación responde y que la documentación interactiva está disponible.

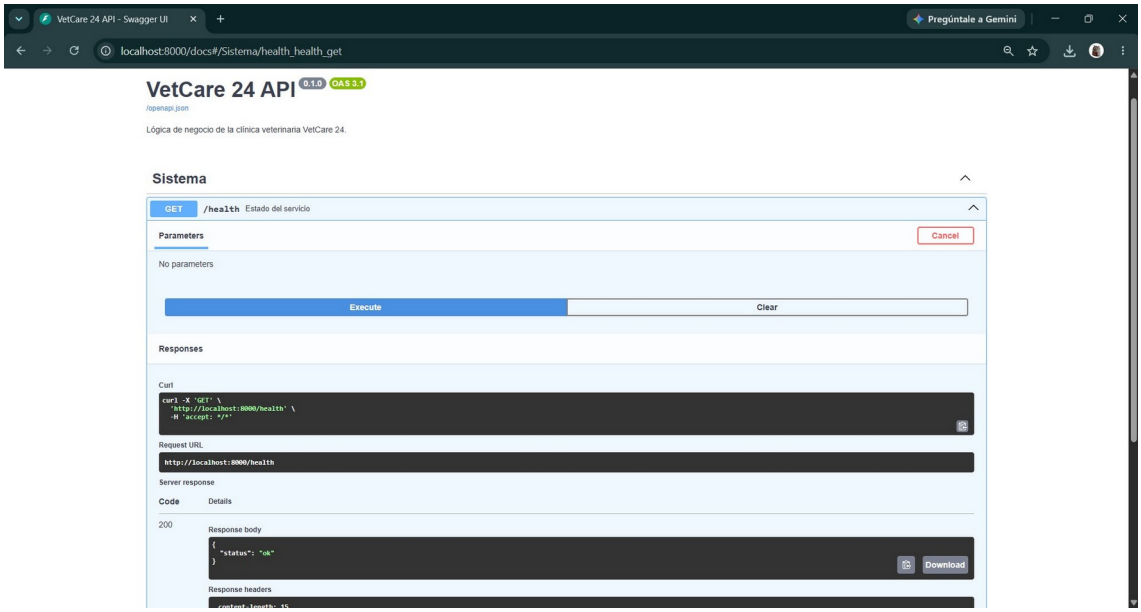


Figura 24. *Etapa 1 · /health devuelve 200 con {"status": "ok"}.*

Se ejecuta la suite de humo dentro del contenedor de pruebas. Muestra el arranque de la base de datos y del contenedor de pruebas.

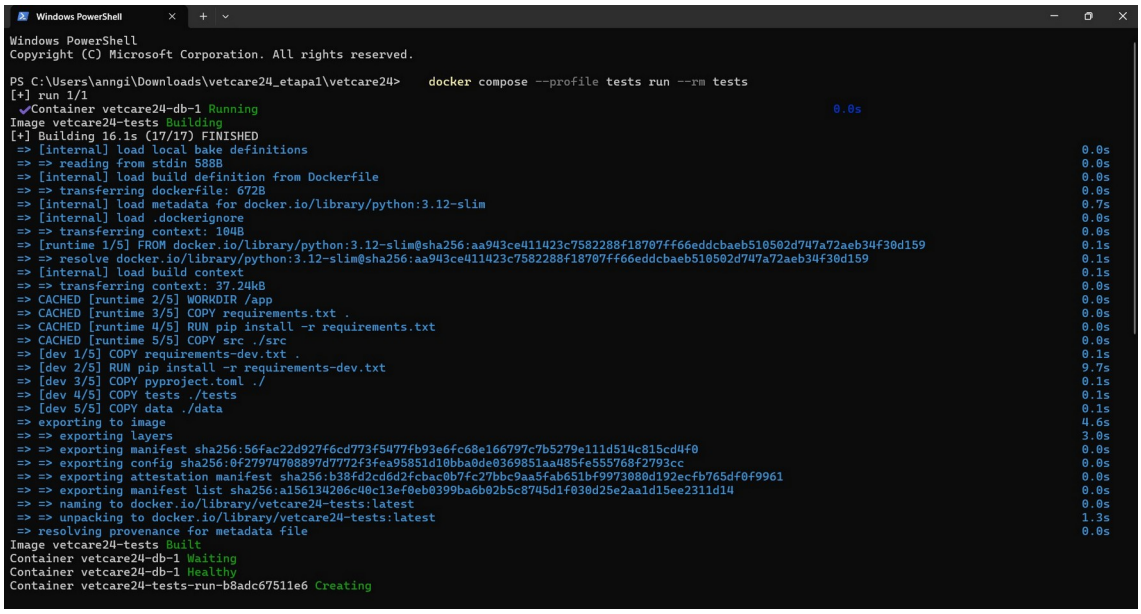


Figura 25. *Etapa 1 · ejecución de docker compose --profile tests run --rm tests.*

Resultado de la suite de la Etapa 1: «collected 4 items» y «Total coverage: 100.00%». Establece la línea base del proyecto.

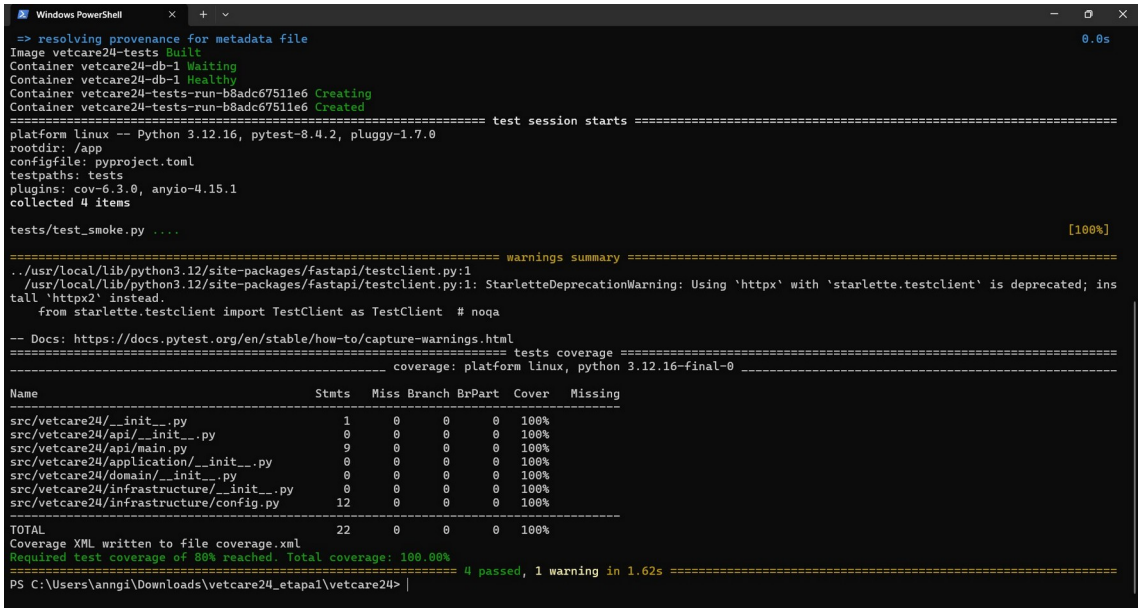


Figura 26. *Etapa 1 · 4 pruebas y cobertura 100 %.*

Registro del servicio de base de datos y apagado ordenado con eliminación de contenedores y red. Con ello se comprueba que la infraestructura se levanta y se limpia con un comando.

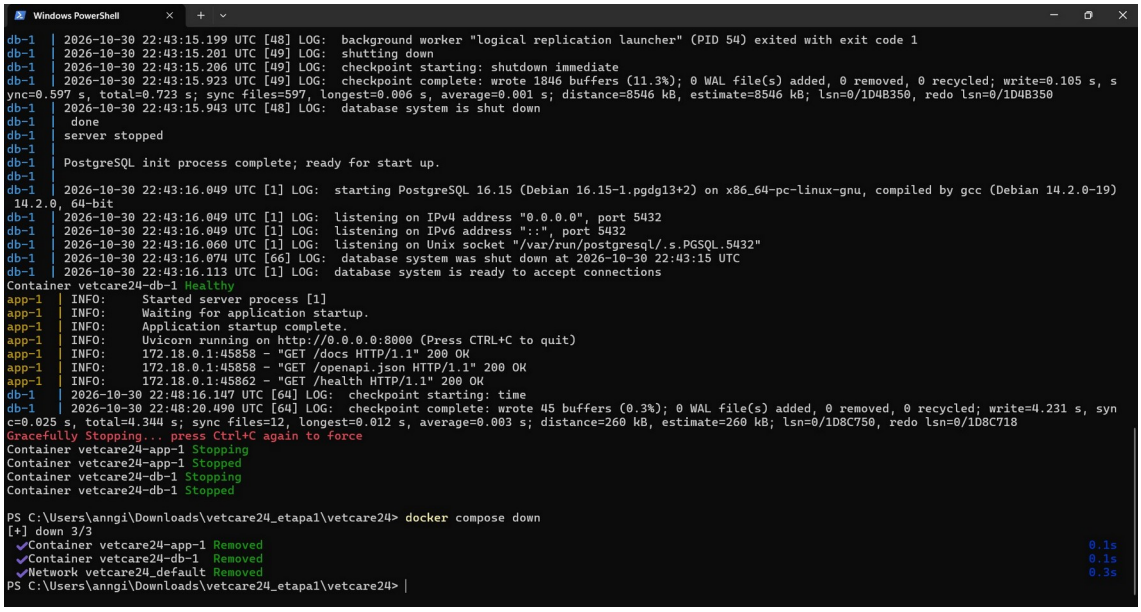


Figura 27. *Etapa 1 · arranque de PostgreSQL 16.15 y docker compose down.*

Se incorpora el importador de CSV y el oráculo que calcula los resultados esperados de los 120 casos y las 20 transiciones del paquete. El oráculo es la fuente independiente de «resultado esperado» para las pruebas posteriores.

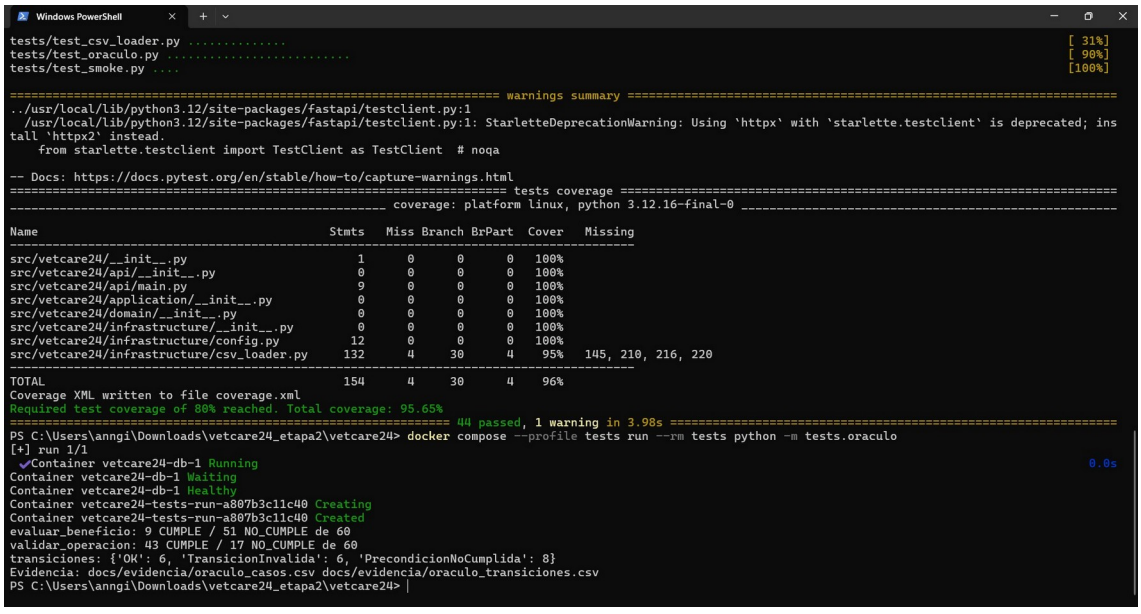


Figura 28. *Etapa 2 · importador y oráculo: cobertura 95.65 %.*

Reconstrucción de la imagen con el dominio de la Etapa 3. Queda documentado el procedimiento de reconstrucción tras cambiar archivos.

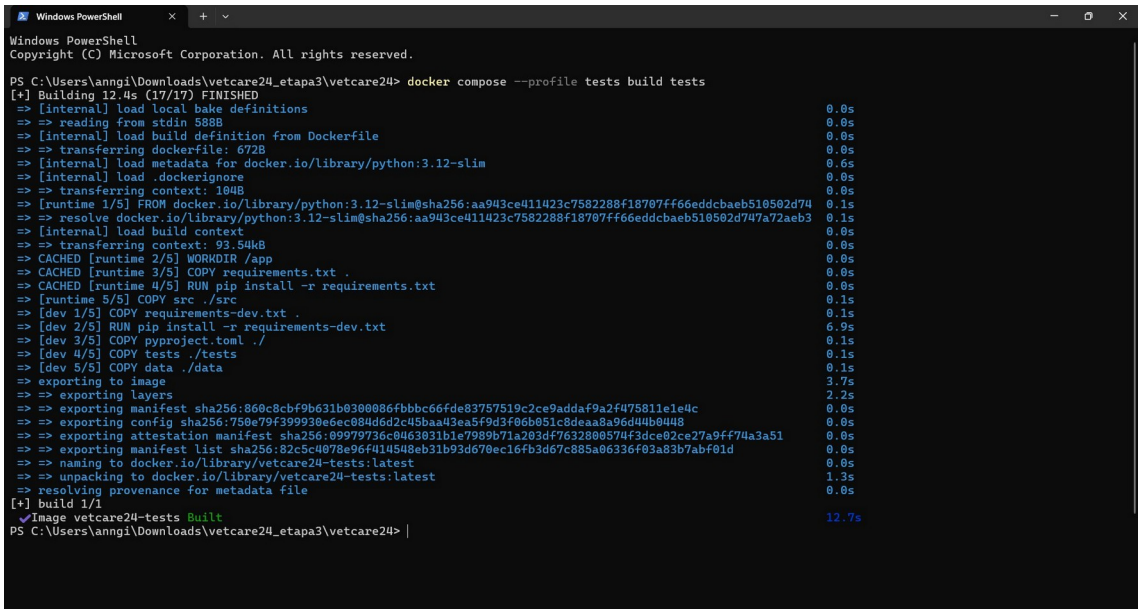


Figura 29. *Etapa 3 · construcción de la imagen de pruebas.*

Entidades y objetos de valor con sus invariantes (Money, Cita, Mascota, Propietario). Reglas puras probadas sin base de datos.

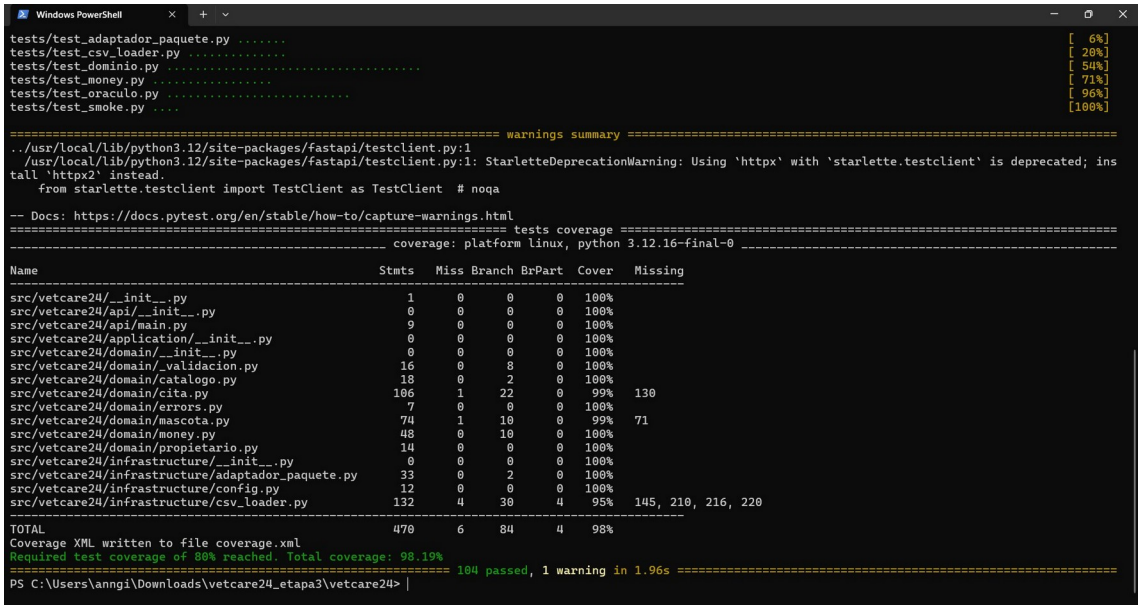


Figura 30. *Etapa 3 · dominio: cobertura 98.19 %.*

Reconstrucción de la imagen para la Etapa 4. Procedimiento repetible.

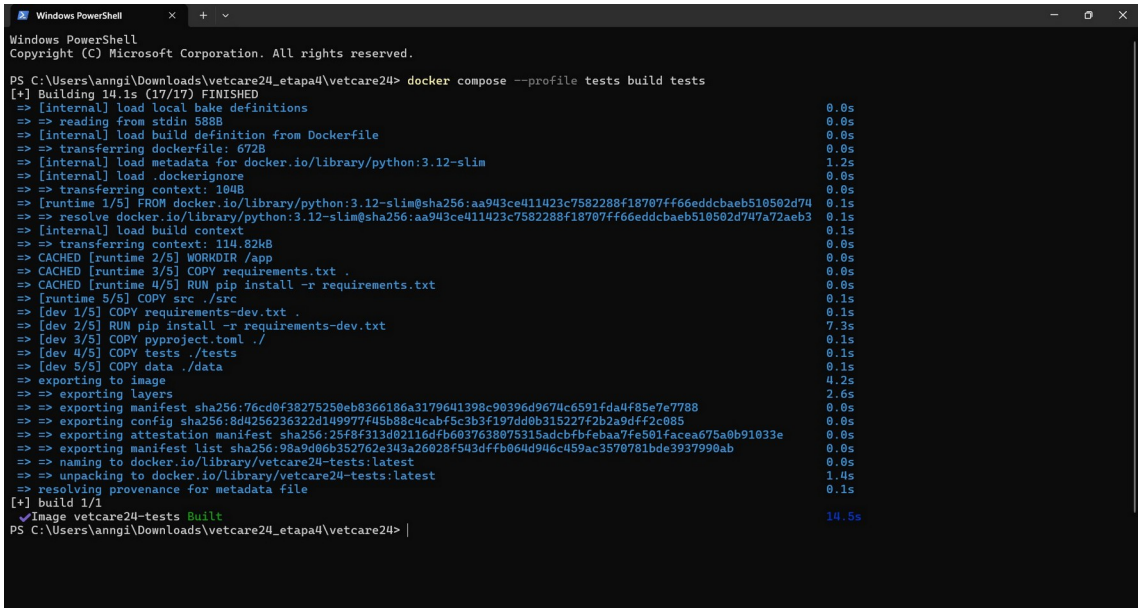


Figura 31. *Etapa 4 · construcción de la imagen de pruebas.*

Reglas de elegibilidad componibles (AND, OR, NOT) que devuelven cumplimiento y motivos. Base de las reglas CapacidadSuficiente y TotalMinimo que usa la matriz.

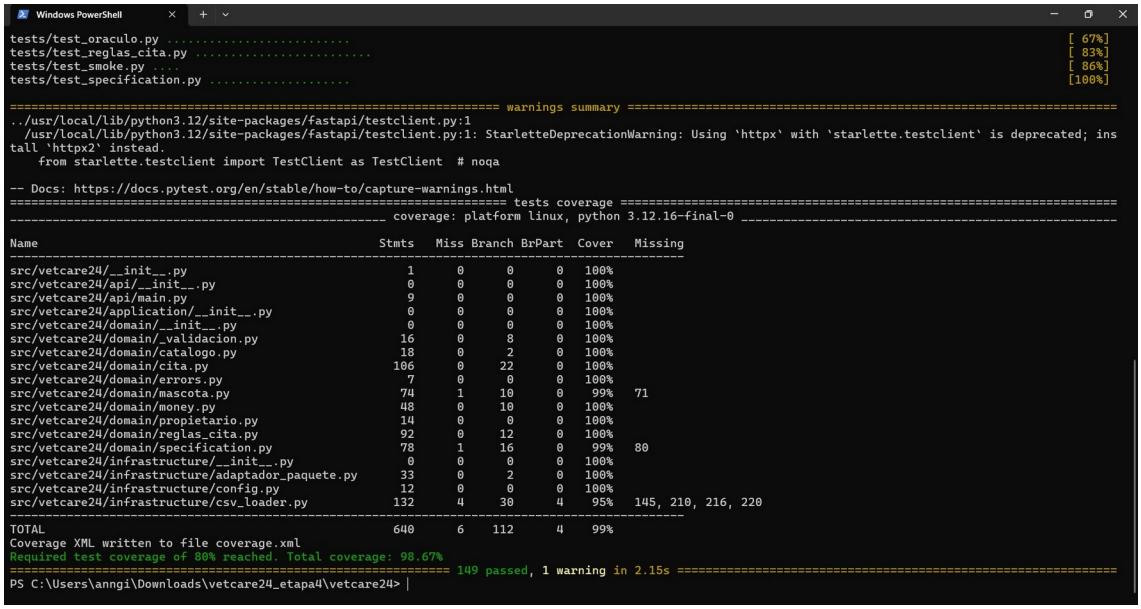


Figura 32. *Etapa 4 · Specifications: cobertura 98.67 %.*

Capas del Dockerfile y reinstalación de requirements-dev.txt. Muestra el uso de caché de capas.

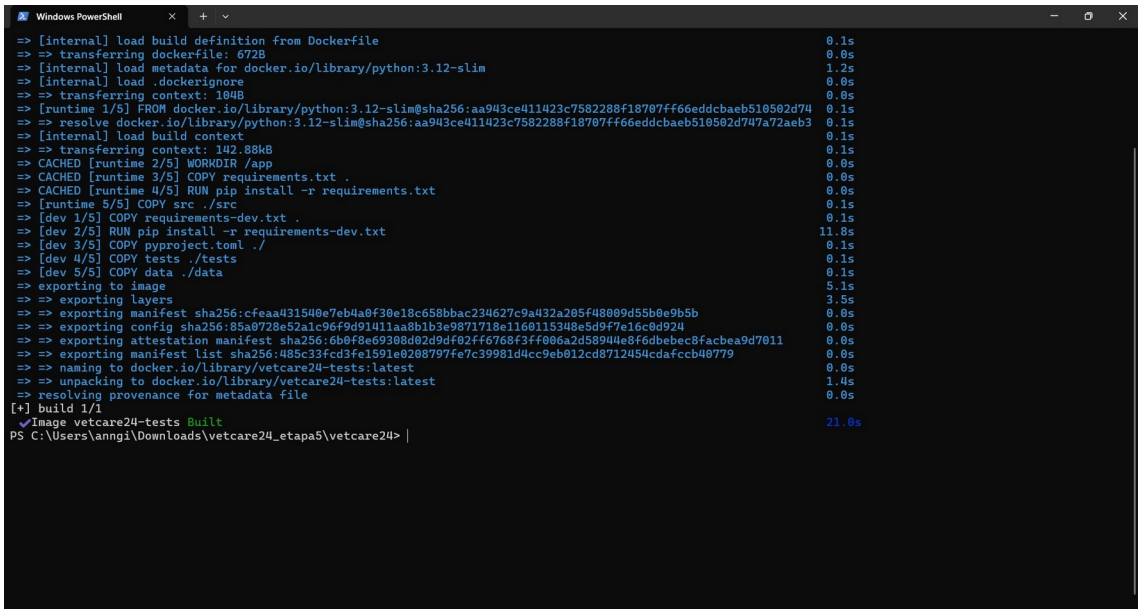


Figura 33. *Etapa 5 · construcción de la imagen de pruebas.*

Pruebas de smoke y specification al 100 % y resumen de advertencias. Confirma la estabilidad de la etapa.

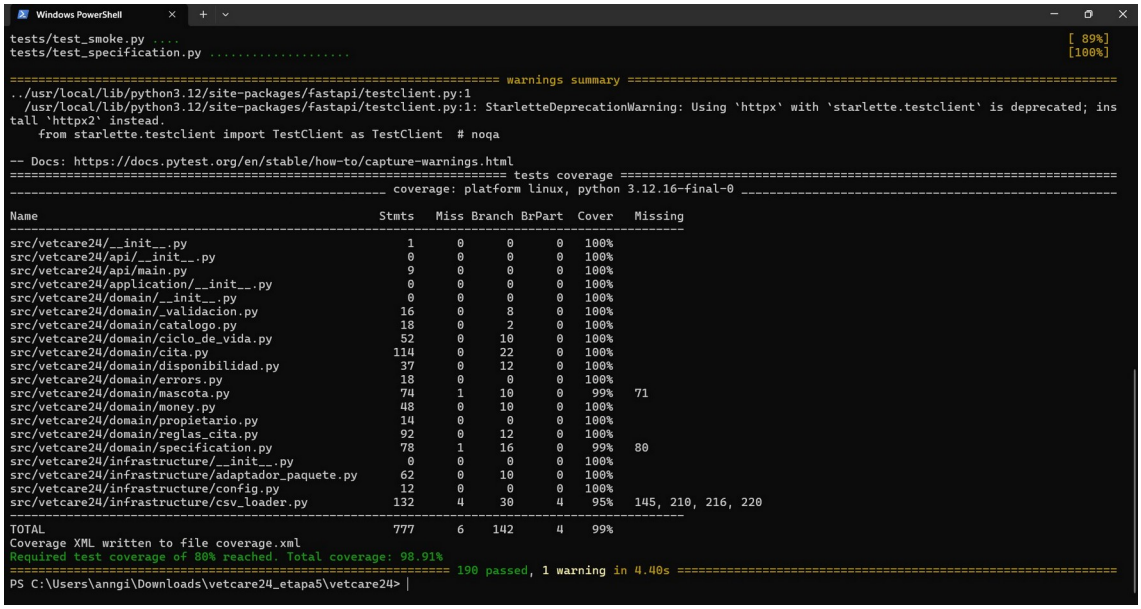


Figura 34. *Etapa 5 · fin de la suite.*

Ciclo de vida de la cita y control de capacidad por servicio. Soporta el caso de límite CP-03 (10 de 10).

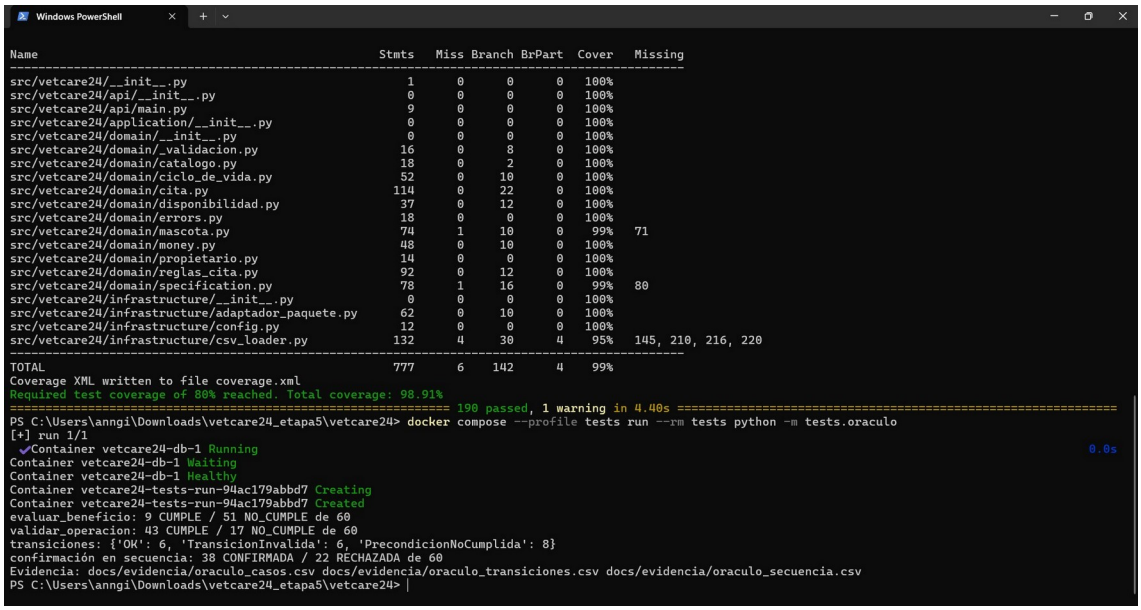


Figura 35. *Etapa 5 · máquina de estados y capacidad: cobertura 98.91 %.*

Reconstrucción con los Domain Events. Procedimiento repetible.

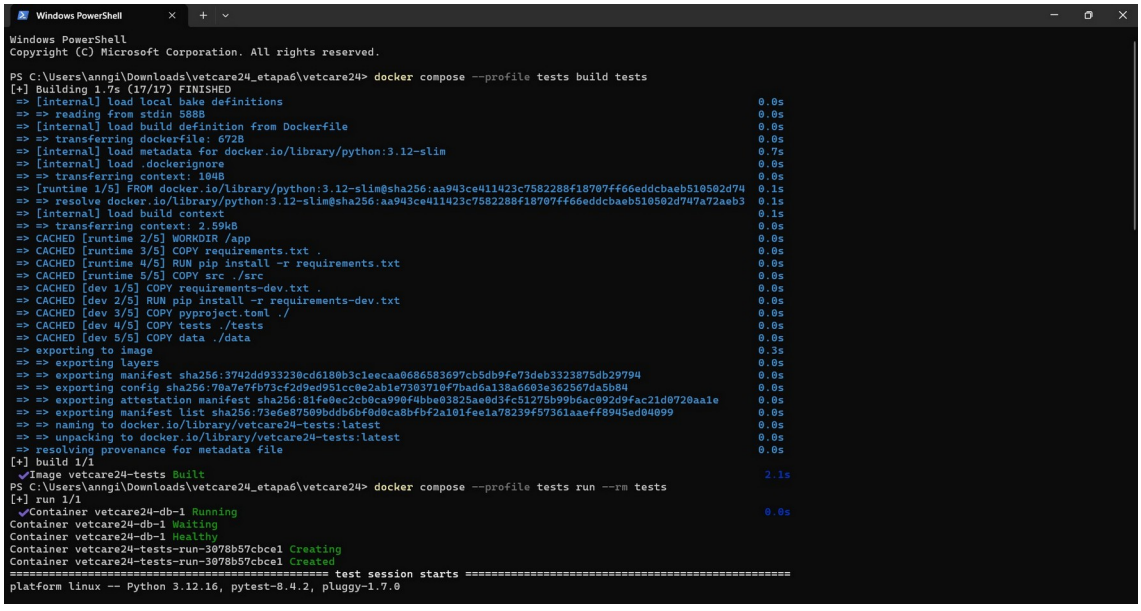


Figura 36. *Etapa 6 · construcción de la imagen de pruebas.*

Avance de las pruebas por archivo. Crecimiento de la suite.

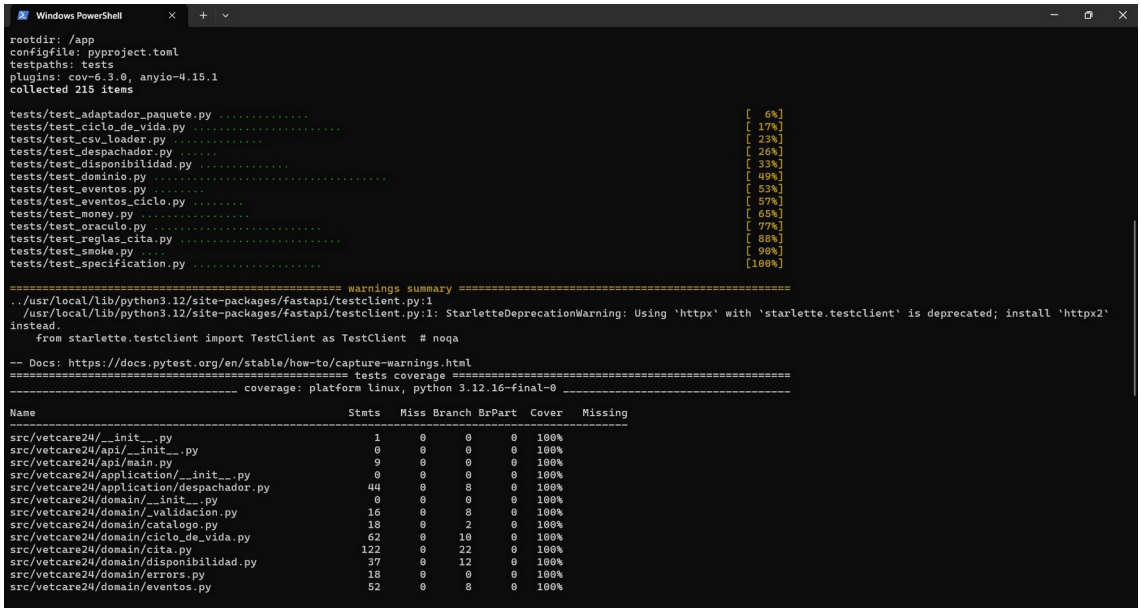


Figura 37. *Etapa 6 · 215 pruebas recolectadas.*

Eventos de dominio publicados por los agregados y registrados. Soporta la regla de eventos (D-14).

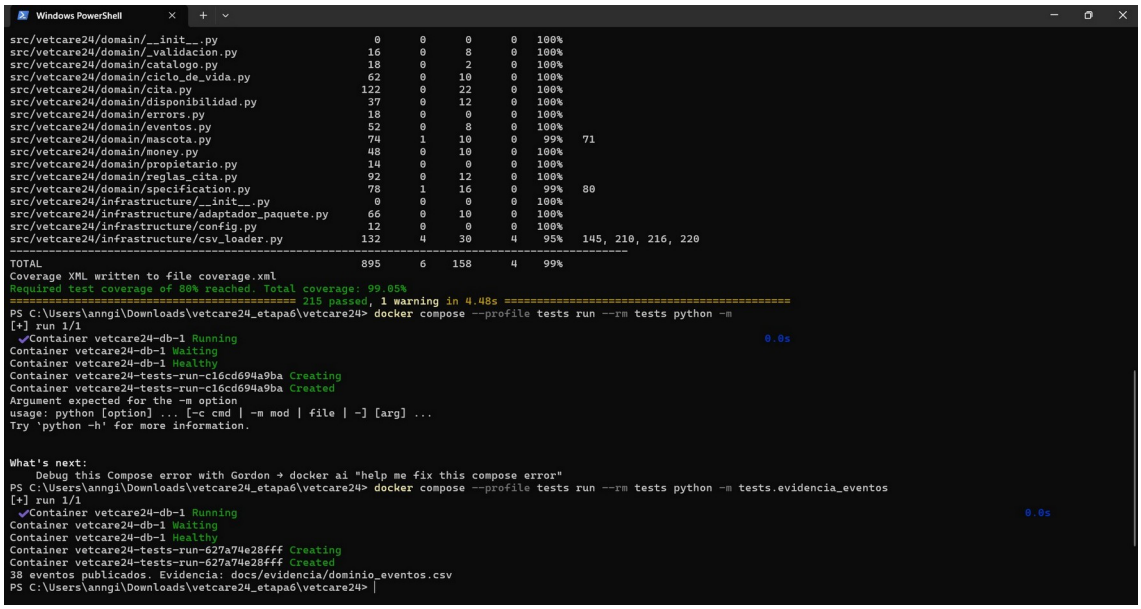


Figura 38. *Etapa 6 · Domain Events: 215 passed, cobertura 99.05 %.*

Imagen con aplicación, persistencia y API. Procedimiento repetible.

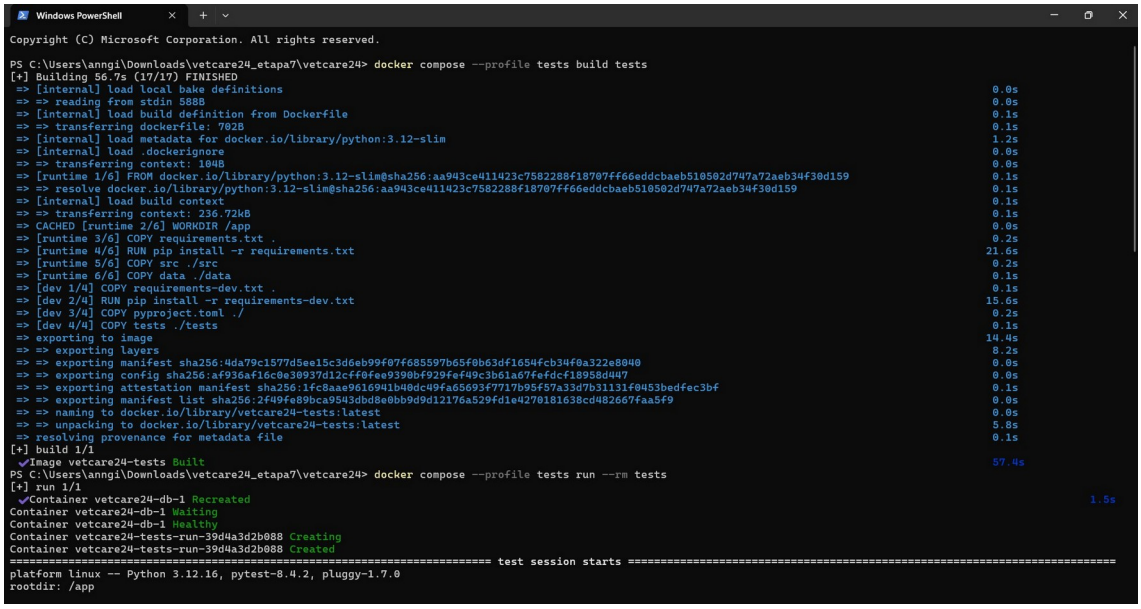


Figura 39. *Etapa 7 · construcción de la imagen de pruebas.*

Pruebas de base de datos, API y aplicación. Suite de integración.

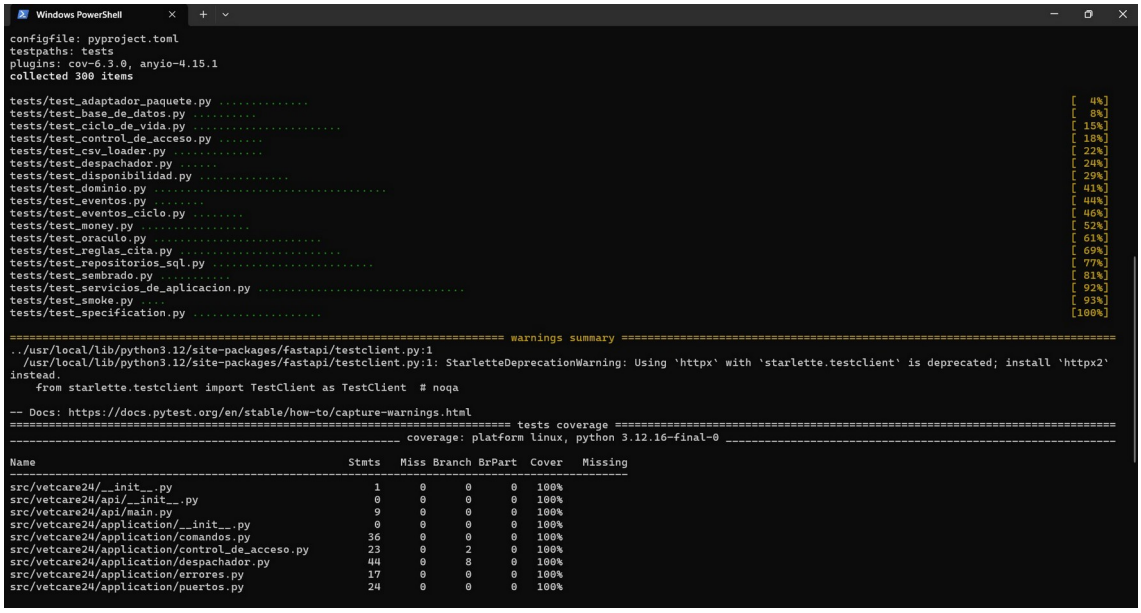


Figura 40. *Etapa 7 · 300 pruebas recolectadas.*

Casos de uso, repositorios con SQL explícito y API REST con roles. Primera versión completa de la API.

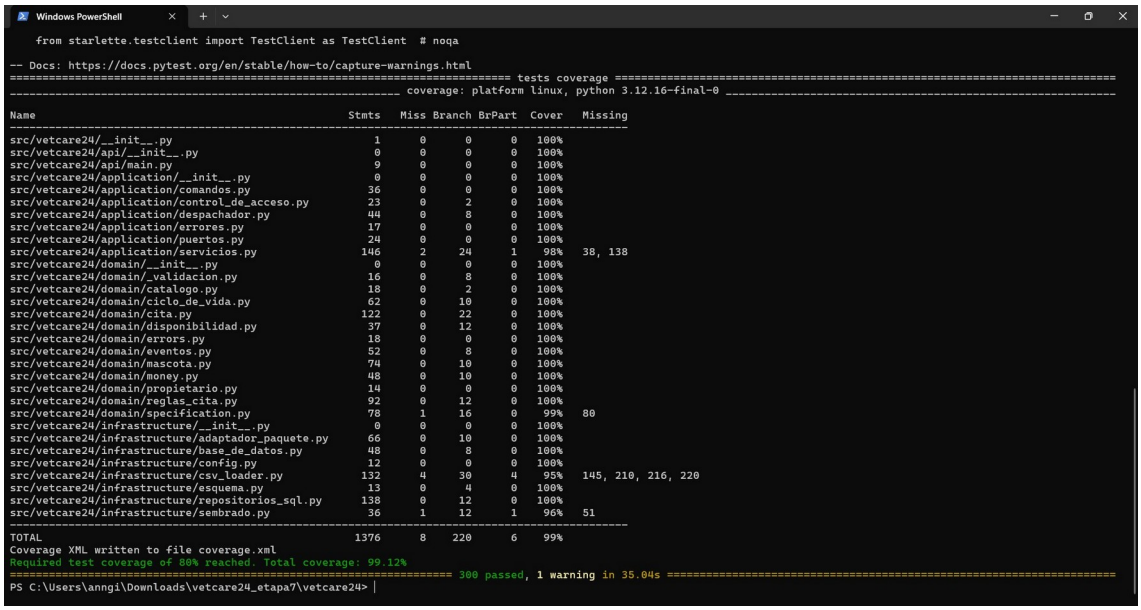


Figura 41. *Etapa 7 · 300 passed, cobertura 99.12 %.*

docker compose build app y carga: 20 servicios, 40 propietarios, 40 mascotas y 60 citas. Datos reales del paquete cargados en PostgreSQL.

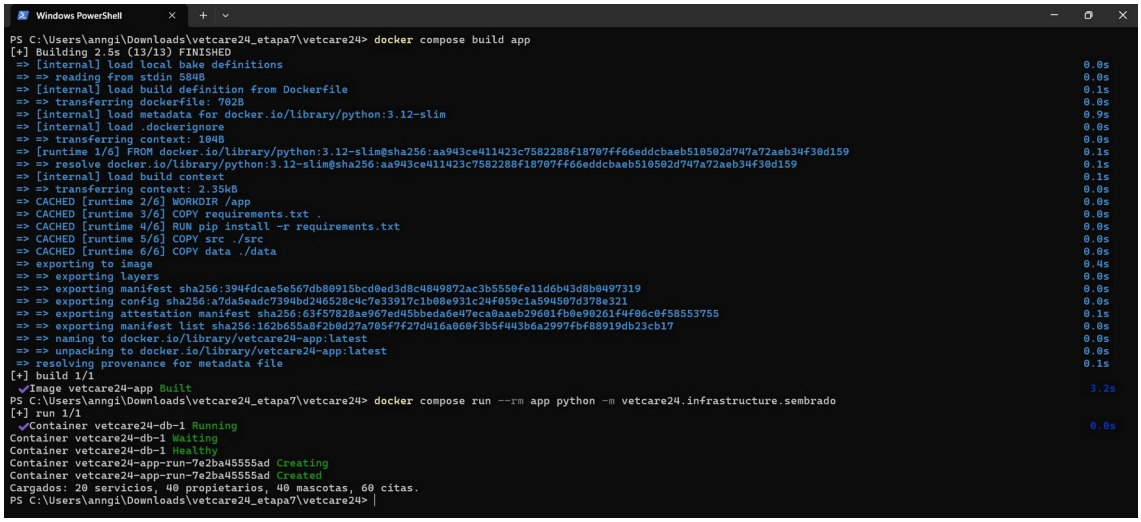


Figura 42. *Etapa 7 · sembrado de datos.*

Repetición del sembrado con la imagen reconstruida. Confirma que la carga es reproducible.

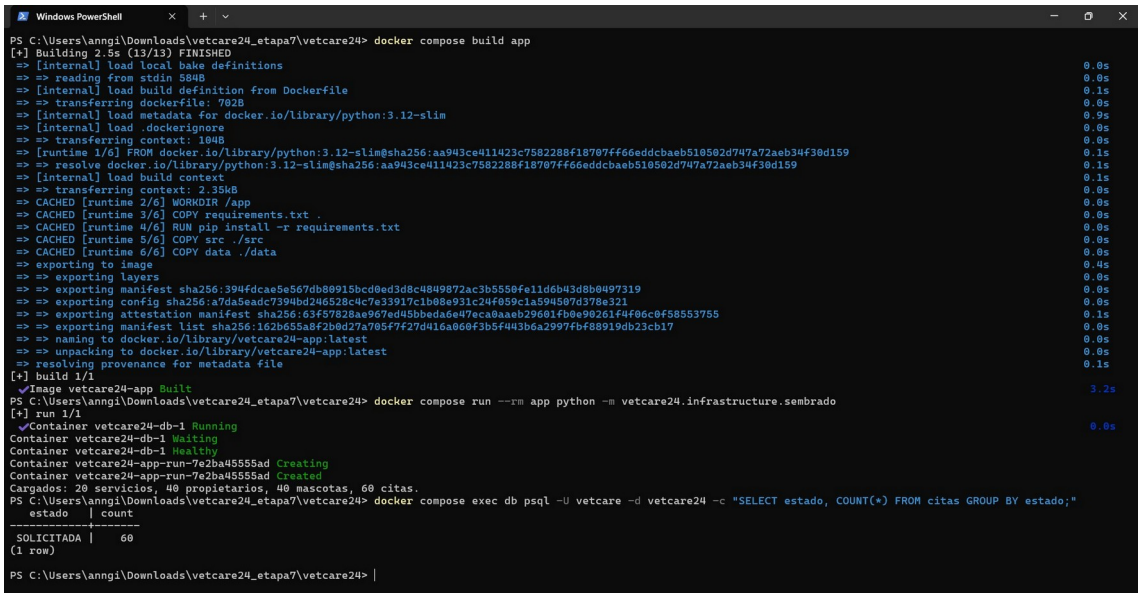


Figura 43. *Etapa 7 · sembrado repetido.*

Revisión de la API: códigos HTTP y mensajes. Procedimiento repetible.

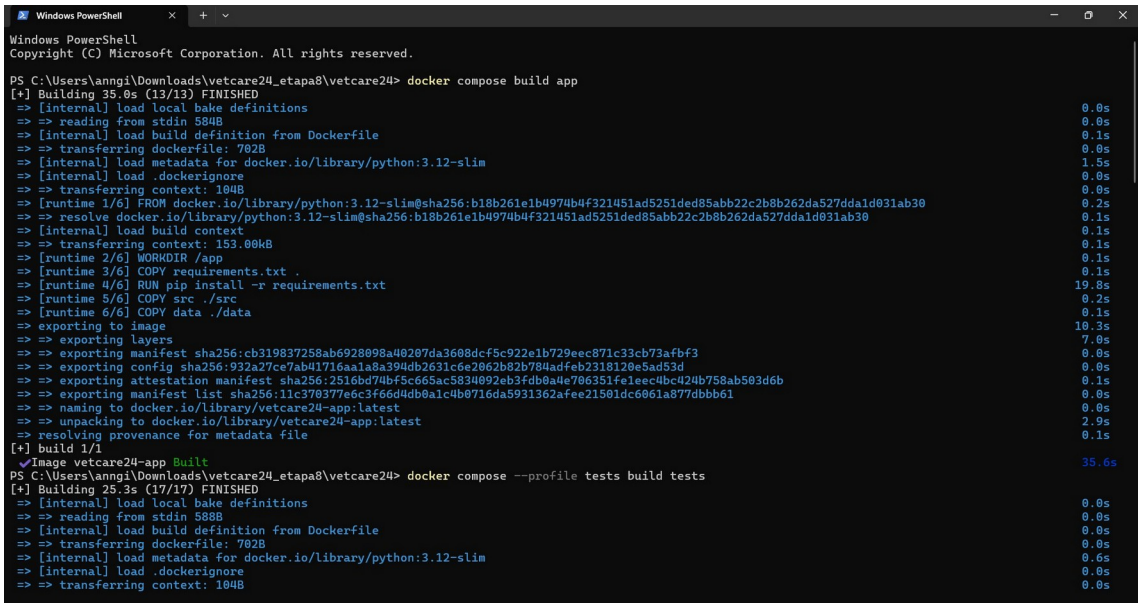


Figura 44. *Etapa 8 · construcción de la imagen de la aplicación.*

Imagen y contenedor de pruebas. Procedimiento repetible.

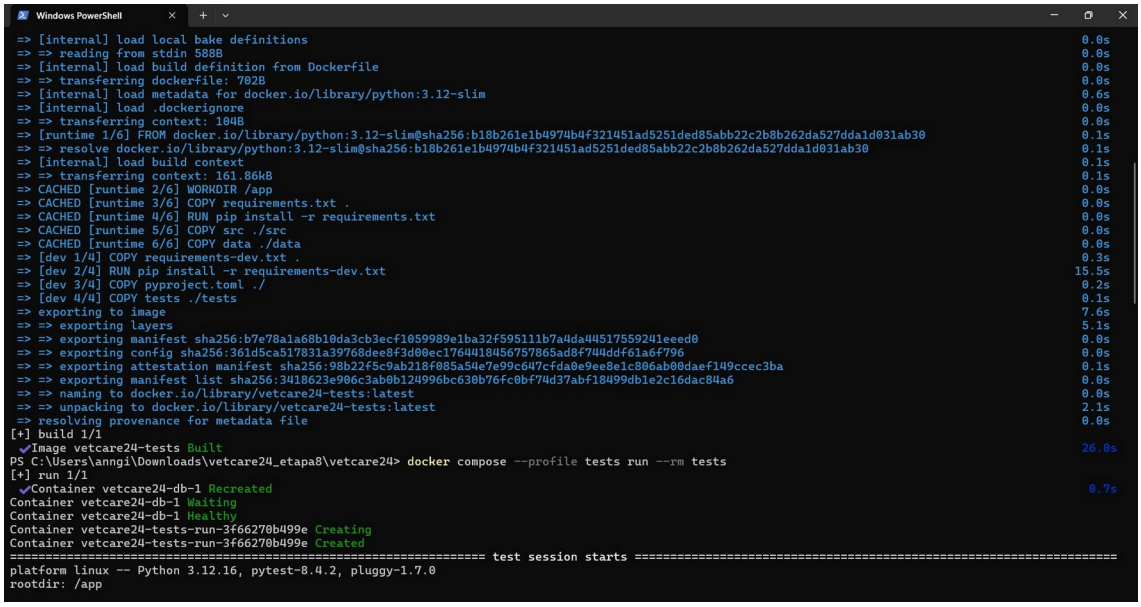


Figura 45. *Etapa 8 · construcción y arranque de la suite.*

Suite ampliada con pruebas de errores de la API. Crecimiento de la suite.

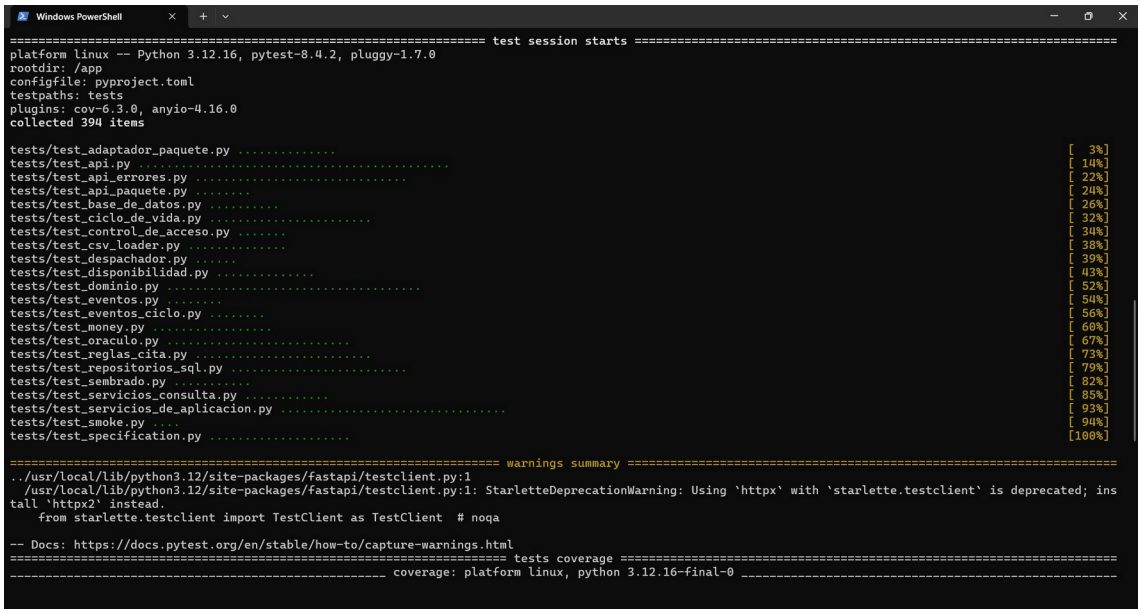


Figura 46. *Etapa 8 · 394 pruebas recolectadas.*

Resultado de la revisión de la API. Calidad sostenida.

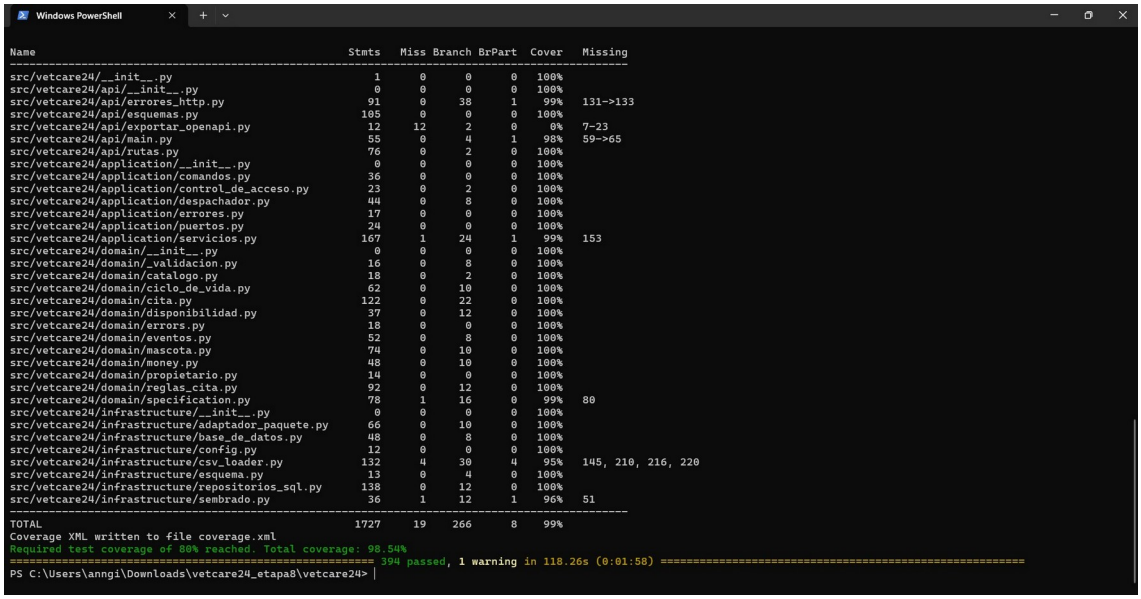


Figura 47. *Etapa 8 · 394 passed, cobertura 98.54 %.*

python -m tests.evidencia_api compara por HTTP los 120 casos y las 20 transiciones con el oráculo y exporta openapi.json (14 rutas). La API reproduce exactamente lo que predice el oráculo.

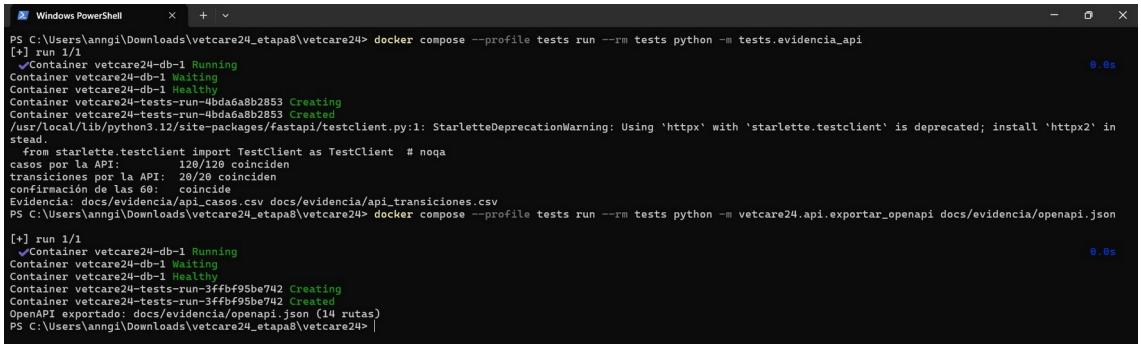


Figura 48. *Etapa 8 · evidencia de la API: 120/120, 20/20 y exportación OpenAPI.*

Se reinicia la base (cambió la tabla de eventos) y se vuelve a sembrar. Procedimiento de migración sencilla para la etapa.

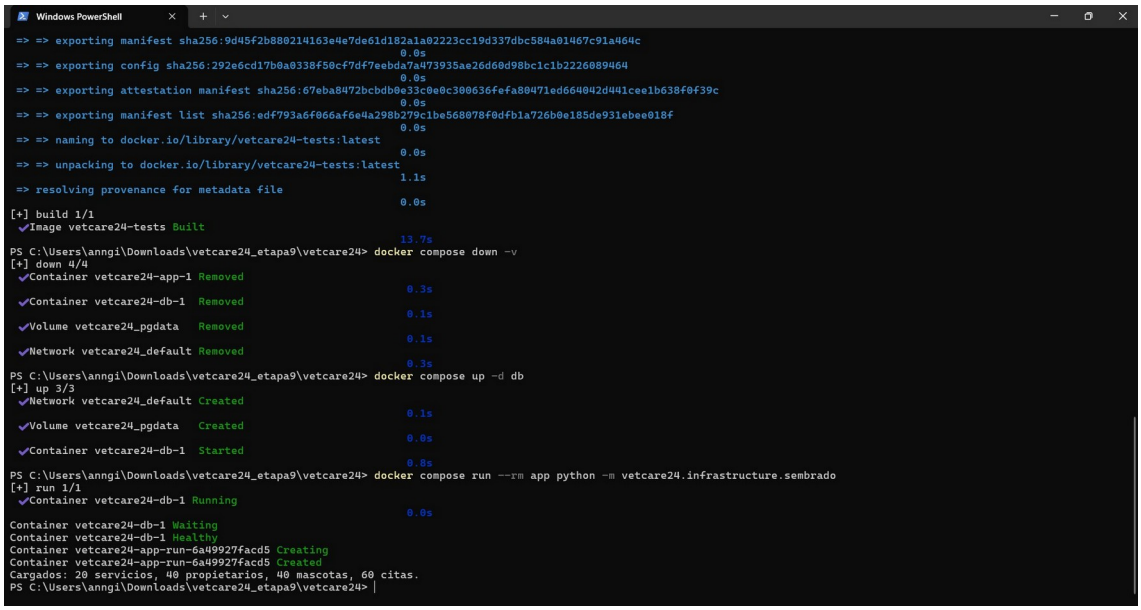


Figura 49. *Etapa 9 · reconstrucción, down -v y sembrado.*

Registro de vacunas, seguimiento y recordatorios. Cierre de la etapa previa a emergencias.

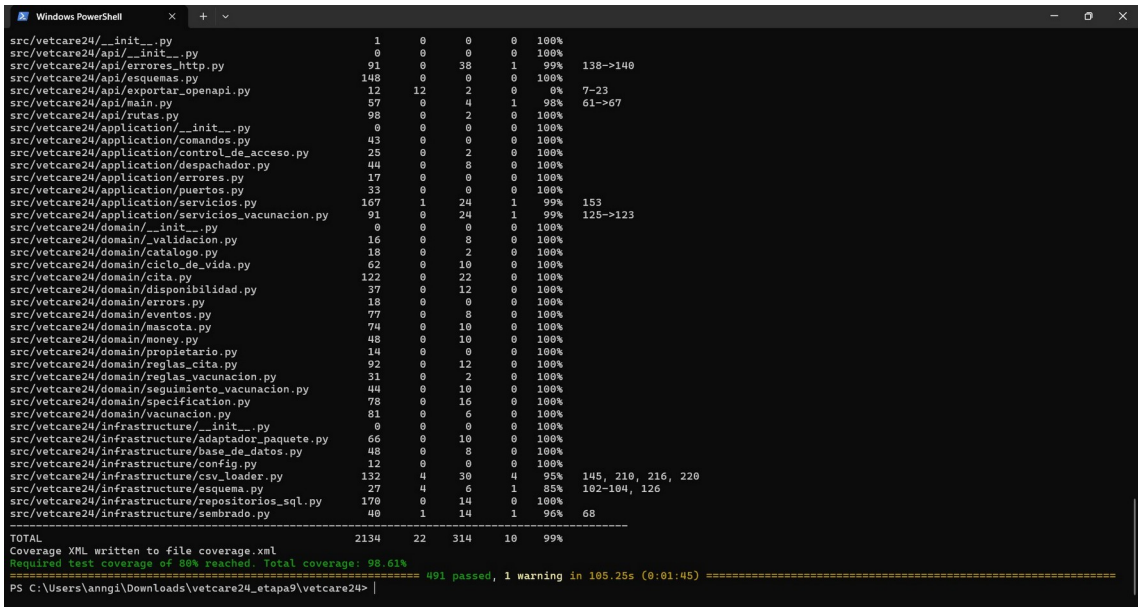


Figura 50. *Etapa 9 · vacunación: 491 passed, cobertura 98.61 %.*

2.10 VetCare 24: Etapa 10, Emergencias (CU-05)

La etapa 10 correspondió al módulo de emergencias, que incorpora el registro de emergencias sin cita, la activación del protocolo, el aviso al veterinario y el inicio

de la atención por el personal clínico. De esta etapa provienen dos de las tres reglas seleccionadas para la matriz de la unidad, por lo que se documenta con mayor detalle: las pruebas y la cobertura, el arranque de la API, el recorrido manual en Swagger, la base de datos, la evidencia por HTTP y el flujo de integración continua.

2.10.1 Pruebas y cobertura

En la captura se observan docker compose down, la copia de respaldo, Expand-Archive del ZIP y el listado de emergencia.py, reglas_emergencia.py, servicios_emergencias.py, notificador.py y sus pruebas, con la guía guia_etapa10.md. Esto demuestra qué archivos incorpora la etapa.

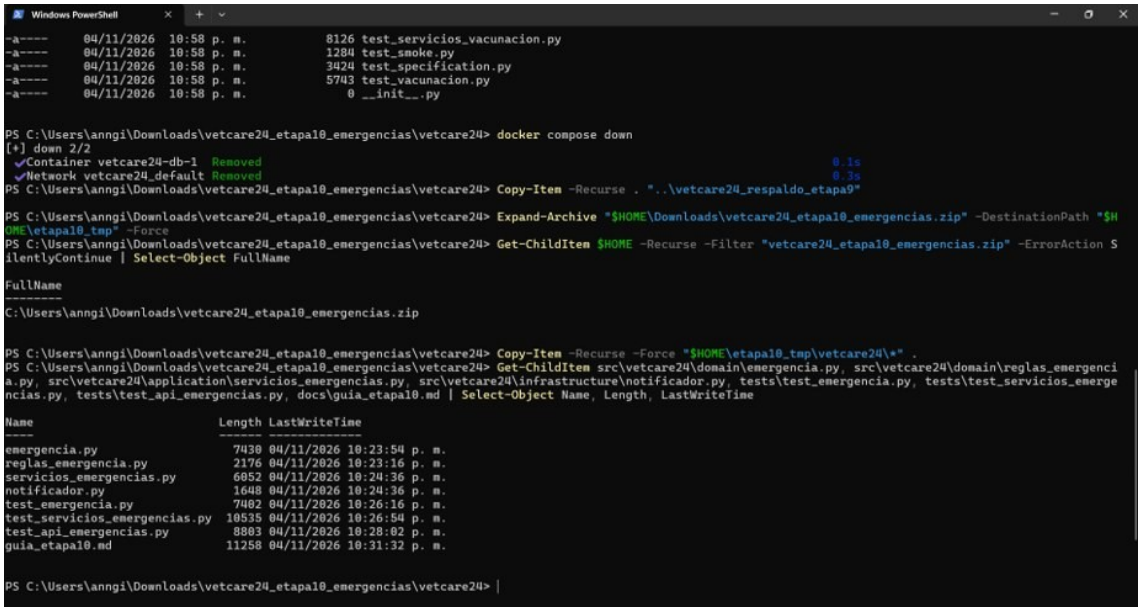


Figura 51. *Respaldo de la etapa 9, descompresión de la etapa 10 y listado de archivos nuevos.*

Cierre de la ejecución: «Total coverage: 98.81%» y «574 passed, 1 warning in 106.13s». El umbral exigido es 80 %.

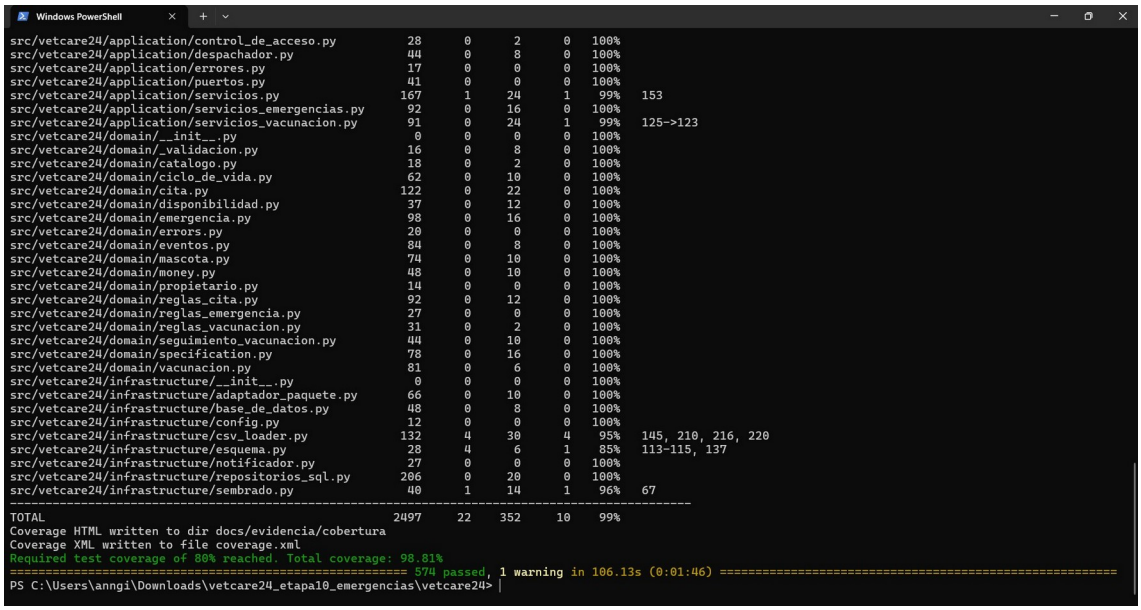


Figura 52. *Suite de la Etapa 10: 574 passed y cobertura 98.81 %.*

Con «collected 574 items / 570 deselected / 4 selected» pasan test_emergencia_sin_cita_se_registra y test_notifica_veterinario_tras_confirmar, cada una con las variantes [sqlite] y [postgres]. Con ello se comprueba que la misma prueba se ejecuta contra ambos motores.

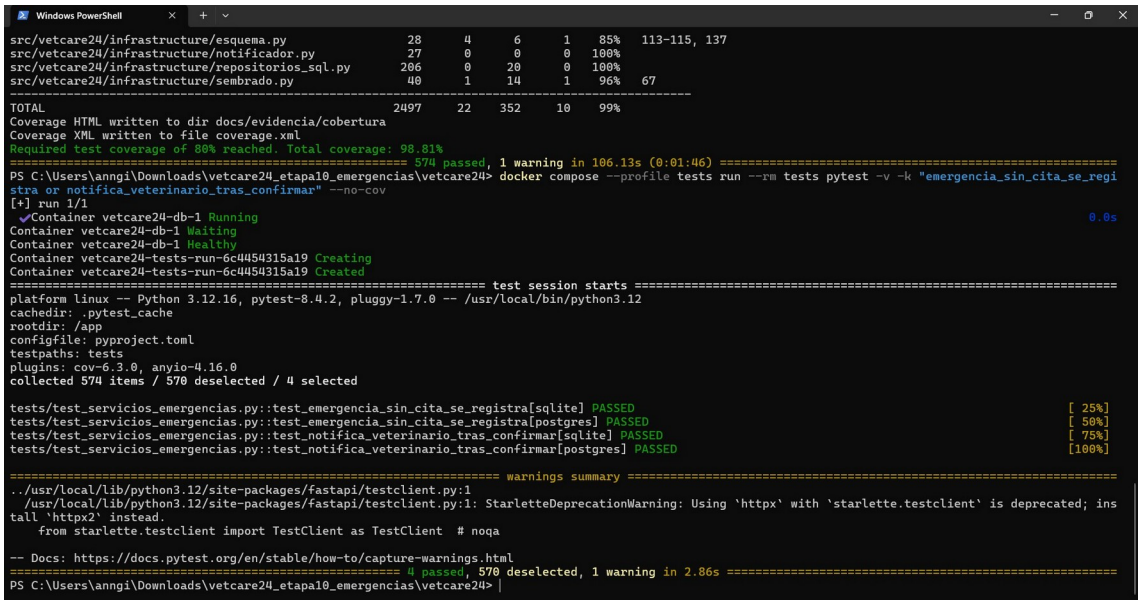
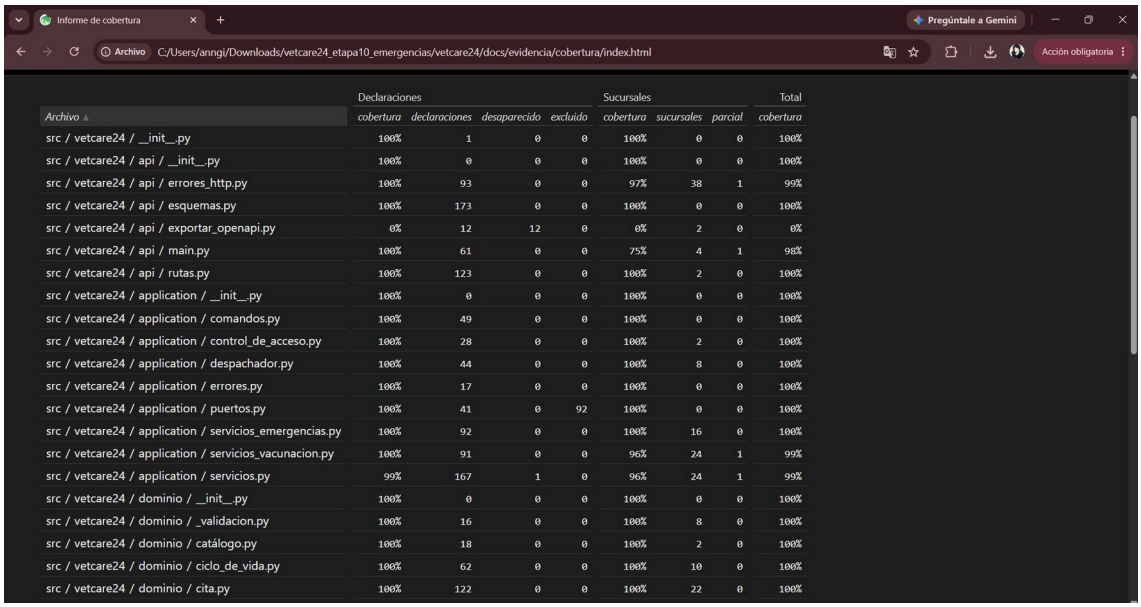


Figura 53. *Prueba dirigida con -k: 4 pruebas de emergencias PASSED en SQLite y PostgreSQL.*

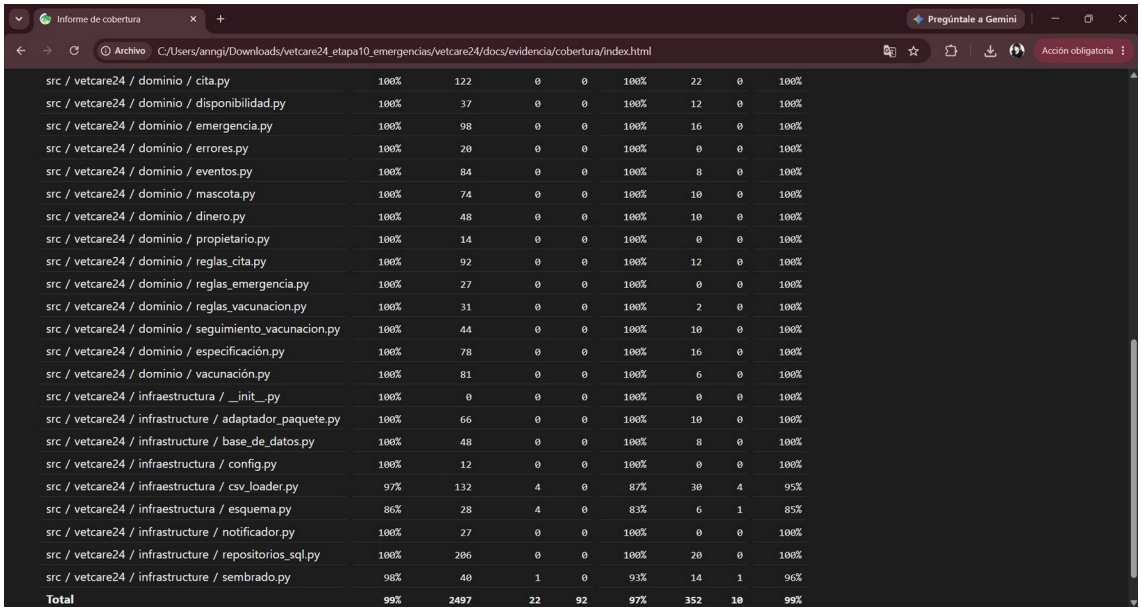
El reporte docs/evidencia/cobertura/index.html lista api/, application/ y dominio con 100 % en casi todos los archivos.



	Declaraciones				Sucursales			Total
Archivo	cobertura	declaraciones	desaparecido	excluido	cobertura	sucursales	parcial	cobertura
src / vetcare24 / __init__.py	100%	1	0	0	100%	0	0	100%
src / vetcare24 / api / __init__.py	100%	0	0	0	100%	0	0	100%
src / vetcare24 / api / errores_http.py	100%	93	0	0	97%	38	1	99%
src / vetcare24 / api / esquemas.py	100%	173	0	0	100%	0	0	100%
src / vetcare24 / api / exportar_openapi.py	0%	12	12	0	0%	2	0	0%
src / vetcare24 / api / main.py	100%	61	0	0	75%	4	1	98%
src / vetcare24 / api / rutas.py	100%	123	0	0	100%	2	0	100%
src / vetcare24 / application / __init__.py	100%	0	0	0	100%	0	0	100%
src / vetcare24 / application / comandos.py	100%	49	0	0	100%	0	0	100%
src / vetcare24 / application / control_de_acceso.py	100%	28	0	0	100%	2	0	100%
src / vetcare24 / application / despachador.py	100%	44	0	0	100%	8	0	100%
src / vetcare24 / application / errores.py	100%	17	0	0	100%	0	0	100%
src / vetcare24 / application / puertos.py	100%	41	0	92	100%	0	0	100%
src / vetcare24 / application / servicios_emergencias.py	100%	92	0	0	100%	16	0	100%
src / vetcare24 / application / servicios_vacunacion.py	100%	91	0	0	96%	24	1	99%
src / vetcare24 / application / servicios.py	99%	167	1	0	96%	24	1	99%
src / vetcare24 / dominio / __init__.py	100%	0	0	0	100%	0	0	100%
src / vetcare24 / dominio / _validacion.py	100%	16	0	0	100%	8	0	100%
src / vetcare24 / dominio / catálogo.py	100%	18	0	0	100%	2	0	100%
src / vetcare24 / dominio / ciclo_de_vida.py	100%	62	0	0	100%	10	0	100%
src / vetcare24 / dominio / cita.py	100%	122	0	0	100%	22	0	100%

Figura 54. *Reporte HTML de cobertura (1.ª parte).*

La fila final indica 99 % con 2497 declaraciones y 22 sin cubrir. Los pocos archivos por debajo del 100 % son de apoyo (importador CSV, esquema y sembrado) y no pertenecen a las reglas de emergencias.



src / vetcare24 / dominio / cita.py	100%	122	0	0	100%	22	0	100%
src / vetcare24 / dominio / disponibilidad.py	100%	37	0	0	100%	12	0	100%
src / vetcare24 / dominio / emergencia.py	100%	98	0	0	100%	16	0	100%
src / vetcare24 / dominio / errores.py	100%	20	0	0	100%	0	0	100%
src / vetcare24 / dominio / eventos.py	100%	84	0	0	100%	8	0	100%
src / vetcare24 / dominio / mascota.py	100%	74	0	0	100%	10	0	100%
src / vetcare24 / dominio / dinero.py	100%	48	0	0	100%	10	0	100%
src / vetcare24 / dominio / propietario.py	100%	14	0	0	100%	0	0	100%
src / vetcare24 / dominio / reglas_cita.py	100%	92	0	0	100%	12	0	100%
src / vetcare24 / dominio / reglas_emergencia.py	100%	27	0	0	100%	0	0	100%
src / vetcare24 / dominio / reglas_vacunacion.py	100%	31	0	0	100%	2	0	100%
src / vetcare24 / dominio / seguimiento_vacunacion.py	100%	44	0	0	100%	10	0	100%
src / vetcare24 / dominio / especificación.py	100%	78	0	0	100%	16	0	100%
src / vetcare24 / dominio / vacunación.py	100%	81	0	0	100%	6	0	100%
src / vetcare24 / infraestructura / __init__.py	100%	0	0	0	100%	0	0	100%
src / vetcare24 / infrastructure / adaptador_paquete.py	100%	66	0	0	100%	10	0	100%
src / vetcare24 / infrastructure / base_de_datos.py	100%	48	0	0	100%	8	0	100%
src / vetcare24 / infraestructura / config.py	100%	12	0	0	100%	0	0	100%
src / vetcare24 / infraestructura / csv_loader.py	97%	132	4	0	87%	30	4	95%
src / vetcare24 / infraestructura / esquema.py	86%	28	4	0	83%	6	1	85%
src / vetcare24 / infrastructure / notificador.py	100%	27	0	0	100%	0	0	100%
src / vetcare24 / infrastructure / repositorios_sql.py	100%	206	0	0	100%	20	0	100%
src / vetcare24 / infrastructure / sembrado.py	98%	40	1	0	93%	14	1	96%
Total	**99%**	**2497**	**22**	**92**	**97%**	**352**	**10**	**99%**

Figura 55. *Reporte HTML de cobertura (2.ª parte) con el total.*

2.10.2 Arranque de la API y documentación

Tras reconstruir la imagen se repite el sembrado y no duplica datos. Con ello se comprueba que el proceso se puede repetir con seguridad.

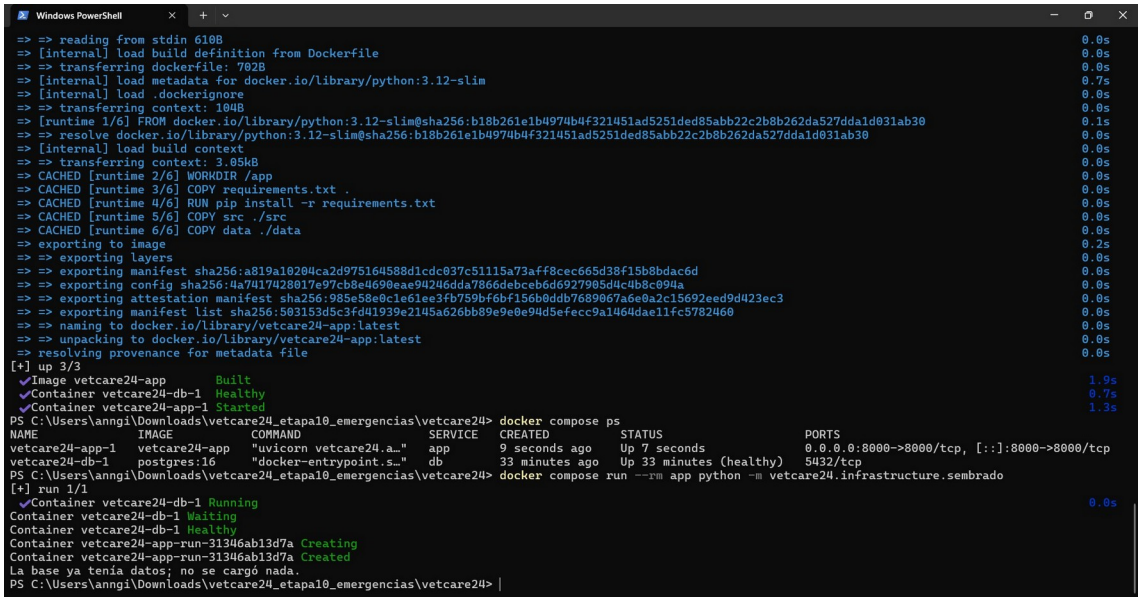


Figura 56. *Sembrado idempotente: «La base ya tenía datos; no se cargó nada».*

La descripción de la API enumera los códigos de error (400, 401, 403, 404, 409 y 422) y el encabezado X-Rol para elegir el rol.

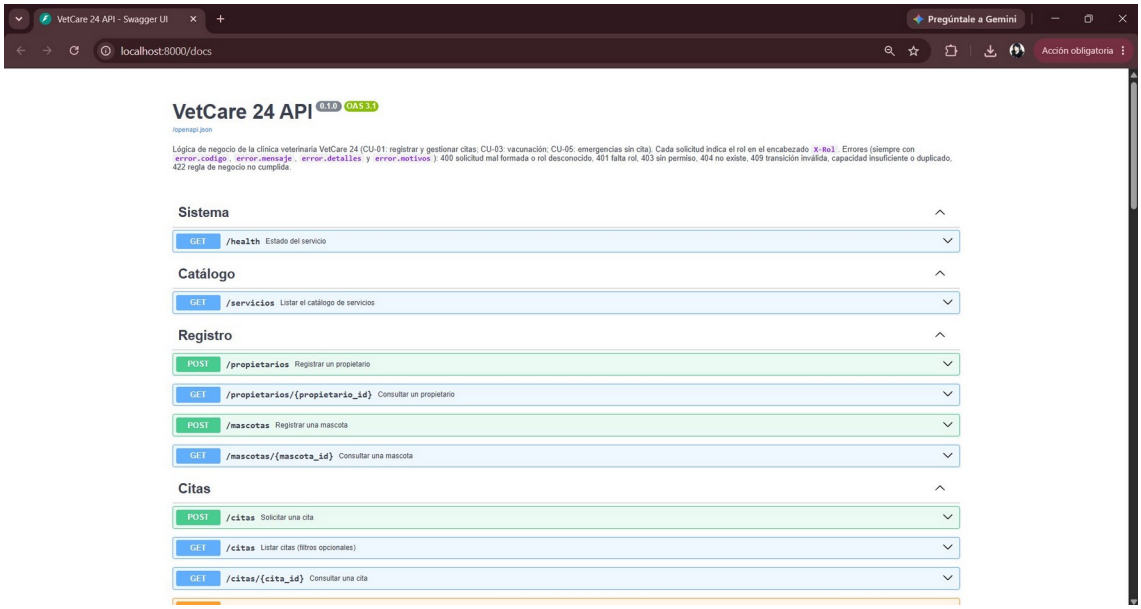


Figura 57. *Swagger UI: secciones Sistema, Catálogo, Registro y Citas.*

Aparece POST /emergencias junto a los demás grupos de endpoints, con su descripción.

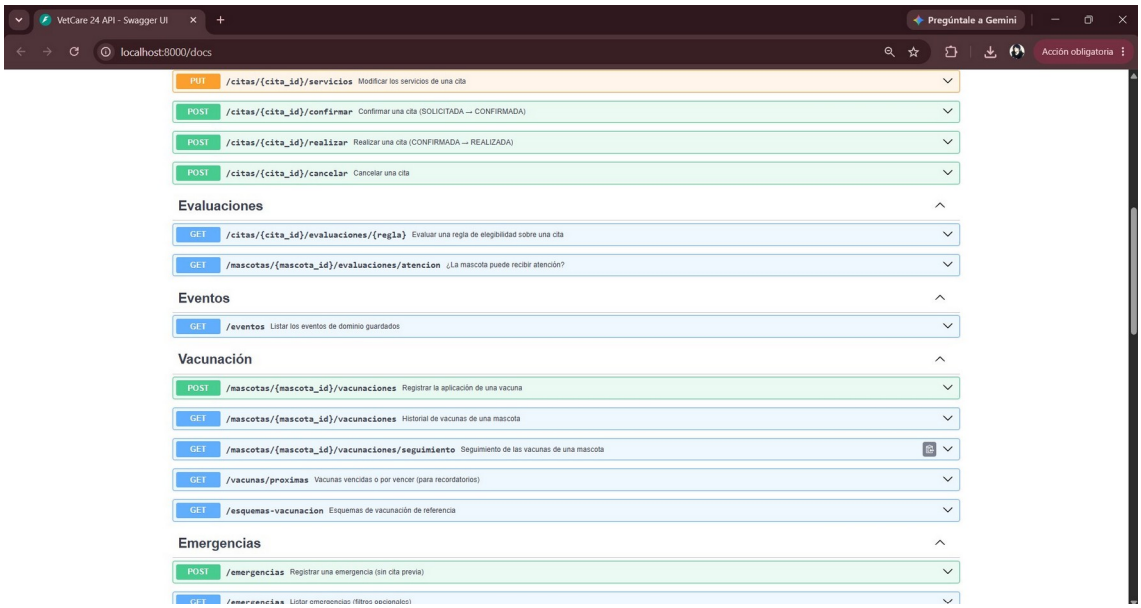


Figura 58. *Swagger UI: Evaluaciones, Eventos, Vacunación y Emergencias.*

En la captura se ven iniciar-atencion, consulta y listado, /avisos-veterinario y los primeros esquemas (AvisoOut, EmergenciaIn, EmergenciaOut).

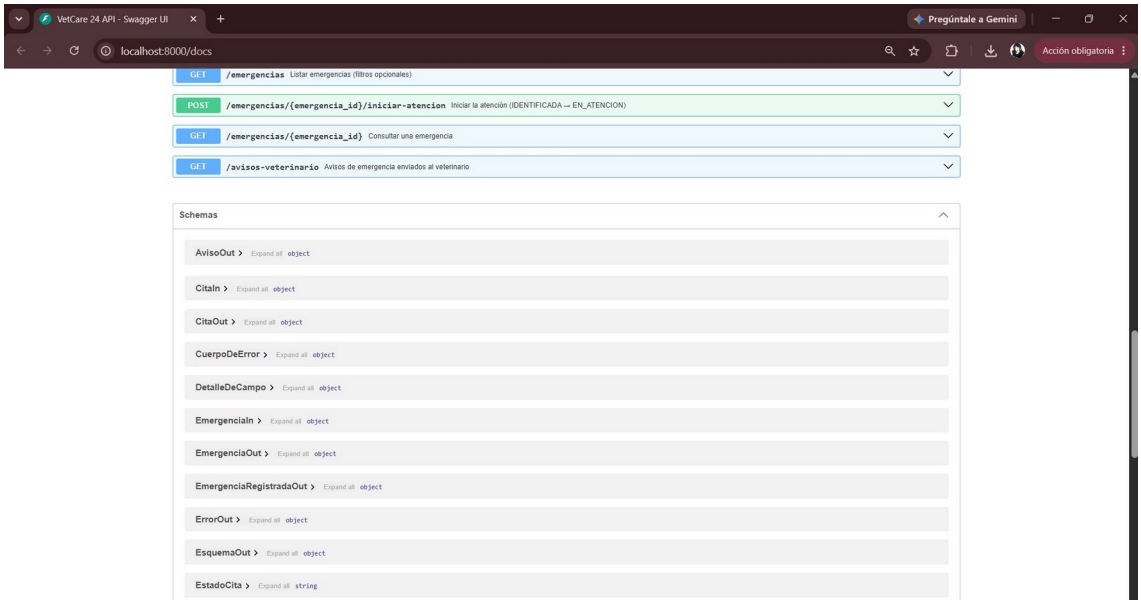


Figura 59. *Swagger UI: endpoints de emergencias y comienzo de los esquemas.*

Los esquemas (EstadoEmergencia, EvaluacionOut, EventoOut, MascotaIn, VacunacionOut…) forman el contrato de datos que valida la prueba del contrato OpenAPI.

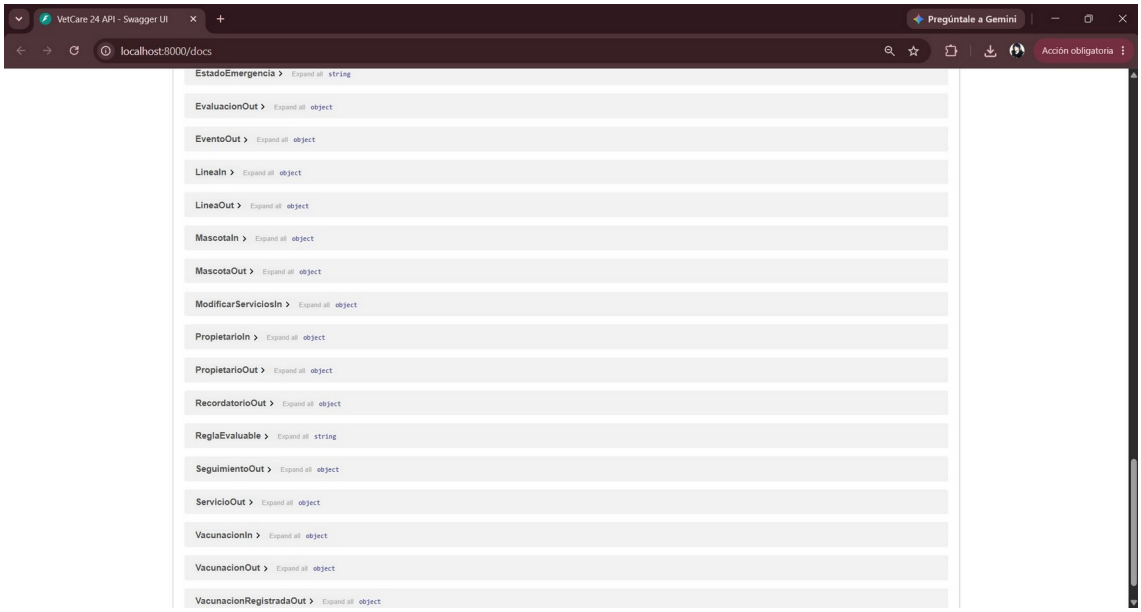


Figura 60. *Swagger UI: lista de esquemas.*

2.10.3 Pruebas manuales de los endpoints (Swagger UI)

Posteriormente a las pruebas automáticas, se recorrió manualmente el caso de uso completo desde Swagger UI. En cada captura se aprecian la petición con su rol y su cuerpo, la respuesta y el código HTTP.

GET /mascotas/VET-C002-MASCOTA/evaluaciones/atencion con X-Rol: veterinario responde 200 con los motivos SIN_CITA_CONFIRMADA y SIN_EMERGENCIA_ACTIVA. Esto demuestra la regla R-06 en su caso negativo.

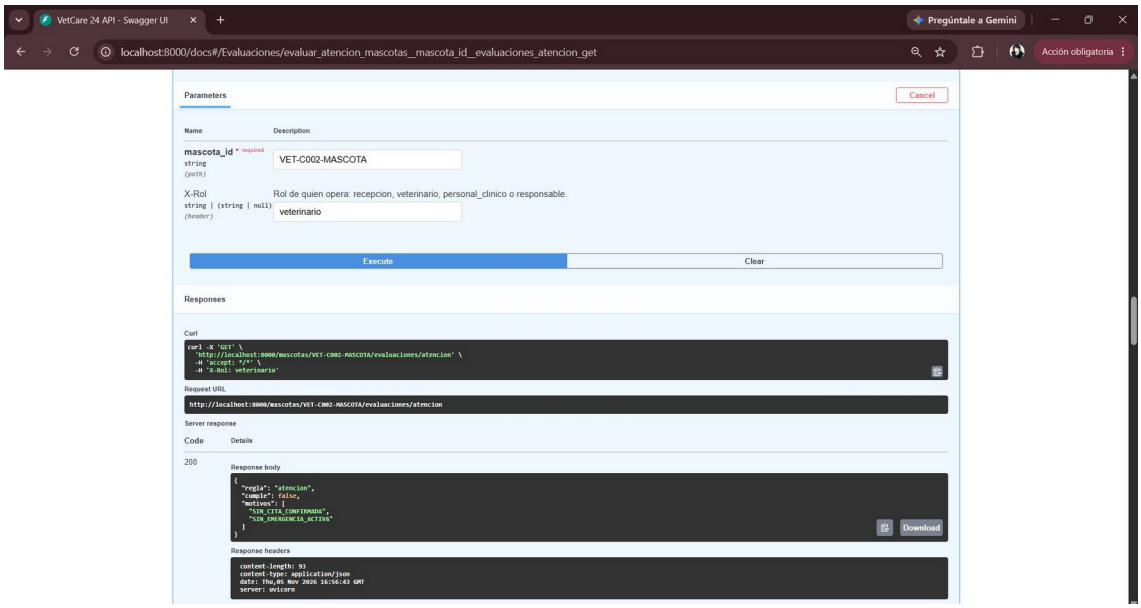


Figura 61. *CU-05 · evaluación de atención antes de la emergencia: cumple = false.*

Con X-Rol: recepcion y el cuerpo {"motivo": "Atropellamiento", "mascota_id": "VET-C002-MASCOTA"} se obtiene 201, estado IDENTIFICADA, protocolo_activado = true y notificación «notificada». Es el caso de éxito del registro.

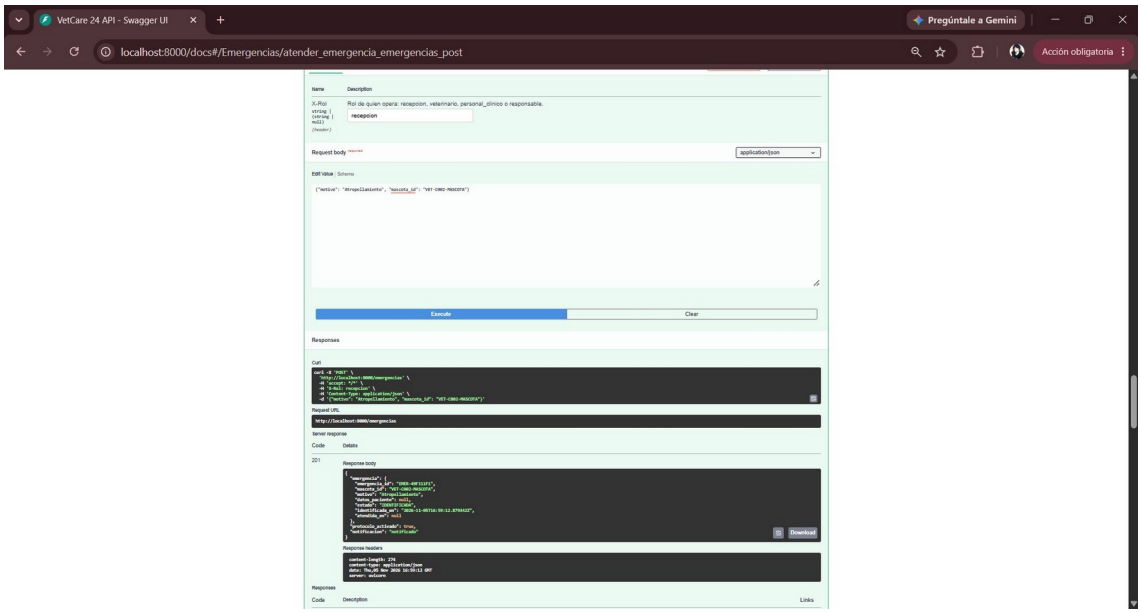


Figura 62. *CU-05 · POST /emergencias → 201 IDENTIFICADA.*

La misma consulta ahora responde cumple = true con motivos vacíos: TieneEmergenciaActiva satisface la regla aunque no exista cita confirmada.

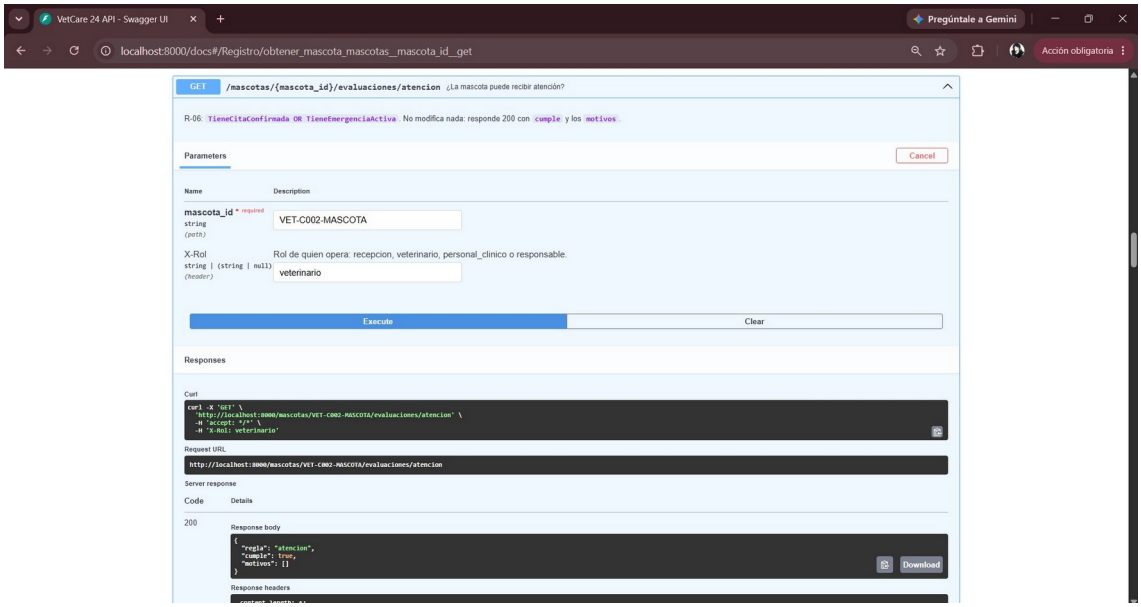


Figura 63. *CU-05 · evaluación de atención tras el alta: cumple = true.*

El veterinario ve el aviso de la emergencia (bandeja del notificador). Esto demuestra la notificación al veterinario.

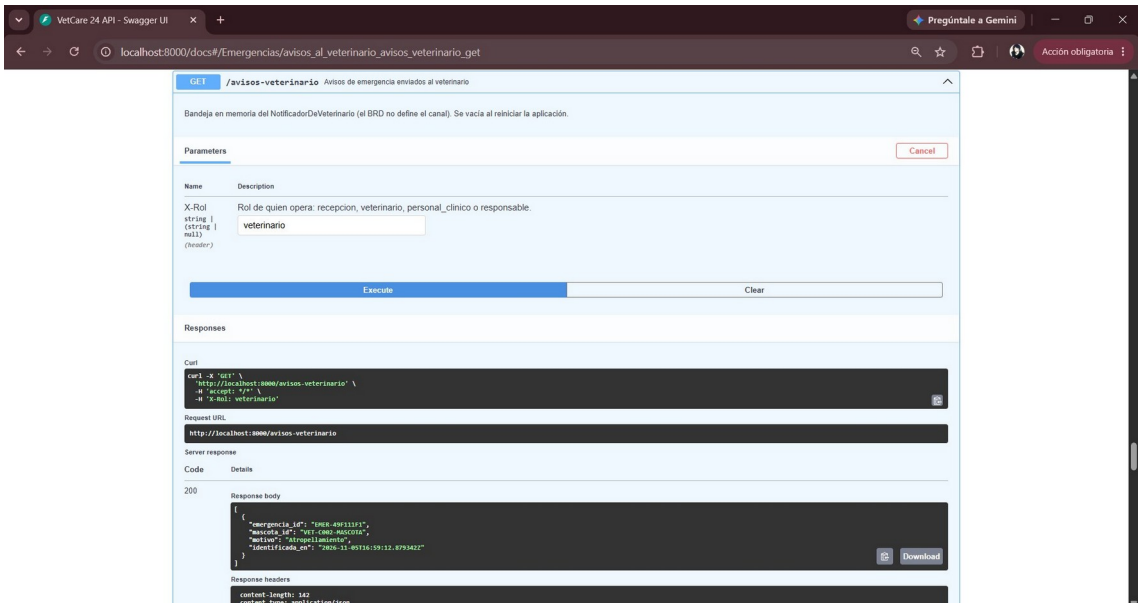


Figura 64. *CU-05 · GET /avisos-veterinario con el aviso registrado.*

Con motivo «Intoxicación» y datos_paciente «Gato atigrado, sin registro» se registra la emergencia. Confirma la decisión P3: la mascota es opcional.

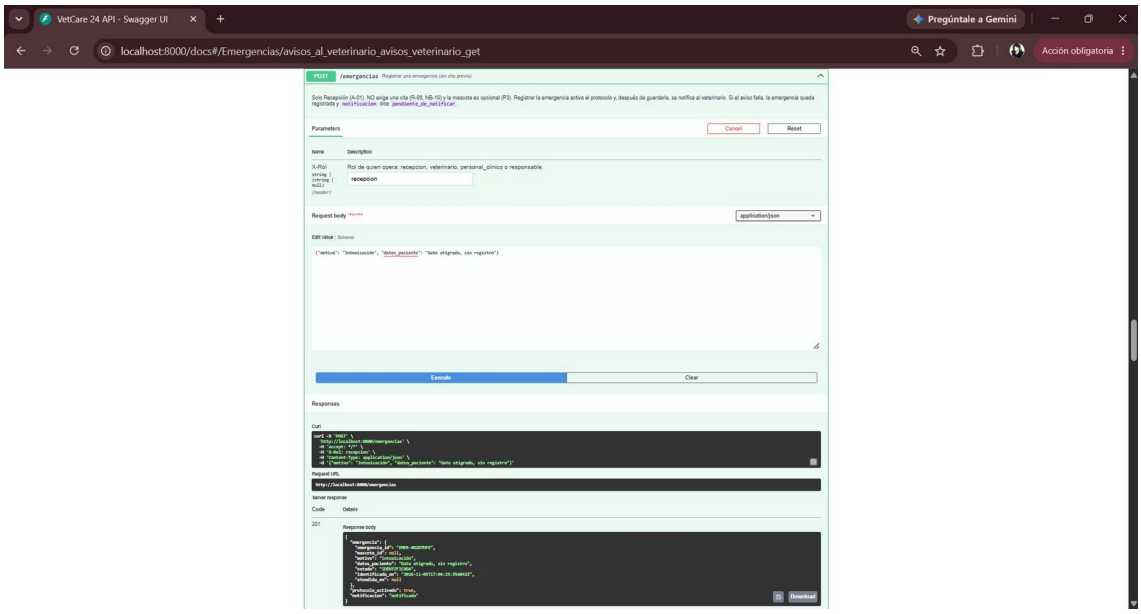


Figura 65. *CU-05 · POST /emergencias sin mascota registrada → 201.*

Responde 403 con código ACCESO_DENEGADO. Con ello se comprueba que solo Recepción registra emergencias.

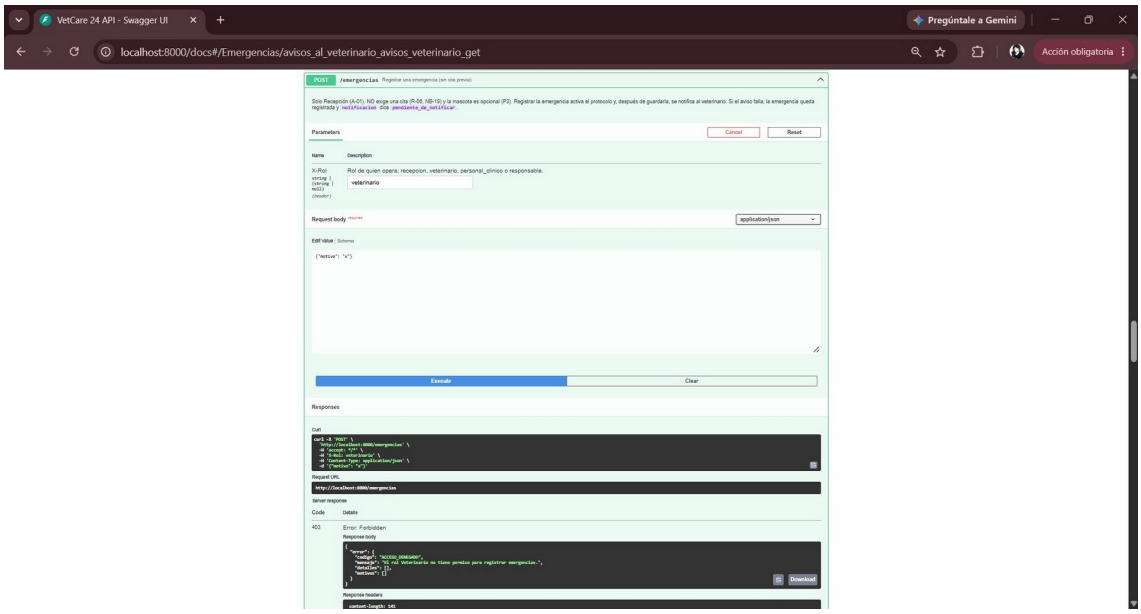


Figura 66. *CU-05 · POST /emergencias como veterinario → 403.*

La emergencia pasa de IDENTIFICADA a EN_ATENCION y se registra la hora de atención.

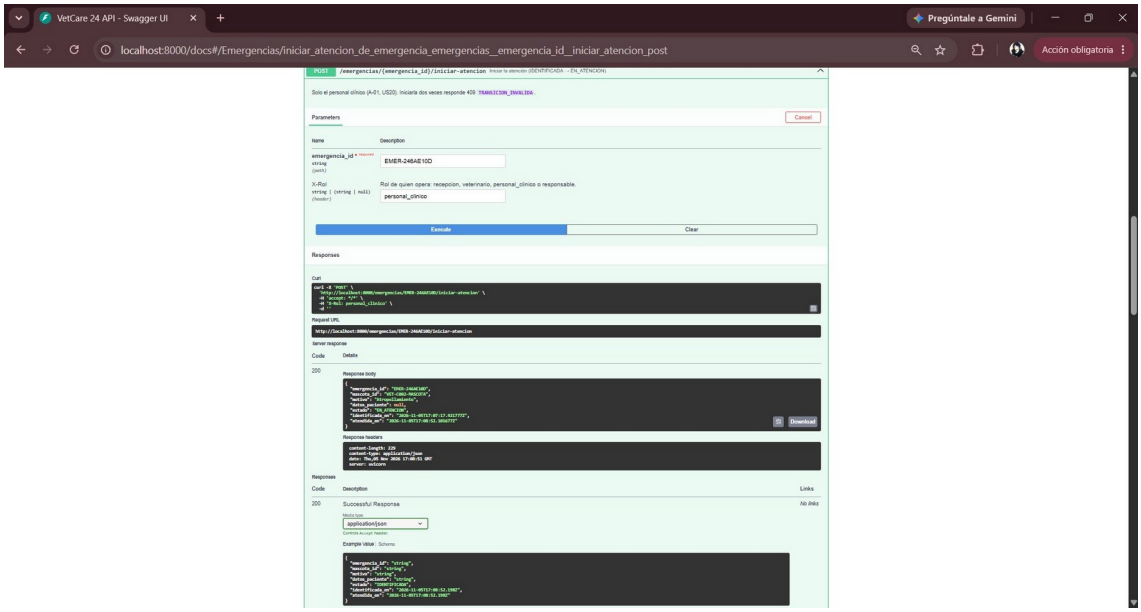


Figura 67. *CU-05 · iniciar-atencion como personal clínico → 200 EN_ATENCION.*

La máquina de estados impide repetir la transición y responde 409 Conflict con un mensaje en términos de la emergencia.

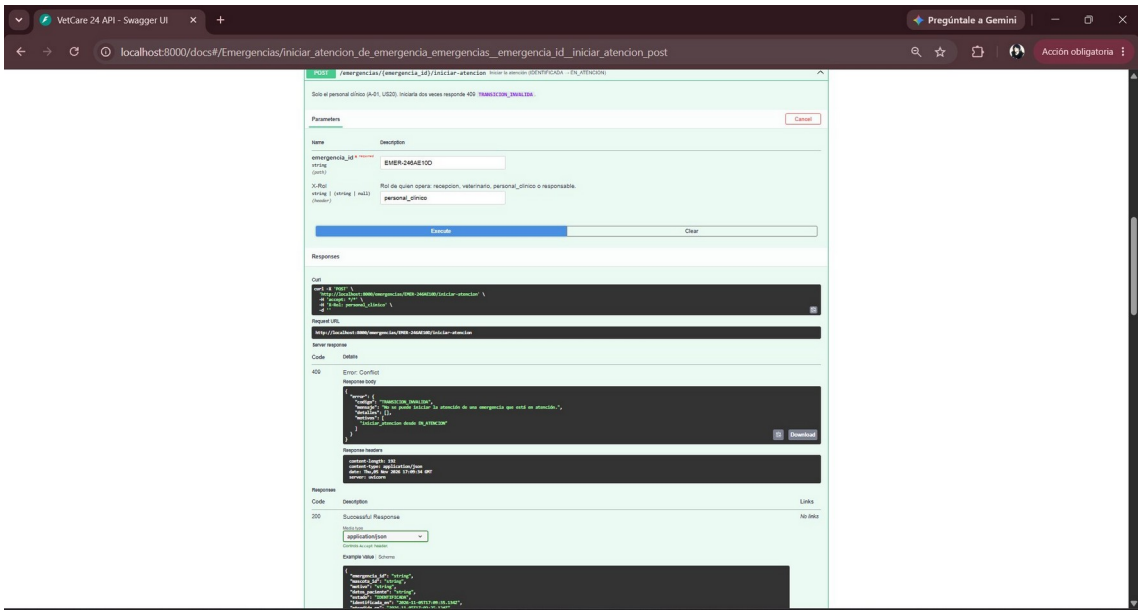


Figura 68. *CU-05 · iniciar-atencion repetido → 409 TRANSICION_INVALIDA.*

Responde 200 con los eventos publicados, entre ellos EmergenciaIdentificada. Con ello se comprueba que los eventos se guardan.

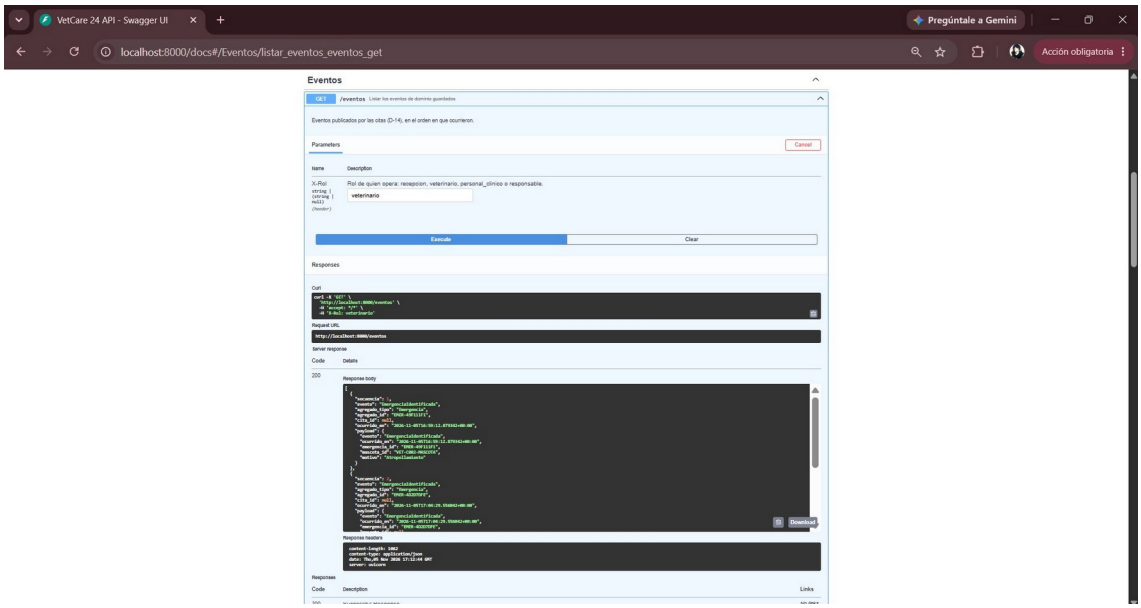


Figura 69. *CU-05 · GET /eventos con los eventos de dominio.*

La respuesta incluye la lista de roles válidos. Esto demuestra la validación del encabezado X-Rol.

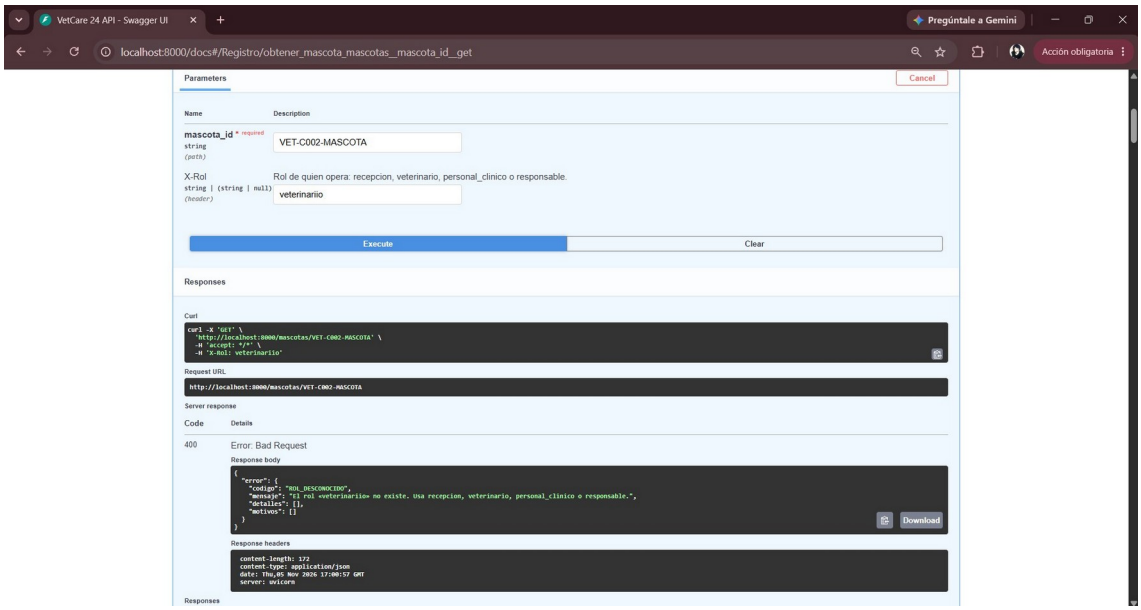


Figura 70. *Errores de entrada · rol mal escrito → 400 ROL_DESCONOCIDO.*

La regla de negocio rechaza el motivo vacío con 422 y el mensaje «Falta indicar el motivo». Es el caso de error del registro de emergencias (CP-08).

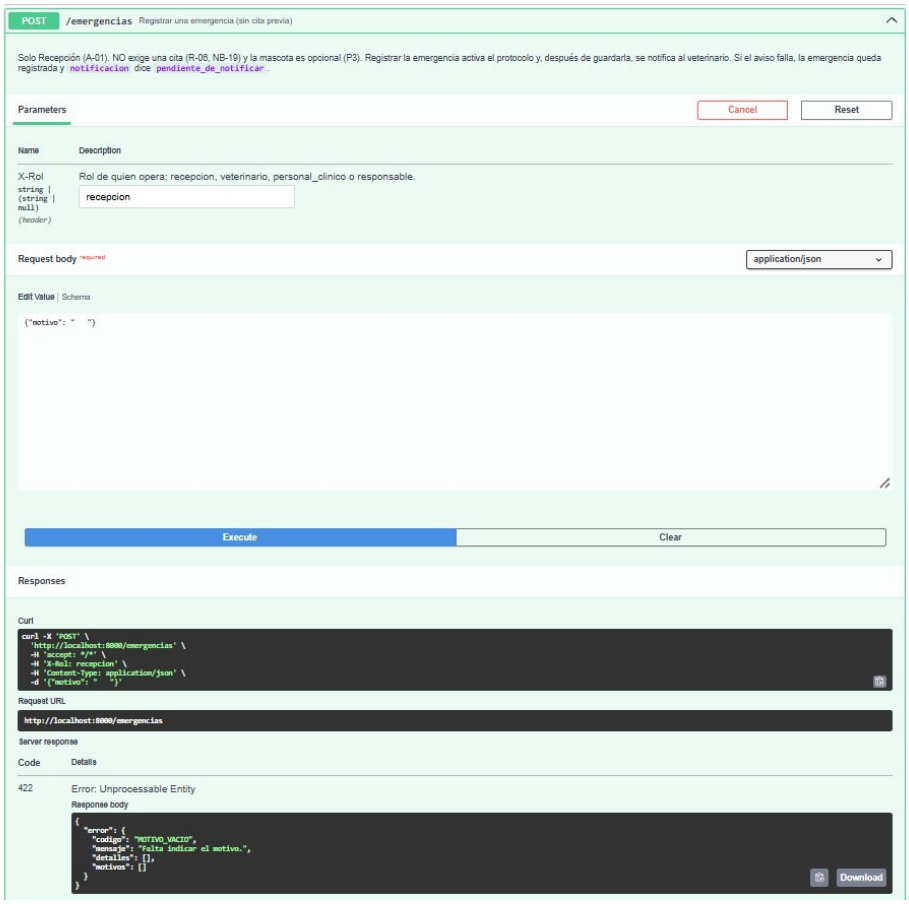


Figura 71. *Errores de entrada · motivo en blanco → 422 MOTIVO_VACIO.*

2.10.4 Persistencia en PostgreSQL

En la captura se observan dos filas IDENTIFICADA y una EN_ATENCION con atendida_en, tres eventos EmergenciaIdentificada en eventos_dominio y el esquema con restricciones CHECK (motivo no vacío, estado válido, coherencia entre estado y atendida_en) y clave foránea a mascotas. Con ello se comprueba que la base rechaza datos imposibles aunque el código falle.

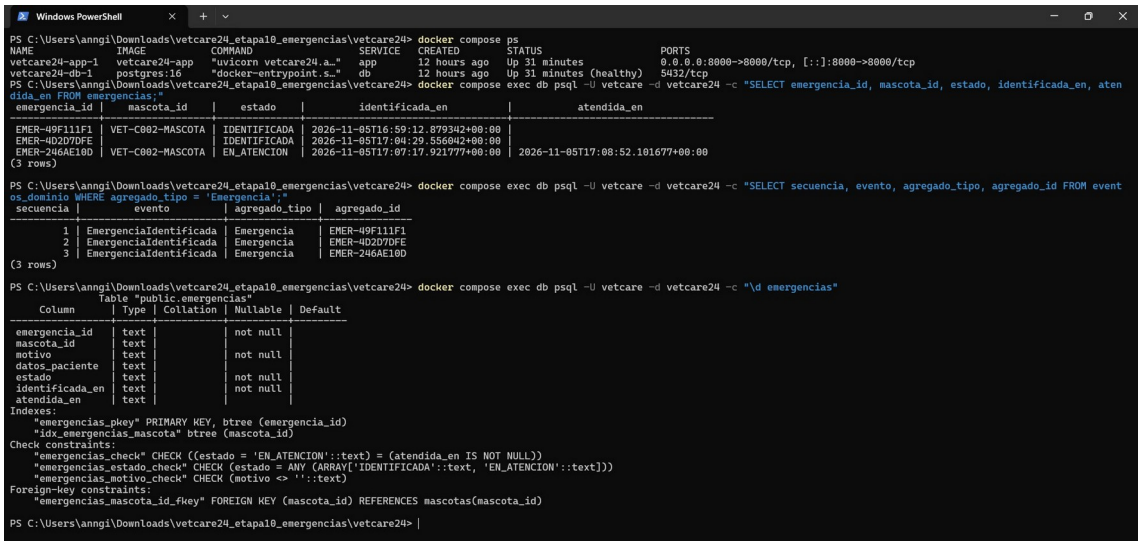


Figura 72. *Consultas psql: tres emergencias, tres eventos y el esquema con restricciones.*

2.10.5 Evidencia de los 20 escenarios por HTTP

Los generadores de evidencia comparan lo esperado con lo obtenido por la API.

Todos los escenarios coinciden y la exportación del contrato registra 23 rutas.

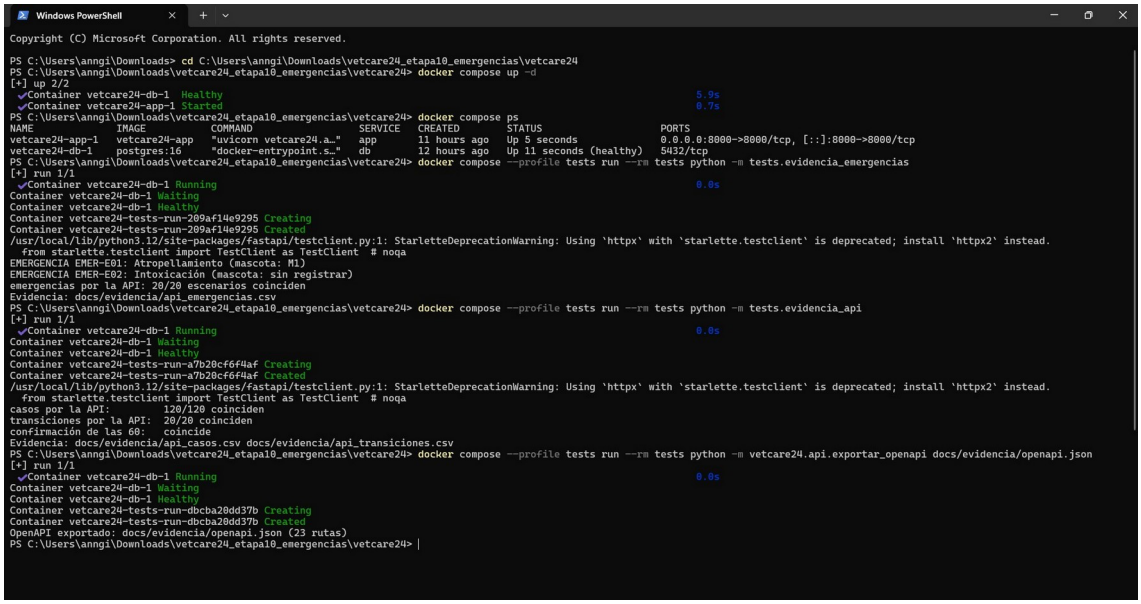


Figura 73. *Evidencia por HTTP: 20/20 escenarios, 120/120 casos y 20/20 transiciones; OpenAPI con 23 rutas.*

Cada fila indica rol, método, ruta, código HTTP esperado y obtenido, resultado y regla. El detalle completo se reproduce en la tabla 4.2.C de este reporte.

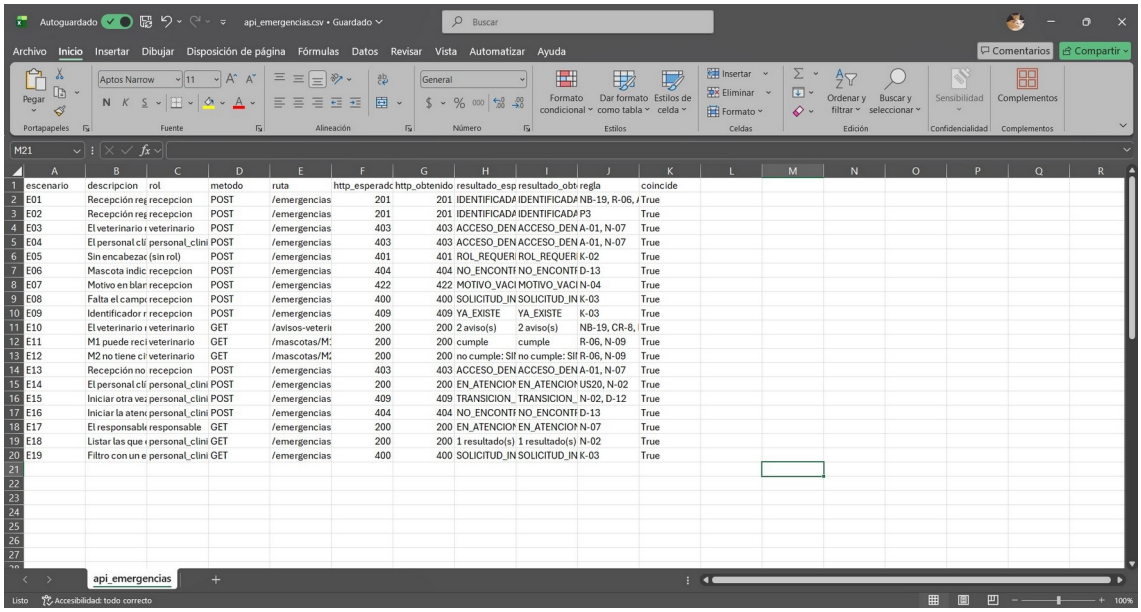


Figura 74. *api_emergencias.csv abierto en Excel: 20 escenarios, todos «coincide = True».*

2.10.6 Pipeline del proyecto

Resumen del workflow con los tres trabajos encadenados y 2 artefactos.

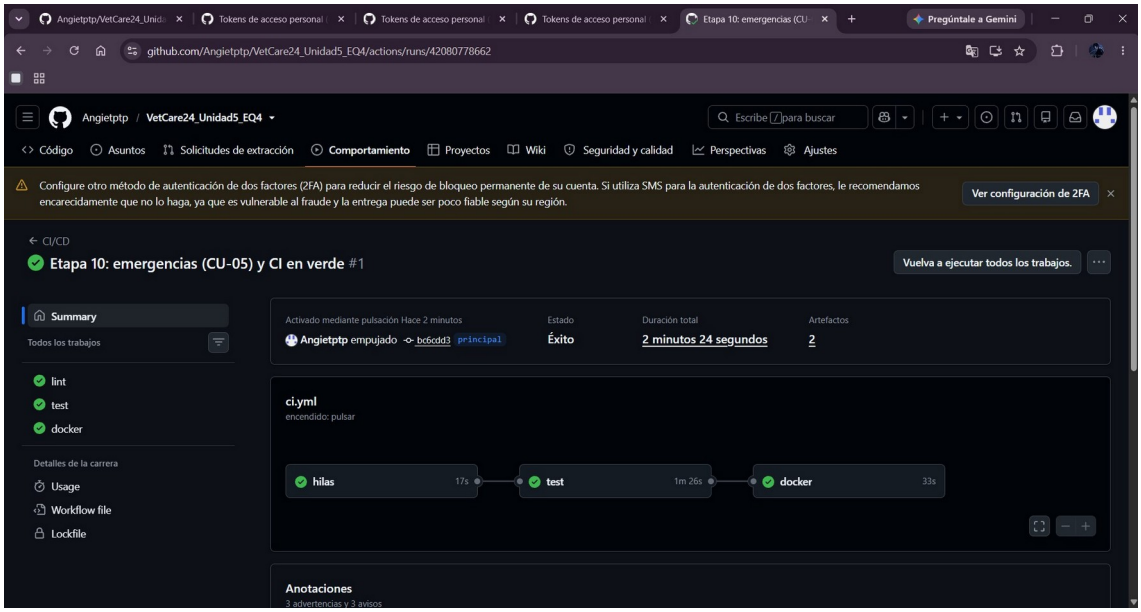


Figura 75. *Ejecución #1 de ci.yml: lint, test y docker en verde (2 min 24 s).*

Registro del paso upload-artifact que publica el reporte HTML de cobertura.

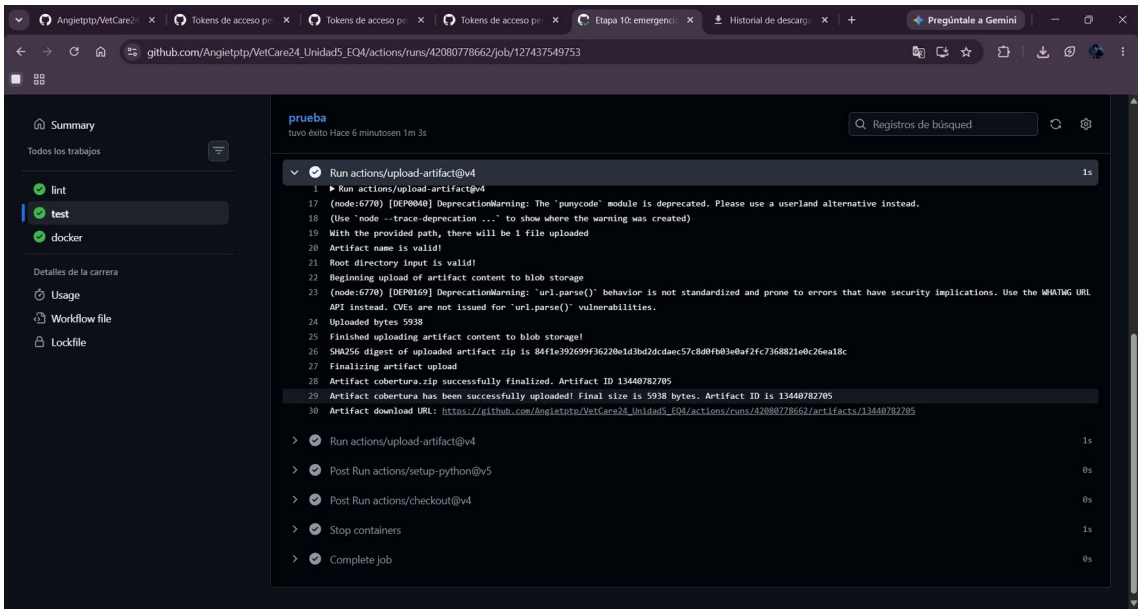


Figura 76. *Job test: se sube el artefacto cobertura.zip (5938 bytes).*

Segundo upload-artifact: los CSV de evidencia del proyecto quedan descargables desde la ejecución.

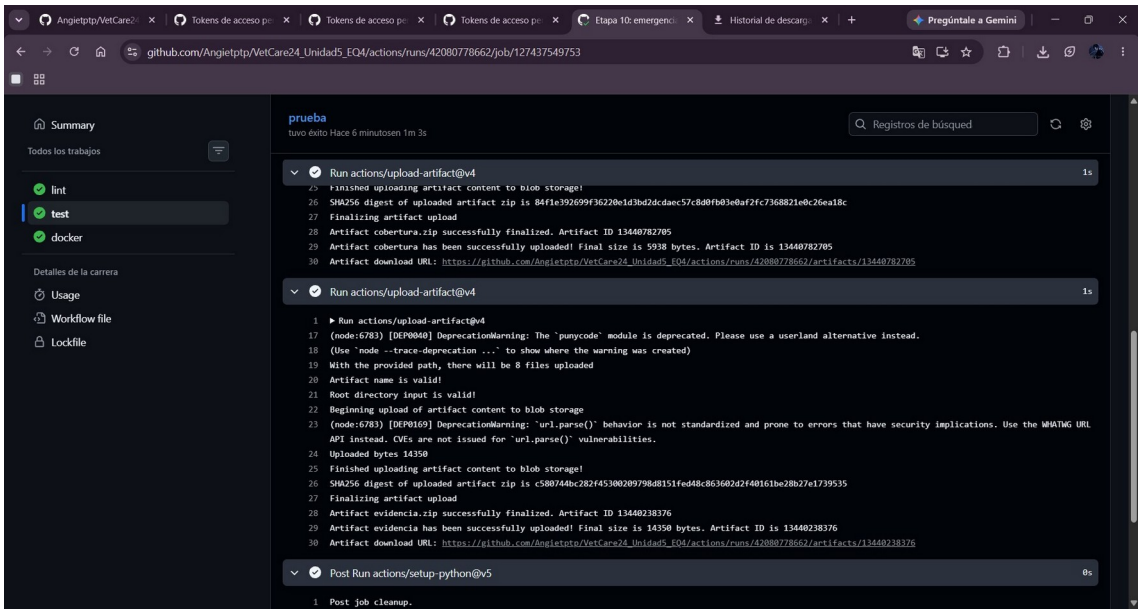


Figura 77. *Job test: se sube el artefacto evidencia.zip (14 350 bytes).*

En la captura se ven los pasos «Run pytest» y «Generar evidencia del proyecto (120 casos, 20 transiciones, CU-05 y OpenAPI)» seguidos de la subida de artefactos.

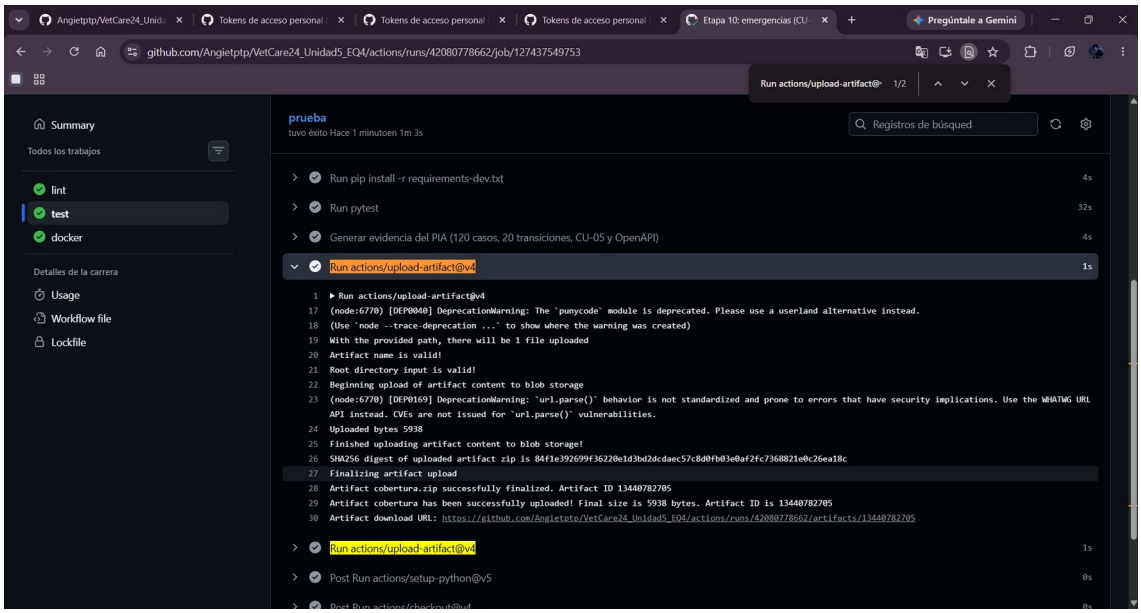


Figura 78. *Registro del job test: pytest, generación de la evidencia y artefactos.*

Una segunda ejecución del mismo commit también termina en éxito, lo que indica que el pipeline es repetible.

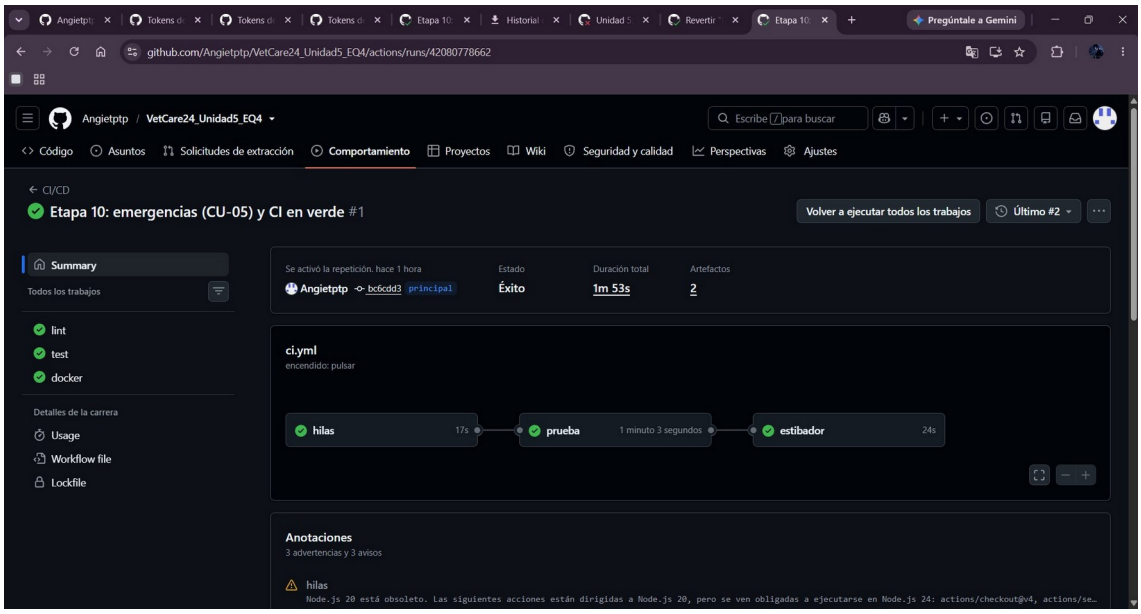


Figura 79. *Resumen de la re-ejecución sobre el commit bc6cdd3: verde en 1 min 53 s.*

2.11 Matriz de la Unidad 5 ejecutada con pytest en Docker (E9)

Para la matriz de la unidad se escribió una prueba que lee el archivo CSV y ejecuta, de forma parametrizada, los casos CP-01 a CP-09. Al correrla dentro de Docker se presentaron dos errores consecutivos, debido a que la imagen de pruebas no incluía la carpeta docs/u5. Se conservan en el reporte porque muestran su resolución: fue suficiente añadir una línea al Dockerfile y reconstruir la imagen.

En la captura se listan matriz_pia_u5.csv y test_u5_matriz.py en la carpeta, pero el contenedor reporta «ERROR: file or directory not found». La imagen era anterior a la copia de los archivos y debía reconstruirse.

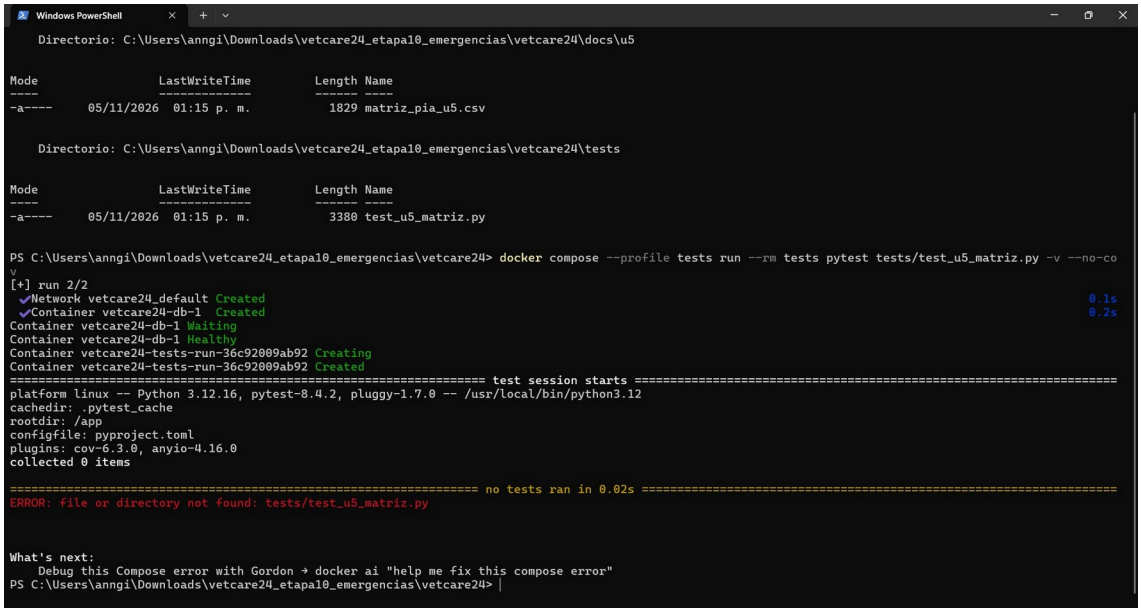
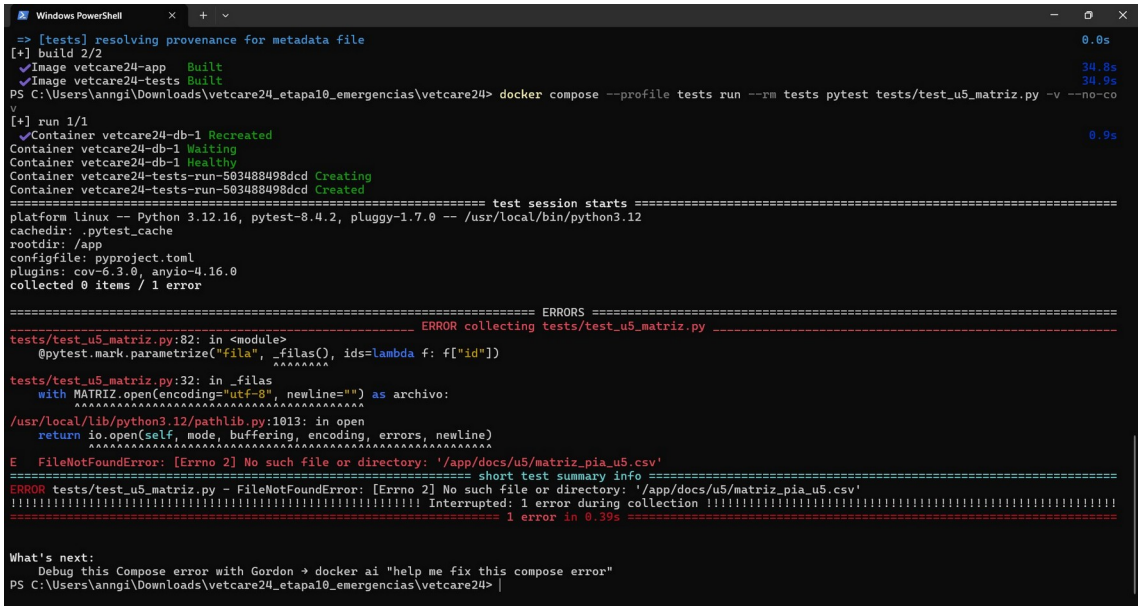


Figura 80. *Primer error: pytest no encuentra tests/test_u5_matriz.py dentro de Docker.*

Tras reconstruir, el archivo de prueba existe pero falta el CSV, porque el Dockerfile copiaba solo src, data y tests. Se añadió «COPY docs/u5 ./docs/u5» en la etapa de desarrollo y se reconstruyó la imagen.


Windows PowerShell
=> [tests] resolving provenance for metadata file 0.0s
[+] build 2/2
✔Image vetcare24-app Built 34.8s
✔Image vetcare24-tests Built 34.9s
PS C:\Users\anngi\Downloads\vetcare24_etapa10_emergencias\vetcare24> docker compose --profile tests run --rm tests pytest tests/test_u5_matriz.py -v --no-co
v
[+] run 1/1
✔Container vetcare24-db-1 Recreated 0.9s
Container vetcare24-db-1 Waiting
Container vetcare24-db-1 Healthy
Container vetcare24-tests-run-503488498dcd Creating
Container vetcare24-tests-run-503488498dcd Created
test session starts
platform linux -- Python 3.12.16, pytest-8.4.2, pluggy-1.7.0 -- /usr/local/bin/python3.12
cachedir: .pytest_cache
rootdir: /app
configfile: pyproject.toml
plugins: cov-6.3.0, anyio-4.16.0
collected 0 items / 1 error
ERRORS
ERROR collecting tests/test_u5_matriz.py
tests/test_u5_matriz.py:82: in <module>
@pytest.mark.parametrize("fila", _filas(), ids=lambda f: f["id"])
tests/test_u5_matriz.py:32: in _filas
with MATRIZ.open(encoding="utf-8", newline="") as archivo:
/usr/local/lib/python3.12/pathlib.py:1013: in open
return io.open(self, mode, buffering, encoding, errors, newline)
E FileNotFoundError: [Errno 2] No such file or directory: '/app/docs/u5/matriz_pia_u5.csv'
short test summary info
ERROR tests/test_u5_matriz.py - FileNotFoundError: [Errno 2] No such file or directory: '/app/docs/u5/matriz_pia_u5.csv'
!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!

Resultado final: «584 passed, 1 warning in 138.58s» y «Required test coverage of 80% reached. Total coverage: 98.81%». Los archivos de las reglas de la matriz (reglas_cita, emergencia, reglas_emergencia, servicios_emergencias, notificador) están al 100 %.

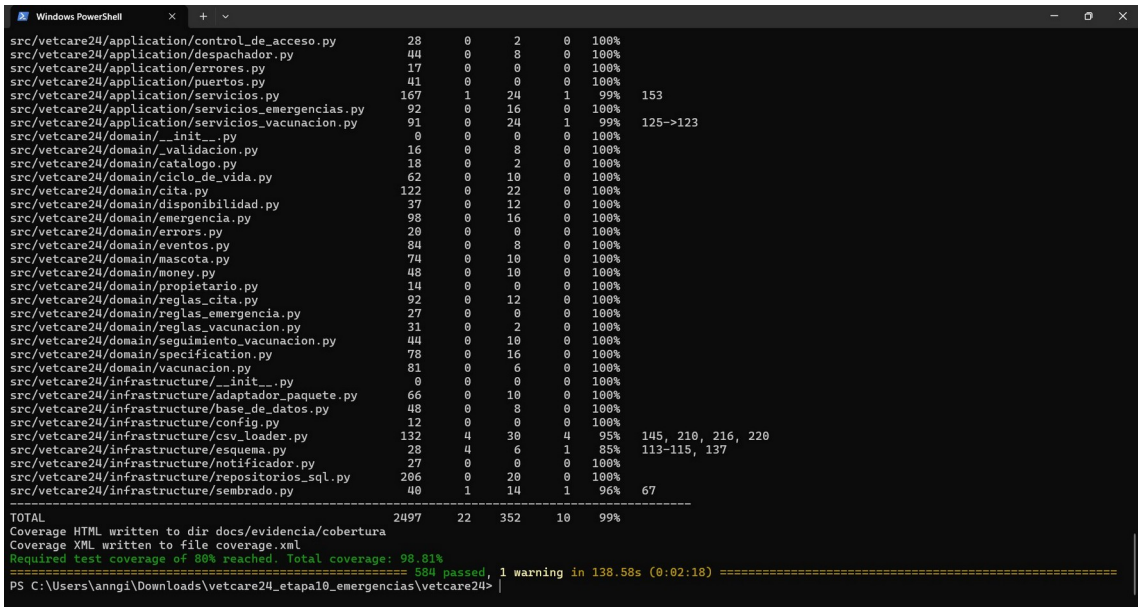


Figura 82. *Suite completa con la matriz incorporada: 584 passed y cobertura 98.81 %.*

2.12 Subida del repositorio del proyecto y errores de autenticación

La última dificultad se presentó al subir el proyecto a GitHub. Dado que ya no se acepta la contraseña para operaciones de Git, fue necesario generar un token personal con los permisos repo y workflow. Los errores surgidos durante el proceso quedaron registrados en las capturas; el token no aparece en ninguna de ellas.

Repositorio público y vacío con las instrucciones de configuración rápida.

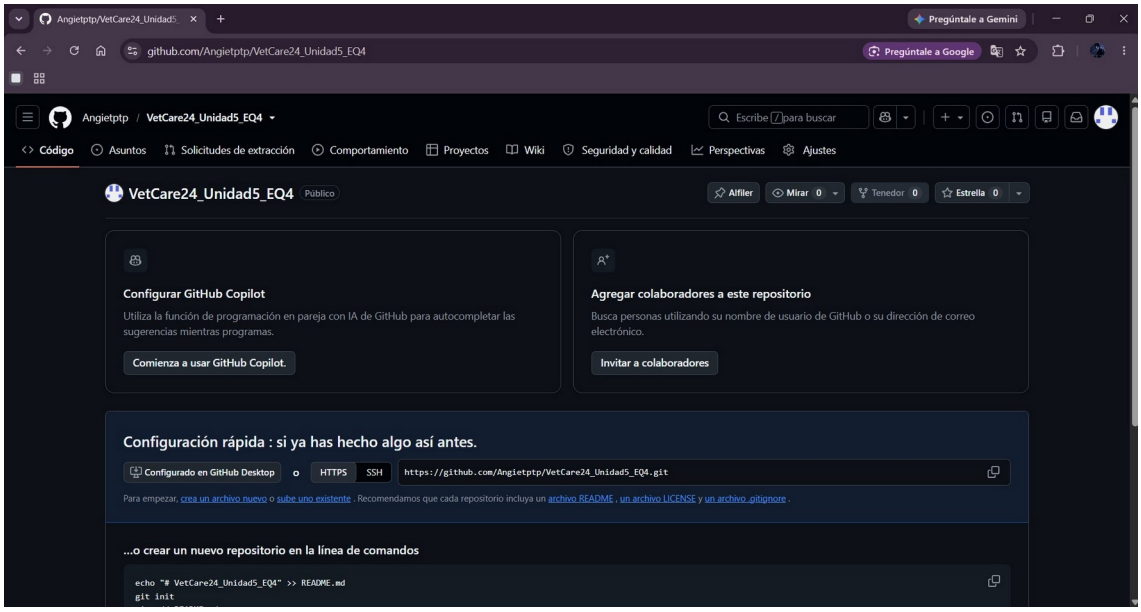


Figura 83. *Repositorio VetCare24_Unidad5_EQ4 creado en GitHub.*

La contraseña no se muestra al escribirla; el usuario sí aparece.

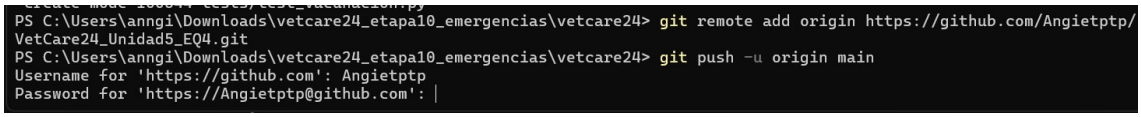


Figura 84. *git push -u origin main solicita usuario y contraseña.*

GitHub rechaza la contraseña: es el primer error de autenticación.

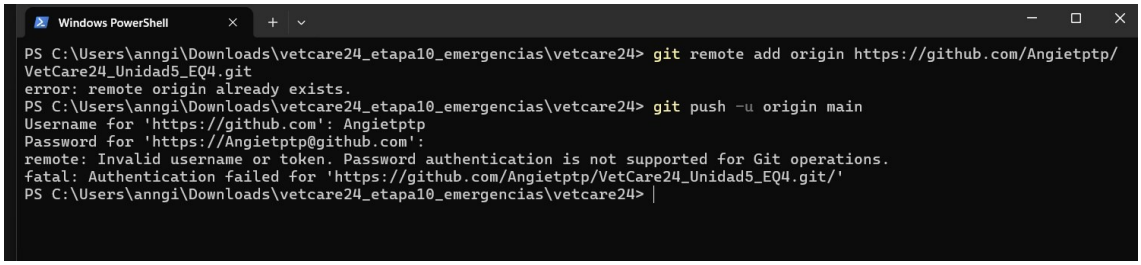


Figura 85. *«Password authentication is not supported for Git operations».*

El mensaje «refusing to allow a Personal Access Token to create or update workflow … without workflow scope» indica que el token no tenía ese alcance; se creó uno con repo y workflow.

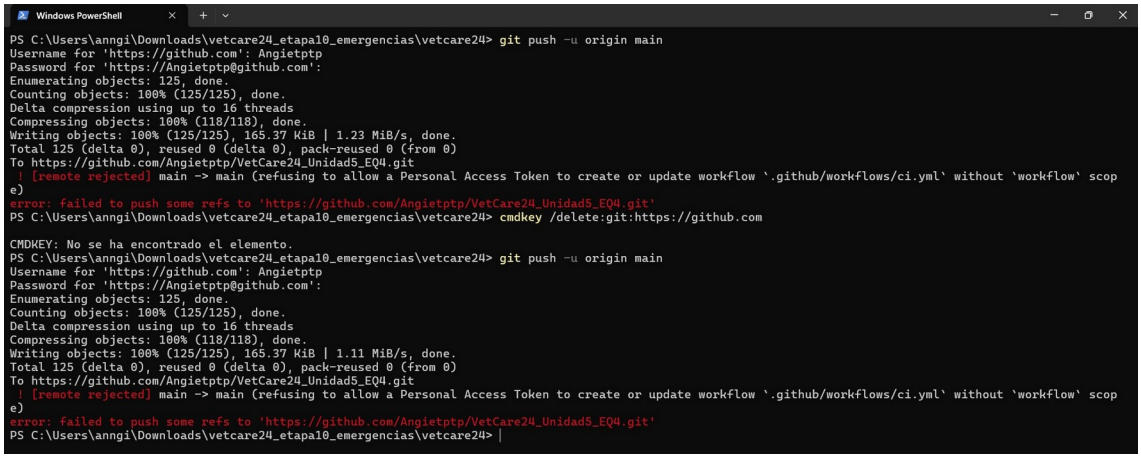


Figura 86. *Push rechazado por falta del permiso workflow.*

125 objetos enviados y rama main enlazada con origin/main.



Figura 87. *Push exitoso: «[new branch] main -> main».*

2.13 Ejecución final de todas las pruebas del laboratorio

Para cerrar el trabajo se corrió el conjunto completo del laboratorio con una sola instrucción, una vez que se agregó la prueba parametrizada que lee los casos de los archivos CSV. Primero se corrigieron con Ruff los avisos de estilo que aparecieron en las pruebas de seguridad (el uso del alias datetime.UTC) y después se repitió la ejecución.

En la captura se observa que las 32 pruebas pasan en 1.62 segundos, incluidas test_gateway_rejected_vip_200 y test_cannot_confirm_twice. La cobertura total es del 100 % con 53 declaraciones y ninguna sin cubrir, y el resumen del script indica que Ruff y las pruebas terminaron correctamente. Los avisos ResourceWarning sobre conexiones de SQLite sin cerrar no afectan el resultado: provienen de las bases en memoria de las pruebas de repositorio.

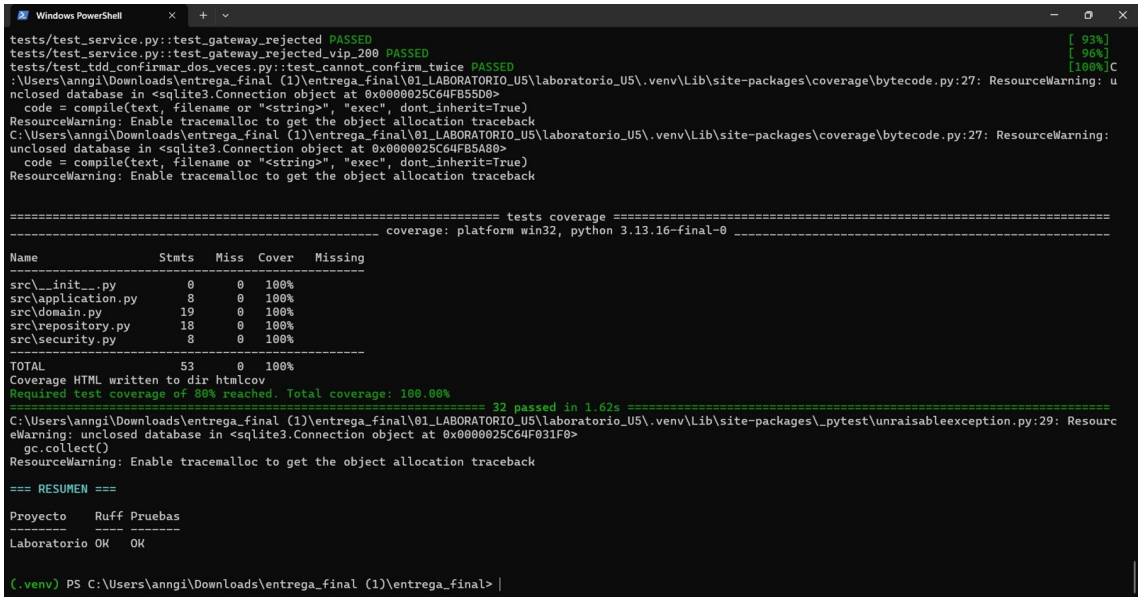


Figura 88. *Ejecución final del laboratorio: 32 pruebas, cobertura del 100 % y revisión de Ruff en verde.*

3. Resultados generales

En este apartado se concentran los resultados de todas las ejecuciones realizadas, tanto del proyecto base como de VetCare 24. Los valores provienen de las capturas del apartado anterior y la columna «Evidencia» indica la figura en que puede comprobarse cada uno.

3.1 Resultados por paso del laboratorio (proyecto base)

Tabla 4. *Resultados de los pasos E1 a E8.*

Paso	Actividad	Pruebas (antes → después)	Resultado	Evidencia
E1	Instalación y suite inicial	— → 13	13 passed in 8.64s	Fig. 1
E2	Dos pruebas de dominio	5 → 7	7 passed in 0.13s	Fig. 5
E3	TDD: RED	7 → 8	1 failed, 7 passed (DID NOT RAISE)	Fig. 6
E3	TDD: GREEN y suite completa	8 → 16 (suite)	8 passed; 16 passed in 1.16s	Fig. 8
E4	Repositorio con id = 2	2 → 3	3 passed in 1.07s	Fig. 10
E5	Mock con rechazo VIP de 200	2 → 3	3 passed in 0.09s	Fig. 11
E6	Autorización: 24 h exactas y admin	4 → 6	6 passed in 0.05s	Fig. 13
E7	Ruff	—	13 errores → All checks passed	Fig. 15
E7	Cobertura	—	20 passed; 100.00 % (umbral 80 %)	Fig. 16
E8	CI #1 / #2 / #3 / #4	—	roja / verde / roja / verde	Fig. 19, Fig. 21, Fig. 22, Fig. 23

3.2 Resultados por etapa del proyecto

Tabla 5. *Evolución de pruebas y cobertura en el proyecto.*

Etapa	Tema	Pruebas	Cobertura	Evidencia
1	Entorno base	4	100 %	Fig. 26
2	Importador y oráculo	—	95.65 %	Fig. 28
3	Dominio	—	98.19 %	Fig. 30
4	Specifications	—	98.67 %	Fig. 32
5	Máquina de estados y capacidad	—	98.91 %	Fig. 35
6	Domain Events	215	99.05 %	Fig. 38
7	Aplicación, persistencia y API	300	99.12 %	Fig. 41
8	Revisión de la API	394	98.54 %	Fig. 47
9	Vacunación	491	98.61 %	Fig. 50
10	Emergencias (CU-05)	574	98.81 %	Fig. 52
10 + U5	Emergencias + matriz de la Unidad 5	584	98.81 %	Fig. 82

3.3 Resultados de la evidencia por HTTP y de la matriz

Tabla 6. *Verificaciones de la API y de la matriz.*

Verificación	Esperado	Obtenido	Resultado	Evidencia
Casos de la API contra el oráculo	120	120	120/120 coinciden	Fig. 48
Transiciones de la API contra el oráculo	20	20	20/20 coinciden	Fig. 48
Escenarios de emergencias por HTTP	20	20	20/20 coinciden	Fig. 73
Casos de la matriz de la Unidad 5 (CP-01 a CP-11)	11	11	11 PASSED (9 parametrizados + 2 de integración)	Fig. 82
Suite completa del proyecto	584	584	584 passed	Fig. 82

Contrato OpenAPI	23 rutas	23 rutas	Exportado	Fig. 73

3.4 Resultados de integración continua

Tabla 7. *Historial de ejecuciones de los pipelines.*

Repositorio	Ejecución	Resultado	Duración	Observación	Evidencia
CLN_U5_Laboratorio_EQ4	#1 Unidad 5: pruebas, TDD y CI	Roja	13 s (job 9 s)	ModuleNotFoundError: src	Fig. 19
CLN_U5_Laboratorio_EQ4	#2 Corrige CI: pythonpath…	Verde	job 11 s	Hallazgo corregido	Fig. 21
CLN_U5_Laboratorio_EQ4	#3 Rompe a propósito una prueba…	Roja	job 14 s	Fallo controlado	Fig. 22
CLN_U5_Laboratorio_EQ4	#4 Revertir «Rompe a propósito…»	Verde	job 12 s	git revert	Fig. 23
VetCare24_Unidad5_EQ4	#1 Etapa 10: emergencias y CI en verde	Verde	2 min 24 s	lint, test y docker; 2 artefactos	Fig. 75
VetCare24_Unidad5_EQ4	Re-ejecución (commit bc6cdd3)	Verde	1 min 53 s	Pipeline repetible	Fig. 79

3.5 Indicadores globales

Tabla 8. *Indicadores globales.*

Indicador	Valor	Meta	Cumple
Pruebas del proyecto base	20 de 20 PASSED	Todas	Sí
Cobertura del proyecto base	100.00 %	≥ 80 %	Sí
Pruebas del proyecto	584 de 584 PASSED	Todas	Sí
Cobertura del proyecto	98.81 %	≥ 80 %	Sí
Casos de la matriz (éxito / error / límite)	3 / 3 / 3 + 2 de integración	≥ 9 casos	Sí
Escenarios por HTTP	20/20 + 120/120 + 20/20	Todos coinciden	Sí
Ruff	All checks passed	Sin errores	Sí
Pipeline con secuencia verde–roja–verde	Realizada (#1 a #4)	Demostrar detección y recuperación	Sí
Incidencias de procedimiento	10 registradas y resueltas	—	Documentadas
Capturas numeradas en este reporte	88	—	—

3.6 Lectura de los resultados

En términos generales, los resultados reflejan un proyecto estable: todas las pruebas pasan, la cobertura se mantiene muy por encima del mínimo y los dos flujos de integración terminan en verde. El mayor aprendizaje provino de las fallas, tanto las provocadas intencionalmente como las que surgieron de forma imprevista: la fase roja del desarrollo guiado mostró que la prueba nueva era capaz de fallar; la primera ejecución del flujo en GitHub reveló una situación que en el equipo local no se advertía; la tercera comprobó que el flujo detecta una regresión; y los errores con Docker evidenciaron que la imagen de pruebas debe incluir todos los archivos que las pruebas leen. En la corrida final del laboratorio, con la prueba de los casos CSV incluida, pasaron las 32 pruebas con cobertura del 100 %.

Como trabajo posterior queda cubrir los escenarios que una cobertura del 100 % no demuestra, como una excepción de la pasarela durante el cobro o un identificador duplicado en el repositorio, y corregir el problema detectado en el servicio de cobro, donde el pedido queda confirmado aunque el pago sea rechazado.

4. Matriz de prueba

Para concluir el reporte se elaboró la matriz de prueba, organizada en cuatro grupos de tablas. En primer lugar se enlistan las **reglas** verificadas; después los **casos**, con su entrada y el resultado esperado; luego la **distribución**, que muestra la cantidad de casos por tipo, regla, capa y archivo; y por último las **evidencias**, que indican dónde se comprueba cada caso. Los identificadores permiten seguir una regla hasta su caso, su prueba de pytest y su captura.

4.1 Tabla de reglas

Tabla 9. *4.1 Reglas probadas (30 reglas: 14 del proyecto base, 14 del proyecto y 2 de calidad).*

ID	Regla	Proyecto	Código	Resultado esperado de la regla
RN-01	Un pedido con total ≤ 0 no puede confirmarse	Base	domain.py · Order.confirm	Error de dominio: ValueError("total must be positive")
RN-02	Un cliente con deuda no puede confirmar un pedido	Base	domain.py · Order.confirm	CustomerHasDebt("outstanding debt")
RN-03	Un pedido VIP recibe 15 % de descuento	Base	domain.py · Order.confirm	discount = round(total × 0.15, 2); importe = total − descuento
RN-04	Un cliente regular no recibe descuento	Base	domain.py · Order.confirm	discount = 0
RN-05	Un pedido ya confirmado no puede confirmarse de nuevo	Base (TDD)	domain.py · Order.confirm	ValueError("already confirmed")
RN-06	Si la pasarela aprueba, el servicio devuelve el importe	Base	application.py · checkout	Importe cobrado = importe del pedido
RN-07	Si la pasarela rechaza, el servicio lanza error	Base	application.py · checkout	RuntimeError("payment rejected")
RN-08	El importe enviado a la pasarela incluye el descuento	Base	application.py · checkout	charge(170) para un VIP de 200
RN-09	Commit hace persistente el pedido	Base	repository.py	La fila existe tras commit
RN-10	Rollback descarta el pedido pendiente	Base	repository.py	La consulta devuelve None
RN-11	Un pedido SHIPPED nunca se cancela, ni siquiera el administrador	Base	security.py · can_cancel	False
RN-12	El administrador cancela pedidos que no están SHIPPED	Base	security.py · can_cancel	True
RN-13	El propietario cancela solo si pasaron menos de 24 h	Base	security.py · can_cancel	True antes de 24 h; False a las 24 h o más
RN-14	Un usuario que no es propietario ni administrador no cancela	Base	security.py · can_cancel	False
RN-15	Capacidad suficiente al confirmar una cita (NB-08 / D-12)	VetCare 24	reglas_cita.py · CapacidadSuficiente	cumple si cantidad ≤ capacidad; si no, CAPACIDAD_INSUFICIENTE:S1
RN-16	Total mínimo de $1500 para el beneficio (D-05)	VetCare 24	reglas_cita.py · TotalMinimo	cumple si precio × cantidad ≥ 1500; si no, TOTAL_INSUFICIENTE
RN-17	La emergencia exige un motivo no vacío (N-04)	VetCare 24	emergencia.py · Emergencia	InvarianteViolada MOTIVO_VACIO (HTTP 422)
RN-18	La mascota de una emergencia es opcional (P3)	VetCare 24	emergencia.py · Emergencia	Estado IDENTIFICADA sin mascota
RN-19	Atención permitida si hay cita confirmada O emergencia activa (R-06)	VetCare 24	reglas_emergencia.py	cumple = TieneCitaConfirmada OR TieneEmergenciaActiva
RN-20	Solo Recepción registra emergencias (A-01, N-07)	VetCare 24	control_de_acceso.py	HTTP 403 ACCESO_DENEGADO para otros roles
RN-21	Solo el personal clínico inicia la atención (US20)	VetCare 24	control_de_acceso.py	HTTP 403 para otros roles
RN-22	No se puede iniciar dos veces la atención (N-02)	VetCare 24	emergencia.py · transiciones	HTTP 409 TRANSICION_INVALIDA
RN-23	Se avisa al veterinario; si el aviso falla, el registro se conserva (NB-19 / CR-8)	VetCare 24	servicios_emergencias.py	Emergencia guardada; notificación PENDIENTE

RN-24	Una mascota indicada que no existe no se registra (D-13)	VetCare 24	servicios_emergencias.py	HTTP 404; no se guarda ni se notifica
RN-25	Un identificador repetido no se acepta	VetCare 24	servicios_emergencias.py	HTTP 409 (ya existe)
RN-26	Sin rol responde 401 y con rol desconocido 400 (K-02 / K-03)	VetCare 24	api/errores_http.py	401 / 400 ROL_DESCONOCIDO
RN-27	Los eventos de dominio quedan guardados (D-14 / N-06)	VetCare 24	despachador y repositorios	EmergenciaIdentificada en eventos_dominio
RN-28	La base de datos rechaza datos imposibles (CHECK)	VetCare 24	esquema.py	Error de integridad ante motivo vacío o estado inconsistente
RN-29	La cobertura mínima es 80 %	Calidad	pyproject.toml · --cov-fail-under	La ejecución falla por debajo del umbral
RN-30	Cada push ejecuta Ruff, pruebas y compilación	Calidad	ci.yml	El pipeline queda rojo ante cualquier fallo

4.2 Tabla de casos

4.2.A Proyecto base: 20 pruebas pytest

Tabla 10. *4.2.A Casos del proyecto base (todos PASSED).*

ID	Prueba pytest	Entrada	Resultado esperado	Tipo	Regla
C-01	test_order.py::test_vip_discount	Order(100, vip=True).confirm()	85; status CONFIRMED	Éxito	RN-03
C-02	test_order.py::test_regular_no_discount	Order(100).confirm()	100	Éxito	RN-04
C-03	test_order.py::test_customer_debt	Order(100).confirm(customer_debt=50)	CustomerHasDebt	Error	RN-02
C-04	test_order.py::test_invalid_total[0]	Order(0).confirm()	ValueError	Límite	RN-01
C-05	test_order.py::test_invalid_total[-1]	Order(-1).confirm()	ValueError	Error	RN-01
C-06	test_order.py::test_vip_discount_200	Order(200, vip=True).confirm()	170; discount 30; CONFIRMED	Éxito	RN-03
C-07	test_order.py::test_regular_customer_200_no_discount	Order(200).confirm()	200; discount 0; CONFIRMED	Éxito	RN-04
C-08	test_order.py::test_cannot_confirm_twice	confirm() dos veces seguidas	ValueError("already confirmed")	Error	RN-05
C-09	test_repository.py::test_repository_commit	save(1, Order(120)) + commit	total == 120	Integración	RN-09
C-10	test_repository.py::test_repository_rollback	save(1, Order(120)) + rollback	consulta → None	Integración	RN-10
C-11	test_repository.py::test_repository_commit_order_250	Order(250).confirm(); save(2) + commit	id 2; total 250; CONFIRMED	Integración	RN-09
C-12	test_service.py::test_gateway_approved	Mock charge → True; checkout(Order(100))	100; charge(100) una vez	Éxito	RN-06
C-13	test_service.py::test_gateway_rejected	Mock charge → False	RuntimeError	Error	RN-07
C-14	test_service.py::test_gateway_rejected_vip_200	Mock charge → False; Order(200, vip=True)	RuntimeError("payment rejected"); charge(170)	Error	RN-07, RN-08
C-15	test_security.py::test_owner_within_24h	NEW; propietario; 1 h	Permitido (True)	Éxito	RN-13
C-16	test_security.py::test_stranger_denied	NEW; otro usuario; no admin	Denegado (False)	Error	RN-14
C-17	test_security.py::test_shipped_denied_even_admin	SHIPPED; admin	Denegado (False)	Error	RN-11
C-18	test_security.py::test_expired_denied	NEW; propietario; 25 h	Denegado (False)	Error	RN-13
C-19	test_security.py::test_admin_cancels_new_order_of_another_user	NEW; admin; otro usuario; 48 h	Permitido (True)	Éxito	RN-12
C-20	test_security.py::test_owner_exactly_24h_denied	NEW; propietario; 24 h exactas	Denegado (False)	Límite	RN-13

4.2.B VetCare 24: matriz de la Unidad 5 (docs/u5/matriz_pia_u5.csv)

Tabla 11. *4.2.B Matriz CSV de 11 casos (VetCare 24).*

ID	Regla	Entrada	Resultado esperado	Tipo	Prueba pytest	Origen
CP-01	Capacidad suficiente al confirmar una cita	cantidad=3; capacidad=10	cumple	Éxito	test_u5_matriz.py::test_caso_de_la_matriz[CP-01]	NB-08 / D-12
CP-02	Capacidad suficiente al confirmar una cita	cantidad=11; capacidad=10	CAPACIDAD_INSUFICIENTE:S1	Error	test_u5_matriz.py::test_caso_de_la_matriz[CP-02]	NB-08 / D-12
CP-03	Capacidad suficiente al confirmar una cita	cantidad=10; capacidad=10	cumple	Límite	test_u5_matriz.py::test_caso_de_la_matriz[CP-03]	NB-08 / D-12
CP-04	Total mínimo para el beneficio ($1500)	precio=500; cantidad=4	cumple	Éxito	test_u5_matriz.py::test_caso_de_la_matriz[CP-04]	D-05

CP-05	Total mínimo para el beneficio ($1500)	precio=500; cantidad=2	TOTAL_INSUFICIENTE	Error	test_u5_matriz.py::test_caso_de_la_matriz[CP-05]	D-05
CP-06	Total mínimo para el beneficio ($1500)	precio=500; cantidad=3	cumple	Límite	test_u5_matriz.py::test_caso_de_la_matriz[CP-06]	D-05
CP-07	Registro de emergencia sin cita (CU-05)	motivo=Atropellamiento; mascota=M1	IDENTIFICADA	Éxito	test_u5_matriz.py::test_caso_de_la_matriz[CP-07]	NB-19 / R-06
CP-08	Registro de emergencia sin cita (CU-05)	motivo= ; mascota=	MOTIVO_VACIO	Error	test_u5_matriz.py::test_caso_de_la_matriz[CP-08]	NB-19 / N-04
CP-09	Registro de emergencia sin cita (CU-05)	motivo=X; mascota=	IDENTIFICADA	Límite	test_u5_matriz.py::test_caso_de_la_matriz[CP-09]	NB-19 / P3
CP-10	Persistencia: rollback descarta y commit conserva	insertar propietario y deshacer / confirmar	rollback deja 0 filas; commit conserva 1	Integración (BD)	test_base_de_datos.py::test_rollback_deshace_y_commit_conserva	D-14
CP-11	Servicio externo: si el aviso falla se conserva el registro	notificador simulado que falla	notificacion PENDIENTE y emergencia IDENTIFICADA	Integración (simulado)	test_servicios_emergencias.py::test_si_la_notificacion_falla_el_registro_se_conserva	NB-19 / CR-8

4.2.C VetCare 24: 20 escenarios de emergencias ejecutados por HTTP (api_emergencias.csv)

Tabla 12. *4.2.C Escenarios por HTTP (los 20 coinciden con lo esperado).*

ID	Escenario	Rol	Método y ruta	HTTP esp. / obt.	Resultado	Regla
E01	Recepción registra una emergencia de M1, que NO tiene cita	recepcion	POST /emergencias	201 / 201	IDENTIFICADA · protocolo · notificado	NB-19, R-06, A-01
E02	Recepción registra una emergencia sin mascota registrada	recepcion	POST /emergencias	201 / 201	IDENTIFICADA · protocolo · notificado	P3
E03	El veterinario no registra emergencias	veterinario	POST /emergencias	403 / 403	ACCESO_DENEGADO	A-01, N-07
E04	El personal clínico no registra emergencias	personal_clinico	POST /emergencias	403 / 403	ACCESO_DENEGADO	A-01, N-07
E05	Sin encabezado X-Rol	(sin rol)	POST /emergencias	401 / 401	ROL_REQUERIDO	K-02
E06	Mascota indicada que no existe	recepcion	POST /emergencias	404 / 404	NO_ENCONTRADO	D-13
E07	Motivo en blanco	recepcion	POST /emergencias	422 / 422	MOTIVO_VACIO	N-04
E08	Falta el campo motivo	recepcion	POST /emergencias	400 / 400	SOLICITUD_INVALIDA	K-03
E09	Identificador repetido	recepcion	POST /emergencias	409 / 409	YA_EXISTE	K-03
E10	El veterinario recibió los dos avisos	veterinario	GET /avisos-veterinario	200 / 200	2 aviso(s)	NB-19, CR-8, N-08
E11	M1 puede recibir atención sin cita (es emergencia)	veterinario	GET /mascotas/M1/evaluaciones/atencion	200 / 200	cumple	R-06, N-09
E12	M2 no tiene cita confirmada ni emergencia	veterinario	GET /mascotas/M2/evaluaciones/atencion	200 / 200	no cumple: SIN_CITA_CONFIRMADA| SIN_EMERGENCIA_ACTIVA	R-06, N-09
E13	Recepción no inicia la atención	recepcion	POST /emergencias/EMER-E01/iniciar-atencion	403 / 403	ACCESO_DENEGADO	A-01, N-07
E14	El personal clínico inicia la atención	personal_clinico	POST /emergencias/EMER-E01/iniciar-atencion	200 / 200	EN_ATENCION	US20, N-02
E15	Iniciar otra vez la misma atención	personal_clinico	POST /emergencias/EMER-E01/iniciar-atencion	409 / 409	TRANSICION_INVALIDA	N-02, D-12
E16	Iniciar la atención de una que no existe	personal_clinico	POST /emergencias/EMER-NO/iniciar-atencion	404 / 404	NO_ENCONTRADO	D-13
E17	El responsable consulta la emergencia	responsable	GET /emergencias/EMER-E01	200 / 200	EN_ATENCION	N-07
E18	Listar las que esperan atención	personal_clinico	GET /emergencias?estado=IDENTIFICADA	200 / 200	1 resultado(s)	N-02
E19	Filtro con un estado que no existe	personal_clinico	GET /emergencias?estado=CERRADA	400 / 400	SOLICITUD_INVALIDA	K-03

E20	Los eventos EmergenciaIdentificada quedaron guardados	veterinario	GET /eventos	200 / 200	2 EmergenciaIdentificada	D-14, N-06

4.2.D VetCare 24: pruebas manuales en Swagger UI

Tabla 13. *4.2.D Pruebas manuales del recorrido CU-05.*

ID	Petición	Rol	Condición	Resultado obtenido	Tipo	Evidencia
SW-01	GET /mascotas/VET-C002-MASCOTA/evaluaciones/atencion	veterinario	Sin cita ni emergencia	200; cumple = false; SIN_CITA_CONFIRMADA y SIN_EMERGENCIA_ACTIVA	Error de regla	Fig. 61
SW-02	POST /emergencias {motivo: Atropellamiento, mascota_id: VET-C002-MASCOTA}	recepcion	Datos válidos	201; IDENTIFICADA; protocolo_activado; notificada	Éxito	Fig. 62
SW-03	GET /mascotas/VET-C002-MASCOTA/evaluaciones/atencion	veterinario	Tras registrar la emergencia	200; cumple = true; motivos vacíos	Éxito	Fig. 63
SW-04	GET /avisos-veterinario	veterinario	Tras la emergencia	200; lista con el aviso	Éxito	Fig. 64
SW-05	POST /emergencias {motivo: Intoxicación, datos_paciente: …}	recepcion	Sin mascota registrada	201; IDENTIFICADA	Límite	Fig. 65
SW-06	POST /emergencias	veterinario	Rol sin permiso	403 ACCESO_DENEGADO	Error	Fig. 66
SW-07	POST /emergencias/{id}/iniciar-atencion	personal_clinico	Emergencia IDENTIFICADA	200; EN_ATENCION	Éxito	Fig. 67
SW-08	POST /emergencias/{id}/iniciar-atencion (repetido)	personal_clinico	Ya en atención	409 TRANSICION_INVALIDA	Error	Fig. 68
SW-09	GET /eventos	veterinario	Eventos existentes	200; EmergenciaIdentificada	Éxito	Fig. 69
SW-10	GET con X-Rol mal escrito	(inválido)	Rol desconocido	400 ROL_DESCONOCIDO	Error	Fig. 70
SW-11	POST /emergencias {motivo en blanco}	recepcion	Motivo vacío	422 MOTIVO_VACIO	Error	Fig. 71

4.2.E VetCare 24: pruebas pytest de la Etapa 10

Las pruebas automáticas de emergencias se agrupan por tema. Las de servicios y las de la API se ejecutan dos veces, una con SQLite y otra con PostgreSQL, por lo que los totales de esos archivos son pares.

Tabla 14. *4.2.E Pruebas pytest de la Etapa 10 (nombre base y número de variantes).*

Archivo	Prueba (nombre base)	Variantes
test_emergencia.py	test_una_emergencia_sin_mascota_registrada_es_valida	1
test_emergencia.py	test_invariantes_al_reconstituir	11
test_emergencia.py	test_identidad_por_id	1
test_emergencia.py	test_identificar_activa_el_protocolo_y_deja_el_evento	1
test_emergencia.py	test_identificar_sin_mascota_deja_el_evento_sin_mascota	1
test_emergencia.py	test_un_motivo_vacio_no_registra_ni_emite_nada	1
test_emergencia.py	test_la_tabla_de_transiciones_es_la_unica_fuente_de_verdad	1
test_emergencia.py	test_iniciar_la_atencion_registra_la_hora	1
test_emergencia.py	test_iniciar_dos_veces_es_transicion_invalida_y_no_cambia_nada	1
test_emergencia.py	test_un_reloj_anterior_a_la_identificacion_se_rechaza_sin_cambios	1
test_emergencia.py	test_la_regla_es_un_or	1
test_emergencia.py	test_atencion_permitida	5
test_emergencia.py	test_el_contexto_ignora_citas_y_emergencias_de_otras_mascotas	1
test_emergencia.py	test_el_contexto_exige_mascota	1
test_servicios_emergencias.py	test_emergencia_sin_cita_se_registra	2
test_servicios_emergencias.py	test_emergencia_sin_mascota_registrada	2
test_servicios_emergencias.py	test_notifica_veterinario_tras_confirmar	2
test_servicios_emergencias.py	test_si_la_notificacion_falla_el_registro_se_conserva	2
test_servicios_emergencias.py	test_el_evento_se_guarda_y_se_publica	2
test_servicios_emergencias.py	test_mascota_inexistente_es_no_encontrado_y_no_guarda_ni_notifica	2
test_servicios_emergencias.py	test_motivo_vacio_no_guarda_nada	2
test_servicios_emergencias.py	test_id_repetido_es_ya_existe	2
test_servicios_emergencias.py	test_se_pueden_registrar_varias_emergencias_de_la_misma_mascota	2
test_servicios_emergencias.py	test_el_personal_clinico_inicia_la_atencion	2

test_servicios_emergencias.py	test_listar_con_filtros	2
test_servicios_emergencias.py	test_atencion_permitida_por_cita_confirmada_o_por_emergencia	2
test_servicios_emergencias.py	test_cada_rol_solo_hace_lo_suyo	2
test_servicios_emergencias.py	test_la_base_rechaza_datos_imposibles	2
test_servicios_emergencias.py	test_guardar_una_que_no_existe_es_no_encontrado	2
test_api_emergencias.py	test_los_20_escenarios_del_recorrido_coinciden	2
test_api_emergencias.py	test_los_ids_de_escenario_son_unicos	1
test_api_emergencias.py	test_registrar_responde_201_con_protocolo_y_notificacion	2
test_api_emergencias.py	test_si_el_aviso_falla_responde_201_pendiente_de_notificar	2
test_api_emergencias.py	test_los_mensajes_de_error_hablan_de_la_emergencia	2
test_api_emergencias.py	test_las_citas_conservan_sus_mensajes	2
test_api_emergencias.py	test_los_tipos_son_estrictos	2
test_api_emergencias.py	test_evaluacion_de_atencion_con_cita_confirmada	2
test_api_emergencias.py	test_listar_por_mascota_y_permisos_de_consulta	2
test_api_emergencias.py	test_los_eventos_de_emergencia_conviven_con_los_de_citas	2
test_api_emergencias.py	test_el_contrato_openapi_documenta_las_emergencias	1
test_u5_matriz.py	test_la_matriz_tiene_al_menos_nueve_casos_de_cada_tipo	1
test_u5_matriz.py	test_caso_de_la_matriz	9

4.3 Tabla de distribución

4.3.A Por tipo de caso

Tabla 15. *4.3.A Distribución de los casos por tipo.*

Tipo	Proyecto base (20)	Matriz del proyecto (11)	Total (31)	% del total
Éxito	7	3	10	32.3 %
Error	8	3	11	35.5 %
Límite	2	3	5	16.1 %
Integración	3	2	5	16.1 %
Total	20	11	31	100 %

4.3.B Por regla

Tabla 16. *4.3.B Distribución por regla (casos de la tabla 4.2.A, matriz del proyecto y escenarios por HTTP).*

Regla	Descripción corta	Casos base	Matriz y escenarios VetCare 24	Total
RN-01	Un pedido con total ≤ 0 no puede confirmarse	2	0	2
RN-02	Un cliente con deuda no puede confirmar un pedido	1	0	1
RN-03	Un pedido VIP recibe 15 % de descuento	2	0	2
RN-04	Un cliente regular no recibe descuento	2	0	2
RN-05	Un pedido ya confirmado no puede confirmarse de nuevo	1	0	1
RN-06	Si la pasarela aprueba, el servicio devuelve el importe	1	0	1
RN-07	Si la pasarela rechaza, el servicio lanza error	2	0	2
RN-08	El importe enviado a la pasarela incluye el descuento	1	0	1
RN-09	Commit hace persistente el pedido	2	0	2
RN-10	Rollback descarta el pedido pendiente	1	0	1
RN-11	Un pedido SHIPPED nunca se cancela, ni siquiera el administr…	1	0	1
RN-12	El administrador cancela pedidos que no están SHIPPED	1	0	1
RN-13	El propietario cancela solo si pasaron menos de 24 h	3	0	3
RN-14	Un usuario que no es propietario ni administrador no cancela	1	0	1
RN-15	Capacidad suficiente al confirmar una cita (NB-08 / D-12)	0	3	3
RN-16	Total mínimo de $1500 para el beneficio (D-05)	0	3	3
RN-17	La emergencia exige un motivo no vacío (N-04)	0	2	2
RN-18	La mascota de una emergencia es opcional (P3)	0	4	4
RN-19	Atención permitida si hay cita confirmada O emergencia activ…	0	2	2
RN-20	Solo Recepción registra emergencias (A-01, N-07)	0	3	3
RN-21	Solo el personal clínico inicia la atención (US20)	0	2	2
RN-22	No se puede iniciar dos veces la atención (N-02)	0	1	1
RN-23	Se avisa al veterinario; si el aviso falla, el registro se c…	0	2	2
RN-24	Una mascota indicada que no existe no se registra (D-13)	0	2	2
RN-25	Un identificador repetido no se acepta	0	1	1
RN-26	Sin rol responde 401 y con rol desconocido 400 (K-02 / K-03)	0	3	3
RN-27	Los eventos de dominio quedan guardados (D-14 / N-06)	0	2	2
RN-28	La base de datos rechaza datos imposibles (CHECK)	0	1	1
RN-29	La cobertura mínima es 80 %	0	0	0
RN-30	Cada push ejecuta Ruff, pruebas y compilación	0	0	0

Las reglas RN-29 y RN-30, de calidad, no cuentan con un caso individual: se comprueban mediante el umbral de cobertura y el flujo de integración (secciones 2.7 y 2.8).

4.3.C Escenarios por HTTP según el código de respuesta y el rol

Tabla 17. *4.3.C Distribución de los 20 escenarios por código HTTP.*

Código HTTP	Significado	Escenarios	Cantidad
200	OK (consulta o transición exitosa)	E10, E11, E12, E14, E17, E18, E20	7
201	Creado (emergencia registrada)	E01, E02	2
400	Solicitud mal formada o rol/filtro desconocido	E08, E19	2
401	Falta el rol	E05	1
403	Rol sin permiso	E03, E04, E13	3
404	No existe	E06, E16	2
409	Conflicto (transición inválida o id repetido)	E09, E15	2
422	Regla de negocio no cumplida	E07	1
Total			20

Tabla 18. *4.3.D Distribución de los 20 escenarios por rol.*

Rol	Escenarios	Cantidad
recepcion	E01, E02, E06, E07, E08, E09, E13	7
personal_clinico	E04, E14, E15, E16, E18, E19	6
veterinario	E03, E10, E11, E12, E20	5
(sin rol)	E05	1
responsable	E17	1
Total		20

4.3.E Pruebas del proyecto base por archivo

Tabla 19. *4.3.E Distribución por archivo del proyecto base.*

Archivo	Pruebas	Tipo predominante	Resultado
test_order.py	8	Unitaria / TDD	PASSED
test_repository.py	3	Integración	PASSED
test_security.py	6	Autorización	PASSED
test_service.py	3	Mock	PASSED
Total	20		20 PASSED

4.3.F Las 584 pruebas del proyecto por capa y por archivo

Tabla 20. *4.3.F Distribución de las 584 pruebas por capa.*

Capa	Archivos	Pruebas	% de 584
Dominio	12	219	37.5 %
Aplicación	6	106	18.2 %
Infraestructura	6	97	16.6 %
API	4	122	20.9 %
Oráculo, humo y matriz U5	3	40	6.8 %
Total	31	584	100 %

Tabla 21. *4.3.G Distribución de las 584 pruebas por archivo.*

Capa	Archivo	Pruebas
Dominio	test_dominio.py	36
Dominio	test_money.py	17

Dominio	test_ciclo_de_vida.py	23
Dominio	test_reglas_cita.py	25
Dominio	test_specification.py	20
Dominio	test_disponibilidad.py	14
Dominio	test_emergencia.py	28
Dominio	test_vacunacion.py	22
Dominio	test_reglas_vacunacion.py	6
Dominio	test_seguimiento_vacunacion.py	12
Dominio	test_eventos.py	8
Dominio	test_eventos_ciclo.py	8
Aplicación	test_servicios_de_aplicacion.py	32
Aplicación	test_servicios_consulta.py	12
Aplicación	test_servicios_vacunacion.py	18
Aplicación	test_servicios_emergencias.py	30
Aplicación	test_control_de_acceso.py	8
Aplicación	test_despachador.py	6
Infraestructura	test_csv_loader.py	14
Infraestructura	test_adaptador_paquete.py	14
Infraestructura	test_base_de_datos.py	15
Infraestructura	test_repositorios_sql.py	25
Infraestructura	test_repositorios_vacunacion.py	16
Infraestructura	test_sembrado.py	13
API	test_api.py	63
API	test_api_errores.py	31
API	test_api_paquete.py	8
API	test_api_emergencias.py	20
Oráculo, humo y matriz U5	test_oraculo.py	26
Oráculo, humo y matriz U5	test_smoke.py	4
Oráculo, humo y matriz U5	test_u5_matriz.py	10

4.4 Tabla de evidencias

En esta tabla se relaciona cada caso con su prueba de pytest (o su petición HTTP), el resultado observado y la figura donde puede verse.

Tabla 22. *4.4.A Relación entre casos, pruebas y figuras.*

Caso(s)	Prueba pytest o petición	Resultado observado	Evidencia
C-01 a C-05, C-08	test_order.py (5 originales + cannot_confirm_twice)	7 passed → 8 con la prueba TDD	Fig. 5, Fig. 6
C-06, C-07	test_vip_discount_200 y test_regular_customer_200_no_discount	PASSED (7 passed in 0.13s)	Fig. 5
C-08	test_cannot_confirm_twice	RED: DID NOT RAISE; GREEN: 8 passed	Fig. 6, Fig. 8
C-09 a C-11	test_repository.py	3 passed in 1.07s	Fig. 9, Fig. 10
C-12 a C-14	test_service.py	3 passed in 0.09s; estado CONFIRMED tras rechazo	Fig. 11, Fig. 12
C-15 a C-20	test_security.py	6 passed in 0.05s	Fig. 13
Ruff (calidad)	python -m ruff check src tests	13 errores → All checks passed	Fig. 14, Fig. 15
Cobertura base (RN-29)	pytest --cov=src --cov-fail-under=80	20 passed; 100.00 %	Fig. 16, Fig. 17, Fig. 18
CI base (RN-30)	CI Business Logic #1 a #4	roja, verde, roja, verde	Fig. 19, Fig. 20, Fig. 21, Fig. 22, Fig. 23
CP-01 a CP-03	test_u5_matriz.py::test_caso_de_la_matriz[CP-01..03]	PASSED dentro de 584 passed	Fig. 82
CP-04 a CP-06	test_u5_matriz.py::test_caso_de_la_matriz[CP-04..06]	PASSED dentro de 584 passed	Fig. 82
CP-07 a CP-09	test_u5_matriz.py::test_caso_de_la_matriz[CP-07..09]	PASSED; CP-08 coincide con 422 MOTIVO_VACIO manual	Fig. 82, Fig. 71
CP-10	test_base_de_datos.py::test_rollback_deshace_y_commit_conserva	PASSED (SQLite y PostgreSQL)	Fig. 82, Fig. 53
CP-11	test_servicios_emergencias.py::test_si_la_notificacion_falla_el_registro_se_conserva	PASSED [sqlite] y [postgres]	Fig. 82
E01, E02	POST /emergencias como recepcion	201 IDENTIFICADA	Fig. 62, Fig. 65, Fig. 74
E03, E04, E13	Roles sin permiso	403 ACCESO_DENEGADO	Fig. 66, Fig. 74

E05, E19	Sin rol / filtro inexistente	401 / 400	Fig. 70, Fig. 74
E07	Motivo en blanco	422 MOTIVO_VACIO	Fig. 71, Fig. 74
E10	GET /avisos-veterinario	200 con los avisos	Fig. 64, Fig. 74
E11, E12	Evaluación de atención (R-06)	cumple = true / false	Fig. 61, Fig. 63, Fig. 74
E14, E15	iniciar-atencion	200 EN_ATENCION / 409	Fig. 67, Fig. 68, Fig. 74
E20	GET /eventos	200 con EmergenciaIdentificada	Fig. 69, Fig. 74
Persistencia PostgreSQL (RN-28)	Consultas psql de emergencias, eventos y esquema	3 filas, 3 eventos y CHECK	Fig. 72
120/120 y 20/20	python -m tests.evidencia_api	Coinciden con el oráculo	Fig. 48, Fig. 73
Suite del proyecto	docker compose --profile tests run --rm tests	584 passed; 98.81 %	Fig. 82
CI del proyecto (RN-30)	ci.yml lint → test → docker	Verde; 2 artefactos	Fig. 75, Fig. 76, Fig. 77, Fig. 78, Fig. 79

4.4.B Trazabilidad de regla a evidencia

Tabla 23. *4.4.B Trazabilidad de reglas a casos, pruebas y evidencias.*

Regla(s)	Componente	Casos	Prueba automática	Evidencia
RN-01, RN-02, RN-03, RN-04	Order.confirm	C-01 a C-07	test_order.py	Fig. 5
RN-05	Order.confirm (TDD)	C-08	test_cannot_confirm_twice	Fig. 6, Fig. 8
RN-06, RN-07, RN-08	OrderService.checkout	C-12 a C-14	test_service.py	Fig. 11, Fig. 12
RN-09, RN-10	SqlOrderRepository	C-09 a C-11	test_repository.py	Fig. 10
RN-11 a RN-14	can_cancel	C-15 a C-20	test_security.py	Fig. 13
RN-15	CapacidadSuficiente	CP-01 a CP-03	test_caso_de_la_matriz	Fig. 82
RN-16	TotalMinimo	CP-04 a CP-06	test_caso_de_la_matriz	Fig. 82
RN-17, RN-18	Emergencia	CP-07 a CP-09; E01, E02, E07; SW-02, SW-05, SW-11	test_emergencia.py y test_caso_de_la_matriz	Fig. 62, Fig. 65, Fig. 71
RN-19	reglas_emergencia (R-06)	E11, E12; SW-01, SW-03	test_atencion_permitida[…]	Fig. 61, Fig. 63
RN-20, RN-21	control_de_acceso	E03, E04, E13, E14; SW-06, SW-07	test_cada_rol_solo_hace_lo_suyo	Fig. 66, Fig. 67
RN-22	Transiciones de Emergencia	E15; SW-08	test_iniciar_dos_veces_es_transicion_invalida…	Fig. 68
RN-23	Notificación al veterinario	E10; CP-11; SW-04	test_si_la_notificacion_falla…	Fig. 64, Fig. 82
RN-24, RN-25	Servicios de emergencias	E06, E09, E16	test_mascota_inexistente… y test_id_repetido…	Fig. 74
RN-26	errores_http	E05, E08, E19; SW-10	test_los_mensajes_de_error_hablan_de_la_emergencia	Fig. 70, Fig. 74
RN-27	Eventos de dominio	E18, E20; SW-09	test_el_evento_se_guarda_y_se_publica	Fig. 69, Fig. 72
RN-28	Esquema con CHECK	CP-10	test_la_base_rechaza_datos_imposibles	Fig. 72
RN-29	Umbral de cobertura	—	--cov-fail-under=80	Fig. 16, Fig. 82
RN-30	Pipeline de CI	—	ci.yml	Fig. 19, Fig. 21, Fig. 22, Fig. 23, Fig. 75

4.4.C Criterios de aceptación de la matriz

Tabla 24. *4.4.C Criterios de aceptación.*

Criterio	Cómo se comprueba	Resultado
Todos los casos de la matriz tienen entrada y resultado esperado calculados a mano	Revisión del CSV y de la tabla 4.2	Cumple

Hay al menos nueve casos con tres de cada tipo	test_la_matriz_tiene_al_menos_nueve_casos_de_cada_tipo	PASSED
Cada caso se ejecuta con pytest	test_caso_de_la_matriz[CP-01..CP-09] y pruebas existentes CP-10 y CP-11	PASSED
La matriz incluye persistencia	CP-10: rollback deja 0 filas y commit conserva 1	PASSED
La matriz incluye servicio externo simulado	CP-11: notificador simulado que falla	PASSED
La cobertura del código pertinente supera el 80 %	Reporte de cobertura (reglas al 100 %; total 98.81 %)	Cumple
La suite se ejecuta de forma automática	Pipeline ci.yml en GitHub	Verde